
Bioinformática aplicada a enfermedades
emergentes y reemergentes en España

Bioinformatics applied to emerging and
reemerging diseases in Spain



Trabajo de Fin de Grado
Curso 2025–2026

Autor

Carlos Mayorga Santiago

Director

José Luis Ayala Rodrigo

Grado en Ingeniería de Software

Facultad de Informática

Universidad Complutense de Madrid

Bioinformática aplicada a enfermedades emergentes y reemergentes en España

Bioinformatics applied to emerging and reemerging diseases in Spain

Trabajo de Fin de Grado en **Ingeniería de Software**

Autor

Carlos Mayorga Santiago

Director

José Luis Ayala Rodrigo

Convocatoria: *septiembre 2026*

Grado en **Ingeniería de Software**

Facultad de Informática

Universidad Complutense de Madrid

7 de septiembre de 2026

Dedicatoria

*A todos los médicos e investigadores del sector
sanitario por cuidarnos y descubrir nuevos
tratamientos que nos permitan vivir mejor y
más años.*

Agradecimientos

Quiero agradecer a mi familia su apoyo, no solo durante el desarrollo de este Trabajo de Fin de Grado, sino también a lo largo de toda la carrera; a mis amigos, por revisar la estética de la página web y ofrecerme consejos para mejorarla; y a mi tutor, por supervisar mi trabajo y ayudarme a mejorar el proyecto.

Resumen

Bioinformática aplicada a enfermedades emergentes y reemergentes en España

En este proyecto hemos estudiado los casos epidemiológicos de diferentes enfermedades emergentes y reemergentes en España. La investigación se divide por comunidades autónomas y analiza su evolución a lo largo del tiempo. Entre las personas afectadas por estas enfermedades, hemos analizado qué género y qué rango de edad son los más perjudicados.

Disponemos de una página web interactiva en la que se pueden consultar visualmente los diferentes escenarios. A su vez, cuenta con toda la información crucial sobre las enfermedades que se encuentran en el atlas, con el objetivo de comprender mejor la situación que afecta a nuestro país y poder protegernos mejor. Podrás contactar y enviar sugerencias a través de la página web.

Mediante una API, se seleccionan los datos almacenados en la base de datos para mostrarlos en el portal web de forma semiautomática.

Palabras clave

- Enfermedades emergentes
- Enfermedades reemergentes
- Mapa epidemiológico
- Estudio espaciotemporal autonómico
- Página web
- Base de datos SQL
- R Commander
- FastAPI
- Leaflet
- JSON y GeoJSON

Summary

Bioinformatics applied to emerging and reemerging diseases in Spain

This project examines epidemiological cases of several emerging and re-emerging diseases in Spain. The study is organized by autonomous communities and analyzes how these diseases evolve over time. It also identifies the most affected gender and age group.

The interactive website allows users to explore different scenarios visually. It also provides essential information about the diseases included in the atlas, helping users to understand the epidemiological situation in Spain and take appropriate preventive measures. The website also includes a contact form through which users can submit suggestions.

Through an API, data from the database are selected and displayed semi-automatically on the web portal.

Keywords

- Emerging diseases
- Reemerging diseases
- Epidemiological map
- Autonomous spatiotemporal study
- Website
- SQL database
- R Commander
- FastAPI
- Leaflet
- JSON and GeoJSON

Índice

1. Introducción	1
1.1. Motivación	2
1.2. Planteamiento	2
1.3. Objetivos	3
1.4. Plan de trabajo	3
1.5. Antecedentes tecnológicos	4
Introduction	7
1.6. Motivation	7
1.7. Approach	8
1.8. Objectives	8
1.9. Work Plan	9
1.10. Technological Background	10
2. Especificaciones	13
2.1. Requisitos del sistema	13
2.1.1. Requisitos funcionales	13
2.1.2. Requisitos no funcionales	14
2.2. Estructura	15
2.2.1. Cadena de procesamiento y tipos de datos	21
2.3. Arquitectura del software	22
2.3.1. Capa de presentación	22
2.3.2. Capa de servicios y API	23
2.3.3. Capa de acceso a datos	23
2.3.4. Capa de persistencia	24
2.3.5. ETL y preprocesamiento de datos	24
2.3.6. Dependencias entre procesos y archivos	25
2.4. Modelo de datos	26
2.4.1. Registro epidemiológico	26
2.4.2. Representación JSON del registro epidemiológico	26
2.4.3. Mensaje de contacto	28
2.4.4. Estructuras de datos utilizadas en JavaScript	28
2.5. Organización de los archivos del proyecto	30
2.5.1. Backend y archivos de datos	30
2.5.2. Frontend	31
3. Tecnología y lenguajes empleados	33
3.1. Desarrollo back-end	33
3.1.1. CSV	33

3.1.2.	R Commander	33
3.1.3.	MySQL	35
3.1.4.	FastAPI	35
3.2.	Desarrollo front-end	47
3.2.1.	HTML, CSS y Bootstrap	47
3.2.2.	JavaScript	48
3.2.3.	Leaflet	50
3.2.4.	JSON y GeoJSON	51
4.	Funcionamiento	55
4.1.	Página de inicio	55
4.2.	Información detallada de las enfermedades	57
4.3.	Elementos comunes y opciones de accesibilidad	57
4.4.	Mapa epidemiológico	58
4.4.1.	Selección de filtros y estado inicial	59
4.4.2.	Representación de los casos notificados	60
4.4.3.	Representación del género predominante	60
4.4.4.	Representación del rango de edad predominante	61
4.4.5.	Descarga de datos	62
4.5.	Contacto y sugerencias	63
4.5.1.	Formulario de contacto	63
4.6.	Ampliación de la aplicación	65
4.6.1.	Incorporación de una nueva enfermedad	65
4.6.2.	Incorporación de un nuevo filtro	65
4.7.	Pruebas	66
4.7.1.	Pruebas de los requisitos funcionales	66
4.7.2.	Pruebas de los requisitos no funcionales	67
5.	Análisis	69
5.1.	Análisis del dengue	69
5.1.1.	Evolución de los casos notificados	69
5.1.2.	Distribución por género	70
5.1.3.	Distribución por edad	71
5.2.	Análisis de la tuberculosis	72
5.2.1.	Evolución de los casos notificados	72
5.2.2.	Distribución por género	73
5.2.3.	Distribución por edad	74
5.3.	Análisis de la fiebre del Nilo occidental	75
5.3.1.	Evolución de los casos notificados	75
5.3.2.	Distribución por género	76
5.3.3.	Distribución por edad	76
5.4.	Análisis de la fiebre hemorrágica de Crimea-Congo	77
5.4.1.	Evolución de los casos notificados	77
5.4.2.	Distribución por género	78

5.4.3. Distribución por edad	78
5.5. Análisis del chikungunya	79
5.5.1. Evolución de los casos notificados	79
5.5.2. Distribución por género	80
5.5.3. Distribución por edad	80
6. Conclusiones y trabajo futuro	83
6.1. Conclusiones	83
6.2. Trabajo futuro	84
Conclusions and Future Work	87
6.3. Conclusions	87
6.4. Future Work	87
Bibliografía	91
A. Manual de instalación	95
A.1. Ubicación y estructura del proyecto	95
A.2. Pasos de instalación	96
A.3. Comprobación de la instalación	102

Índice de figuras

1.1.	Interfaz del Observatorio Mundial de la Salud de la OMS.	5
1.2.	Atlas epidemiológico del ECDC.	5
1.3.	Listado de temas y bases de datos disponibles en CDC WONDER. . .	6
1.4.	Interface of the WHO Global Health Observatory.	10
1.5.	Map, filters, charts, and tables in the ECDC epidemiological atlas. .	11
1.6.	List of topics and databases available in CDC WONDER.	11
2.1.	Diagrama de flujo de la arquitectura de la aplicación.	15
2.2.	Diagrama de flujo para la obtención de información de los archivos JSON.	16
2.3.	Diagrama de flujo detallado de la actualización del mapa epidemiológico.	17
2.4.	Diagrama de flujo del funcionamiento de los filtros del mapa.	18
2.5.	Diagrama de flujo de la generación de las tarjetas de enfermedades. .	19
2.6.	Diagrama de flujo del envío y almacenamiento de los mensajes de contacto.	20
2.7.	Cadena de procesamiento total de los datos del proyecto.	21
2.8.	Ejemplo de representación JSON de un registro epidemiológico. . . .	27
2.9.	Ejemplo de un registro JSON con datos de género y rango de edad para el conjunto de España.	27
2.10.	Organización de las carpetas backend y datos	31
2.11.	Organización de los archivos de la carpeta frontend	32
3.1.	Ejecución y respuesta del endpoint raíz de la API.	38
3.2.	Introducción de los parámetros del endpoint /buscar	39
3.3.	Respuesta y códigos HTTP documentados para el endpoint /buscar . .	39
3.4.	Respuesta del endpoint que devuelve la lista de enfermedades.	40
3.5.	Respuesta del endpoint que devuelve los años disponibles.	41
3.6.	Consulta de la información detallada de una enfermedad.	42
3.7.	Estructura de la respuesta del endpoint de información de enfermedades.	42
3.8.	Introducción de los parámetros del endpoint de distribución por género.	43
3.9.	Respuesta y posibles códigos del endpoint de distribución por género.	43
3.10.	Consulta y respuesta del endpoint de distribución por edad.	44
3.11.	Códigos y estructura de respuesta del endpoint de distribución por edad.	45
3.12.	Ejecución del endpoint que devuelve todos los registros epidemiológicos.	45
3.13.	Cuerpo JSON utilizado para enviar un mensaje mediante el endpoint /contacto	46
3.14.	Respuestas documentadas para el endpoint de contacto.	46
3.15.	Confirmación de la creación y almacenamiento de un mensaje de con- tacto.	47

3.16. Estructura del archivo JSON correspondiente a la fiebre del Nilo occidental.	52
3.17. Fragmento de la estructura del archivo GeoJSON utilizado por el mapa.	53
4.1. Cabecera y portada de la página de inicio.	55
4.2. Contenido introductorio sobre las enfermedades emergentes y reemergentes.	56
4.3. Listado de tarjetas de las enfermedades estudiadas.	56
4.4. Página de información detallada de una enfermedad.	57
4.5. Menú lateral, opciones de accesibilidad y pie de página.	58
4.6. Página de inicio con el modo en blanco y negro y la interfaz ampliada.	58
4.7. Introducción y explicación del funcionamiento del mapa epidemiológico.	59
4.8. Filtros disponibles y estado inicial del mapa epidemiológico.	59
4.9. Representación de los casos notificados de dengue durante el año 2023.	60
4.10. Representación del género predominante en los casos de chikungunya durante el año 2023.	61
4.11. Representación del rango de edad predominante en los casos de tuberculosis durante el año 2024.	61
4.12. Opciones disponibles para la descarga de datos en formato CSV.	62
4.13. Portada y datos disponibles en la página de contacto.	63
4.14. Formulario para el envío de mensajes y sugerencias.	64
4.15. Mensaje de confirmación tras enviar correctamente una sugerencia.	64
4.16. Mensaje mostrado cuando se produce un error al enviar una sugerencia.	64
4.17. Mensaje recibido y almacenado en la tabla <code>mensajes</code> de la base de datos.	65
5.1. Distribución de los casos notificados de dengue durante el año 2024.	70
5.2. Rango de edad predominante en los casos de dengue durante el año 2022.	71
5.3. Distribución de los casos notificados de tuberculosis durante el año 2024.	72
5.4. Género predominante en los casos de tuberculosis durante el año 2023.	73
5.5. Rango de edad predominante en los casos de tuberculosis durante el año 2022.	74
5.6. Distribución de los casos notificados de fiebre del Nilo occidental durante el año 2024.	76
5.7. Distribución de los casos de fiebre hemorrágica de Crimea-Congo incluidos en el mapa durante el año 2024.	78
5.8. Distribución de los casos notificados de chikungunya durante el año 2024.	79
5.9. Rango de edad predominante en los casos de chikungunya durante el año 2024.	81
A.1. Estructura de carpetas y archivos principales del proyecto.	96
A.2. Ejecución correcta del script <code>actualizar_datos.r</code> en R.	97

A.3. Formulario de inserción manual de un registro en la tabla enfermedades mediante phpMyAdmin.	99
A.4. Registros epidemiológicos almacenados en la tabla enfermedades . . .	99
A.5. Servicios Apache y MySQL iniciados en el panel de control de XAMPP.	100
A.6. Ejecución del servidor de desarrollo mediante Uvicorn.	101
A.7. Apertura del frontend mediante la extensión Live Server de Visual Studio Code.	101
A.8. Página principal de la aplicación abierta en el navegador.	102

Índice de tablas

2.1.	Requisitos funcionales del sistema.	13
2.2.	Requisitos no funcionales del sistema.	14
2.3.	Clasificación de los datos según su procesamiento y momento de uso.	22
2.4.	Campos del modelo <code>RegistroEpidemiologico</code>	26
2.5.	Campos del modelo <code>Mensaje</code>	28
2.6.	Estructuras de datos utilizadas por los principales archivos JavaScript.	28
2.7.	Estructuras principales utilizadas en <code>mapa.js</code>	29
3.1.	Endpoints disponibles en la API.	36
4.1.	Tabla de trazabilidad de requisitos funcionales.	66
4.2.	Tabla de trazabilidad de requisitos no funcionales.	67
A.1.	Versiones utilizadas durante las pruebas de la aplicación.	95
A.2.	Configuración de la conexión con la base de datos.	97

Introducción

“Donde no hay lucha no hay fuerza”
— Oprah Winfrey

Tenemos la creencia de que estamos a salvo de enfermedades que afectan a otros países o que fueron peligrosas en el pasado, pero no es cierto. En la actualidad, existen cientos de ecosistemas y millones de especies de seres vivos a lo largo del planeta que no paran de evolucionar. Estos organismos pueden estar más controlados en sus hábitats, pero, en el momento en que llegan a un lugar que no está preparado, pueden ocasionar una situación muy peligrosa. Los viajes entre continentes y países se producen todos los días y nunca se sabe a ciencia cierta todo lo que transita en esas travesías.

Un ejemplo que todos hemos vivido recientemente y cuya gravedad conocemos a grandes rasgos fue el COVID-19, una enfermedad que se transmitió a los humanos a través de otros animales desde una pequeña ciudad de China. En cuestión de un mes, esta enfermedad se propagó de país en país, de ciudad en ciudad y de persona en persona hasta afectar a cerca de 780 millones de habitantes. Cuando se empezó a tener conocimiento de este virus, se intentaron adoptar medidas de precaución, pero no fueron suficientes.

Este suceso puede volver a ocurrir con una nueva enfermedad, ya sea a nivel mundial o a menor escala, por lo que hay que estar atentos a cualquier factor que pueda producir un perjuicio general.

1.1. Motivación

Este proyecto se ha realizado con el fin de concienciar a cualquier persona preocupada por la situación epidemiológica de nuestro país acerca de las últimas enfermedades introducidas y de aquellas que vuelven a ser peligrosas, para que pueda permanecer alerta [21, 32]. Hoy en día es mucho más fácil obtener información de interés gracias a Internet, pero no siempre está al alcance de todos ni se entiende fácilmente lo que quiere decir.

La página web que hemos desarrollado sirve para recopilar información sobre las enfermedades que llegan a España desde otros lugares y pueden extenderse rápidamente, así como sobre las afecciones que vuelven a surgir. De este modo, cualquier persona, sin necesidad de conocimientos previos, puede entender y visualizar esta información con facilidad. Los usuarios podrán conocer en qué comunidades autónomas hay más casos de una enfermedad concreta, qué género ha sido el más afectado y en qué rango de edad se han producido más casos. También podrán observar la evolución de estas enfermedades a lo largo de los años y contemplar su crecimiento o decrecimiento. La web cuenta con un listado de todas las enfermedades analizadas que contiene sus características básicas, síntomas, tratamientos y medidas de prevención, entre otros datos, con el objetivo de minimizar los riesgos en la medida de lo posible.

1.2. Planteamiento

El primer paso fue elaborar una lista de enfermedades emergentes y reemergentes que hubieran ocasionado contratiempos en España en los últimos años. Como nuestro trabajo se encuentra en el ámbito médico, buscar información y datos de pacientes se convierte en un camino arduo, puesto que los registros suelen ser privados y los detalles no se suelen hacer públicos.

Investigamos más de 15 enfermedades, de las cuales solo sobre 5 obtuvimos datos suficientes para poder realizar un trabajo exhaustivo. La idea inicial era realizar la investigación y elaborar el mapa por provincias de España, y no por comunidades autónomas. No obstante, en los documentos que habíamos encontrado sobre estas afecciones, la información estaba dividida por comunidades autónomas, por lo que decidimos representarla de esa manera. Otra de las posibilidades que teníamos era representar en el proyecto y en el mapa todos los países de Europa, pero solo en unos pocos había datos públicos y, en algunos casos, estaban incompletos. A su vez, si decidíamos hacerlo sobre Europa, nos alejábamos bastante del planteamiento inicial de informar a los ciudadanos sobre las enfermedades que amenazan a nuestro país.

Una vez que teníamos datos suficientes y un planteamiento sólido, podíamos empezar a planificar el resto del trabajo e iniciar el desarrollo.

1.3. Objetivos

Los objetivos del trabajo de fin de grado son los siguientes:

- Recopilar información sobre enfermedades emergentes y reemergentes en los últimos años.
- Crear un archivo CSV que contenga los casos epidemiológicos por comunidades autónomas, género y rango de edad.
- Crear una base de datos a partir de archivos CSV.
- Realizar un estudio espaciotemporal autonómico.
- Desarrollar una API con FastAPI que permita obtener los datos de la base de datos desde el backend.
- Implementar una página web (frontend) con información relevante y un atlas interactivo.
- Desarrollar un formulario de sugerencias.
- Obtener las características básicas de cada enfermedad con el fin de informar al usuario.
- Añadir opciones de accesibilidad
- Analizar los datos observados y proponer hipótesis de estudio.

1.4. Plan de trabajo

Para llevar a cabo este proyecto, el trabajo se dividió en distintas fases con el fin de garantizar su correcto desarrollo. Es necesario completar cada fase antes de pasar a la siguiente. A continuación, se describen las distintas fases:

Fase 1: Investigación y recopilación de datos

- Obtención de datos sobre enfermedades emergentes y reemergentes más importantes en España en los últimos años.
- Exploración y selección de lenguajes de programación para bases de datos y páginas web, frameworks para la construcción de API compatibles con la web y SQL, y frameworks adaptados a HTML y JavaScript.
- Extraer información básica y crucial de las enfermedades seleccionadas.

Fase 2: Diseño de la arquitectura del proyecto y del backend

- Creación de archivos CSV con los datos obtenidos de las enfermedades más relevantes.

- Desarrollo de fichero .r que se conecte al INE y actualice la tasa de notificación automáticamente del CSV.
- Modelado de la base de datos a partir de los archivos CSV.
- Desarrollo de una API que obtenga los datos de la base de datos mediante puntos de acceso.

Fase 3: Diseño y implementación del frontend de la página web

- Desarrollo de toda la parte visual e interactiva de la página web.
- Integración de la base de datos en el frontend mediante la API y JavaScript.
- Construcción de un mapa interactivo personalizado.
- Adición de un formulario de sugerencias.
- Adición de opciones de accesibilidad.

Fase 4: Detección y corrección de errores

- Contrastación de los datos obtenidos en la interfaz con los almacenados en la base de datos.
- Comprobación de que los elementos visuales del frontend sean correctos y estén adaptados a todos los tamaños de pantalla.
- Verificación de la accesibilidad.

Fase 5: Análisis de datos

- Análisis de los datos obtenidos.
- Creación de hipótesis tras el estudio.

1.5. Antecedentes tecnológicos

Existen numerosos portales web que ofrecen información sobre enfermedades y otros temas de salud en todo el mundo. Muchos pertenecen a organizaciones sanitarias y reúnen una gran cantidad de datos. A continuación, se presentan tres ejemplos relacionados a elementos que proporciona nuestro proyecto:

Uno de ellos es el Observatorio Mundial de la Salud de la Organización Mundial de la Salud (OMS), disponible en <https://www.who.int/data/gho>. Este portal reúne información sobre enfermedades, salud mental y otros temas sanitarios de numerosos países. Su amplitud lo convierte en un recurso útil para consultar y complementar datos. No obstante, la gran cantidad de secciones y opciones puede dificultar la búsqueda de información concreta para un usuario poco familiarizado con la página.



Figura 1.1: Interfaz del Observatorio Mundial de la Salud de la OMS.

También existen atlas epidemiológicos con funciones similares a las de nuestro proyecto. En particular, el atlas del Centro Europeo para la Prevención y el Control de las Enfermedades (ECDC), disponible en <https://atlas.ecdc.europa.eu/public/>, se ha utilizado como referencia para algunas de las funcionalidades implementadas. Permite consultar distintas enfermedades y aplicar filtros sobre un mapa de Europa, además de visualizar los datos mediante gráficos y tablas. Aunque se trata de una herramienta muy completa, durante su uso la distribución de los elementos nos resultó poco intuitiva y las opciones de accesibilidad, poco visibles. Esta valoración motivó la búsqueda de una interfaz más sencilla para nuestro atlas.

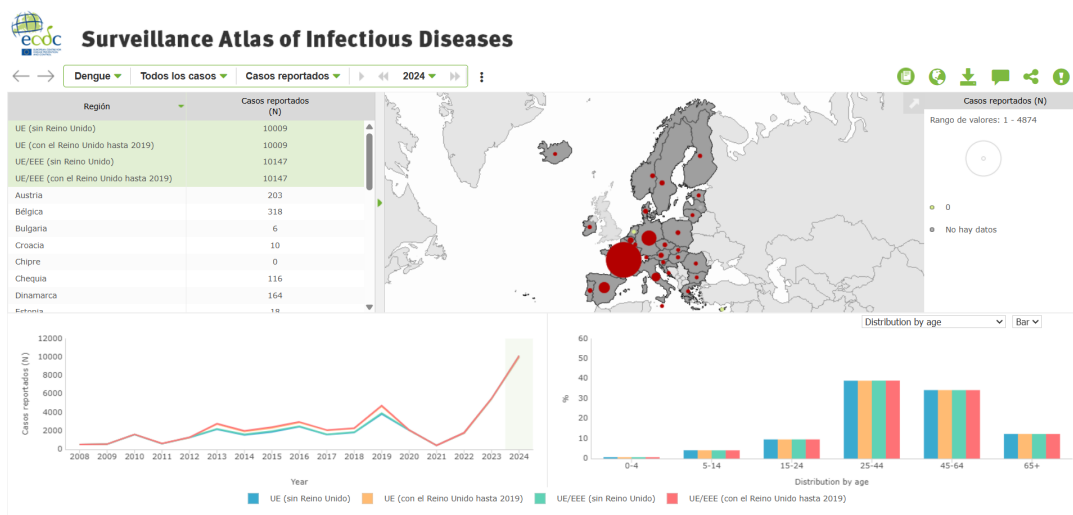


Figura 1.2: Atlas epidemiológico del ECDC.

Otro ejemplo es CDC WONDER, disponible en <https://wonder.cdc.gov/>, que permite acceder a bases de datos sanitarios. Este tipo de portal resulta útil para consultar información específica y descargar datos que puedan utilizarse en otros proyectos. Sin embargo, la interfaz consultada se centra en listados de temas y enlaces a las bases de datos, en lugar de ofrecer una presentación visual orientada a explicar

su contenido al público general. Esta organización puede hacer menos intuitiva la búsqueda para quienes no conocen previamente la información disponible.



Figura 1.3: Listado de temas y bases de datos disponibles en CDC WONDER.

En nuestro proyecto hemos buscado combinar los aspectos positivos de estos portales entre otros muchos que hemos transitado. La consulta de información sanitaria, la representación de datos mediante un mapa interactivo y la posibilidad de descargarlos. Todo ello se integra en una página web sencilla, visual y con opciones de accesibilidad, pensada para facilitar su uso a personas sin conocimientos previos y que un mayor grupo de personas puedan usarla sin complicaciones.

Introduction

“Where there is no struggle, there is no strength”
— Oprah Winfrey

We tend to believe that we are safe from diseases that affect other countries or that were dangerous in the past, but this is not the case. Today, hundreds of ecosystems and millions of species exist across the planet and are constantly evolving. These organisms may remain relatively contained within their natural habitats, but they can create a dangerous situation when they reach an area that is unprepared for them. People travel between countries and continents every day, and it is impossible to know with certainty everything that may be transported during these journeys.

A recent example whose severity we have all witnessed is COVID-19, a disease transmitted to humans through other animals from a small city in China. Within a month, it spread from country to country, city to city, and person to person, eventually affecting nearly 780 million people. Once the virus became known, precautionary measures were introduced, but they were not sufficient.

A similar event could occur again with a new disease, either globally or on a smaller scale. It is therefore important to remain alert to any factor that could cause widespread harm.

1.6. Motivation

This project was undertaken to raise awareness among people concerned about the epidemiological situation in Spain, particularly regarding recently introduced diseases and those that are becoming dangerous again [21, 32]. Although the Internet now makes it much easier to obtain information of interest, this information is not always accessible to everyone or easy to understand.

The website developed in this project brings together information about diseases that reach Spain from other countries and may spread rapidly, as well as diseases that are re-emerging. It enables users without prior knowledge to understand and visualize this information easily. Users can identify the autonomous communities with the highest number of cases of a particular disease, the most affected gender, and the age group with the most cases. They can also observe how these diseases evolve over time and whether their incidence increases or decreases. The website includes a list of all the diseases analyzed, together with their main characteristics, symptoms, treatments, and preventive measures, among other information, with the aim of minimizing risks as far as possible.

1.7. Approach

The first step was to compile a list of emerging and re-emerging diseases that had caused problems in Spain in recent years.

Because this project belongs to the medical field, obtaining information and patient data was particularly difficult, as records are generally private and detailed data are rarely made public. More than 15 diseases were investigated, but sufficient information for an exhaustive study was found for only five of them. The original intention was to conduct the study and represent the data by province rather than by autonomous communities. However, the available documents organized the information by autonomous communities, so this territorial division was used instead. Another possibility was to include every European country, but public data were available for only a small number of them and were sometimes incomplete. In addition, a European scope would have moved the project away from its original aim of informing citizens about the diseases that threaten Spain.

Once sufficient data had been collected and a clear approach established, the remaining work could be planned and development could begin.

1.8. Objectives

The objectives of this bachelor's thesis are as follows:

- Compile information about emerging and re-emerging diseases in recent years.
- Create a CSV file containing epidemiological cases by autonomous community, gender, and age group.
- Create a database from CSV files.
- Conduct a spatiotemporal analysis by autonomous community.
- Develop a FastAPI API that provides access to the database from the backend.
- Implement a website frontend containing relevant information and an interactive atlas.
- Develop a suggestion form.
- Collect the main characteristics of each disease in order to inform users.
- add accessibility options.
- Analyze the observed data and propose research hypotheses.

1.9. Work Plan

The project was divided into several phases to ensure its correct development. Each phase was completed before moving on to the next. These phases are described below:

Phase 1: Research and data collection

- Collection of data on the most significant emerging and reemerging diseases in Spain in recent years.
- Assessment and selection of programming languages for databases and websites, frameworks for building APIs compatible with web technologies and SQL, and frameworks suitable for HTML and JavaScript.
- Collection of essential information about the selected diseases.

Phase 2: Project architecture and backend design

- Creation of CSV files containing the data collected for the most relevant diseases.
- Development of an R script that connects to the INE and automatically updates the notification rates in the CSV file.
- Database modeling based on the CSV files.
- Development of an API that retrieves data from the database through endpoints.

Phase 3: Design and implementation of the website frontend

- Development of the visual and interactive components of the website.
- Integration of the database with the frontend through the API and JavaScript.
- Construction of a customized interactive map.
- Addition of a suggestion form.
- Addition of accessibility options.

Phase 4: Error detection and correction

- Comparison of the data displayed in the interface with the records stored in the database.
- Verification that the visual elements of the frontend are correct and adapt to different screen sizes.
- Verification of the accessibility features.

Phase 5: Data analysis

- Analysis of the collected data.
- Formulation of hypotheses based on the study.

1.10. Technological Background

Numerous web portals provide information on diseases and other health topics worldwide. Many belong to health organizations and bring together large amounts of data. The following three examples offer features related to those provided by our project:

One example is the Global Health Observatory of the World Health Organization (WHO), available at <https://www.who.int/data/gho>. This portal brings together information on diseases, mental health, and other health topics from numerous countries. Its broad coverage makes it a useful resource for consulting and supplementing data. However, the large number of sections and options can make it difficult for users unfamiliar with the website to find specific information.



Figure 1.4: Interface of the WHO Global Health Observatory.

There are also epidemiological atlases with features similar to those of our project. In particular, the atlas of the European Centre for Disease Prevention and Control (ECDC), available at <https://atlas.ecdc.europa.eu/public/>, was used as a reference for some of the features implemented. It allows users to explore different diseases and apply filters to a map of Europe, as well as view data in charts and tables. Although it is a comprehensive tool, we found the layout unintuitive and the accessibility options difficult to locate during use. This assessment motivated the search for a simpler interface for our atlas.

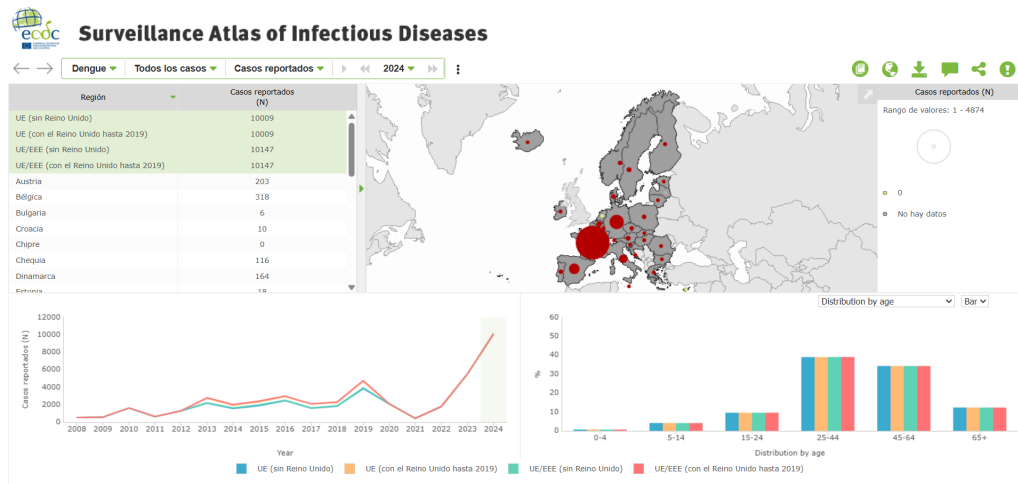


Figure 1.5: Map, filters, charts, and tables in the ECDC epidemiological atlas.

Another example is CDC WONDER, available at <https://wonder.cdc.gov/>, which provides access to health databases. This type of portal is useful for finding specific information and downloading data for use in other projects. However, the interface reviewed focuses on lists of topics and links to databases, rather than a visual presentation designed to explain their content to the general public. This organization can make searching less intuitive for users who are not already familiar with the available information.



Figure 1.6: List of topics and databases available in CDC WONDER.

In our project, we have sought to combine the strengths of these portals and the many others we have explored: access to health information, data visualization through an interactive map, and the ability to download data. These features are integrated into a simple, visual website with accessibility options, designed to be easy to use for people without prior knowledge and to enable a wider audience to use it without difficulty.

Especificaciones

2.1. Requisitos del sistema

Antes de definir la arquitectura y los componentes de la aplicación, se establecen los requisitos que debe cumplir el sistema. Estos permiten determinar las funcionalidades que debe proporcionar la aplicación y las características que debe mantener durante su funcionamiento.

2.1.1. Requisitos funcionales

Los requisitos funcionales describen las acciones que el sistema debe permitir realizar al usuario y los procesos necesarios para gestionar la información.

Tabla 2.1: Requisitos funcionales del sistema.

Código	Descripción
RF-01	El sistema permitirá consultar la lista de enfermedades disponibles.
RF-02	El usuario podrá seleccionar una enfermedad y un año.
RF-03	El sistema representará los casos epidemiológicos por comunidad autónoma.
RF-04	El sistema permitirá visualizar la distribución de los casos por género.
RF-05	El sistema permitirá visualizar la distribución de los casos por rango de edad.
RF-06	El usuario podrá descargar los datos correspondientes a los filtros seleccionados.
RF-07	El usuario podrá enviar mensajes y sugerencias mediante el formulario de contacto.
RF-08	El sistema permitirá actualizar las tasas de notificación utilizando los datos poblacionales del INE.
RF-09	El sistema almacenará en la base de datos los mensajes enviados mediante el formulario de contacto.
RF-10	El usuario podrá consultar la información detallada de las enfermedades desde las tarjetas de la página de inicio.

ID	Descripción
RF-11	El usuario podrá activar el modo de visualización en blanco y negro.
RF-12	El usuario podrá modificar el tamaño de los elementos de la interfaz a su gusto.
RF-13	El usuario podrá descargar todos los datos epidemiológicos almacenados en la base de datos.
RF-14	El sistema permitirá consultar los años para los que existen datos epidemiológicos.
RF-15	El sistema generará automáticamente las tarjetas y las páginas de las enfermedades y las rellenará con sus archivos JSON asociados.
RF-16	El sistema mostrará un mensaje de confirmación o de error después de enviar una sugerencia.

2.1.2. Requisitos no funcionales

Los requisitos no funcionales establecen las condiciones de calidad, organización y funcionamiento general que debe mantener la aplicación.

Tabla 2.2: Requisitos no funcionales del sistema.

Código	Descripción
RNF-01	La aplicación deberá funcionar correctamente en los principales navegadores web actuales.
RNF-02	La interfaz deberá adaptarse a diferentes tamaños y resoluciones de pantalla.
RNF-03	La aplicación deberá ofrecer una navegación clara y opciones que mejoren su accesibilidad.
RNF-04	Las consultas y actualizaciones deberán completarse sin esperas excesivas para el usuario.
RNF-05	La arquitectura deberá permitir incorporar nuevas enfermedades, datos y funcionalidades con facilidad.
RNF-06	Los datos deberán conservar su integridad durante su almacenamiento, consulta y representación.
RNF-07	El formulario y la API deberán validar los datos recibidos antes de procesarlos o almacenarlos.
RNF-08	El mapa deberá actualizar su contenido sin necesidad de recargar la página web y sin pulsar el botón.
RNF-09	La interfaz de usuario deberá permanecer separada del acceso directo a la base de datos.
RNF-10	El código deberá organizarse en módulos independientes para facilitar su mantenimiento y modificación.

2.2. Estructura

Para comprender mejor la estructura y el funcionamiento de cada parte de la aplicación, cómo interactúan las diferentes tecnologías y cuál es el orden de cada proceso, se presentan los siguientes diagramas de flujo.

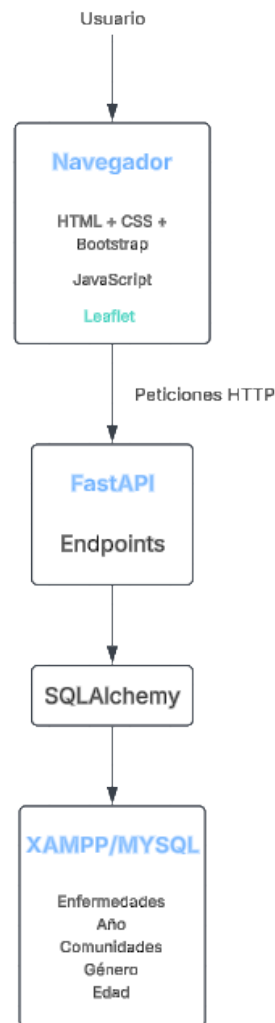


Figura 2.1: Diagrama de flujo de la arquitectura de la aplicación.

La Figura 2.1 representa cómo se conectan las tecnologías desde que el usuario accede a la página web y explora sus diferentes apartados, incluido el mapa interactivo. Cuando el usuario selecciona los filtros del mapa, JavaScript envía a la API una petición HTTP para obtener la información necesaria. Una vez recibida la petición, la API utiliza los endpoints desarrollados para solicitar la información pertinente a la base de datos SQL mediante SQLAlchemy. Tras recuperar los datos, la respuesta recorre el camino inverso hasta mostrarse en el navegador, donde el usuario puede visualizar la información solicitada.

La API no solo se conecta con la base de datos para buscar información que proporcionar a la página web. Para crear las tarjetas de las enfermedades, la API consulta el archivo JSON correspondiente a cada una de ellas y obtiene información adicional. Una vez recopilados los datos del archivo JSON, la API los envía a JavaScript para que los transforme en tarjetas y los muestre visualmente al usuario. El siguiente diagrama representa este proceso alternativo:

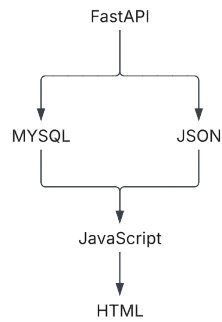


Figura 2.2: Diagrama de flujo para la obtención de información de los archivos JSON.

En el siguiente gráfico se muestra con mayor detalle el proceso que sigue el mapa epidemiológico para obtener los datos y actualizarse:

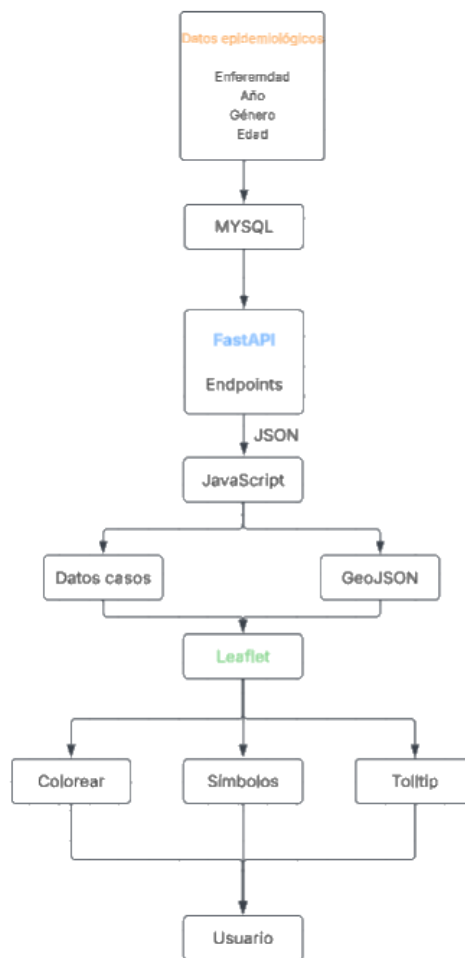


Figura 2.3: Diagrama de flujo detallado de la actualización del mapa epidemiológico.

El proceso representado en este diagrama comienza una vez que se han seleccionado los filtros, se ha pulsado el botón de búsqueda y la API ha solicitado los datos necesarios a la base de datos, que contiene el CSV actualizado. Tras recopilarnos, la API los envía a JavaScript en formato JSON. JavaScript combina estos datos con el archivo GeoJSON, que permite dibujar las comunidades autónomas en el mapa de Leaflet. A continuación, colorea el mapa, añade el símbolo correspondiente y muestra ventanas flotantes mediante tooltips según los filtros seleccionados, para presentar finalmente la información al usuario. Aunque no se refleje de manera explícita en el diagrama, la leyenda también se actualiza según los filtros seleccionados, ya que forma parte de los marcadores del mapa.

El siguiente diagrama representa el funcionamiento de los filtros del mapa:

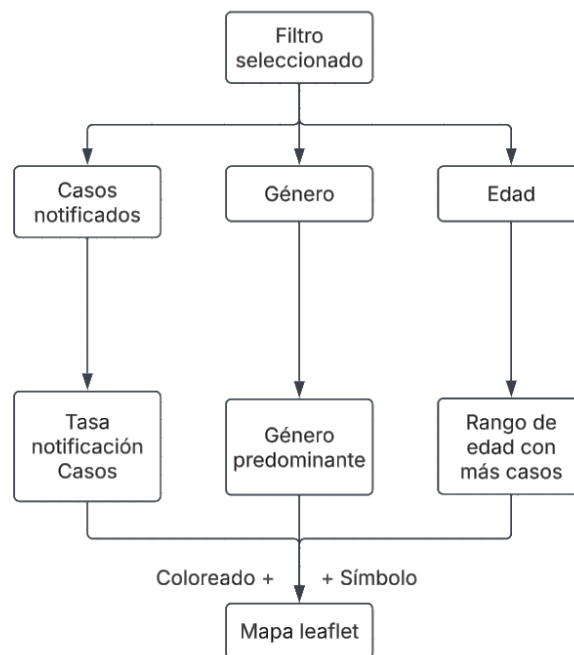


Figura 2.4: Diagrama de flujo del funcionamiento de los filtros del mapa.

Este diagrama amplía la fase de representación del mapa mostrada en el diagrama anterior y permite observar cómo cambia su apariencia según el filtro seleccionado. Si se elige el filtro de casos notificados, se solicitan los casos de una enfermedad y un año concretos en las distintas comunidades autónomas. Una vez obtenidos los datos, cada comunidad se representa mediante los colores cálidos descritos en la leyenda y mediante puntos cuyo tamaño varía según la tasa de notificación. Si se selecciona el filtro de género, se obtiene el número de mujeres y hombres afectados por esa enfermedad durante el año elegido. El mapa utiliza el color correspondiente al género predominante y muestra su símbolo asociado, tal como se indica en la leyenda. Por último, si se selecciona el filtro de edad, se obtienen los casos asociados a cada rango de edad. El mapa se colorea según el rango predominante y añade el símbolo correspondiente definido en la leyenda.

El siguiente diagrama representa el proceso de generación de las tarjetas de información de las enfermedades:

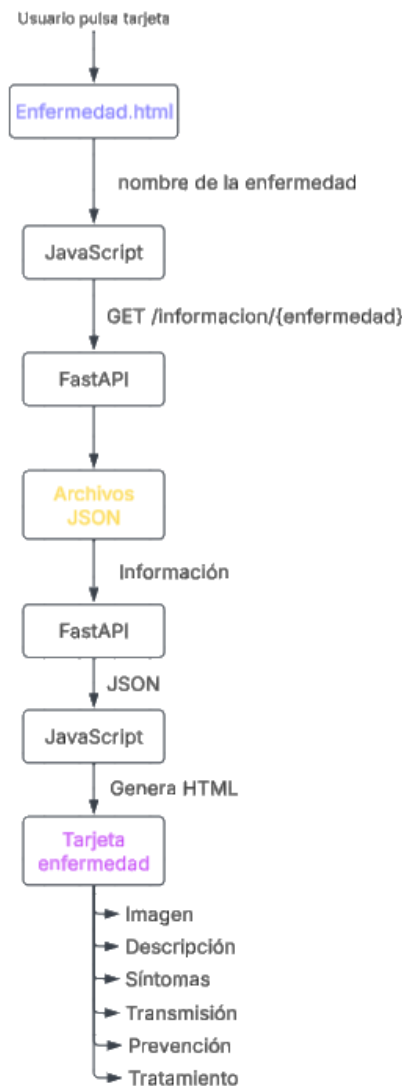


Figura 2.5: Diagrama de flujo de la generación de las tarjetas de enfermedades.

El proceso comienza cuando el usuario pulsa el botón *Saber más* de la tarjeta de una enfermedad en la página principal. Esta acción lo redirige a una página web que contiene información adicional sobre dicha enfermedad. El archivo `enfermedad.html` proporciona el nombre de la tarjeta seleccionada para que JavaScript pueda solicitar a la API los datos de la enfermedad correspondiente. Cuando la API devuelve la información almacenada en el archivo JSON, JavaScript genera dinámicamente el contenido HTML de la página `enfermedad.html` y muestra al usuario los datos adicionales, como los síntomas, el tratamiento, la transmisión y las medidas de prevención, entre otros.

El siguiente diagrama representa el proceso de envío, validación y almacenamiento de los mensajes del formulario de contacto:

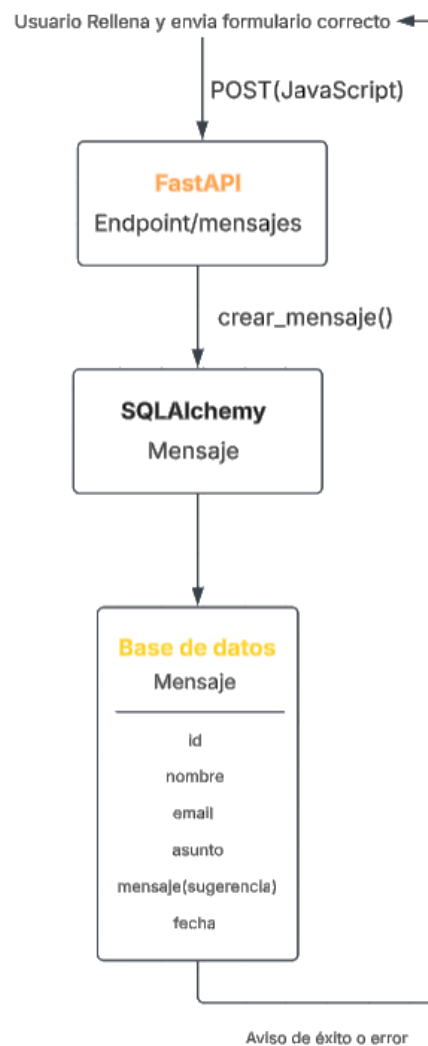


Figura 2.6: Diagrama de flujo del envío y almacenamiento de los mensajes de contacto.

Este diagrama representa el proceso mediante el cual el mensaje enviado por el usuario se almacena en la base de datos. El proceso comienza cuando el usuario completa el formulario de la pestaña de contacto y pulsa el botón de envío. JavaScript recoge los datos introducidos, los convierte a formato JSON y los envía a la API mediante una petición POST. A continuación, la API recibe los datos y los valida, ejecutando posteriormente la función `crear_mensaje()` del backend. Esta función crea el nuevo registro mediante SQLAlchemy y lo almacena en la tabla de mensajes de la base de datos con todos sus campos, incluyendo automáticamente el identificador y la fecha. Tanto si el proceso finaliza correctamente como si se produce algún error durante cualquier parte del proceso, se proporciona al usuario un aviso indicando si el envío se ha realizado correctamente o si se ha producido un error.

2.2.1. Cadena de procesamiento y tipos de datos

El siguiente diagrama resume la cadena completa de procesamiento, desde la recopilación de los datos hasta su representación en la interfaz web.

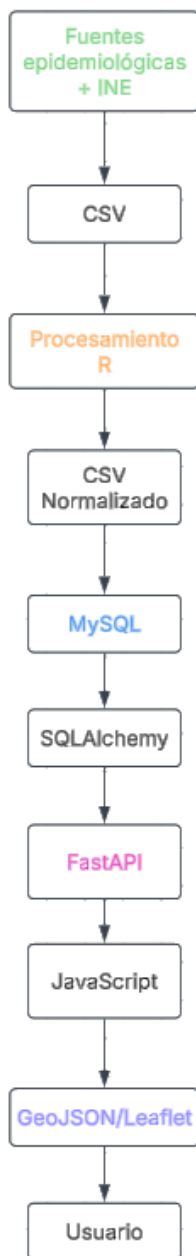


Figura 2.7: Cadena de procesamiento total de los datos del proyecto.

La Tabla 2.3 diferencia los datos estáticos, los datos procesados y los datos generados dinámicamente. También indica si cada elemento se prepara de forma offline o se utiliza ante una petición del usuario.

Tabla 2.3: Clasificación de los datos según su procesamiento y momento de uso.

Tipo de dato	Elemento	Momento	Uso
Estático	CSV original	Offline, antes de ejecutar la aplicación	Fuente inicial de los datos epidemiológicos.
Estático	JSON de enfermedades	Preparado offline y leído ante una petición	Información descriptiva de las enfermedades.
Estático	GeoJSON	Preparado offline y cargado en el navegador	Geometría de las comunidades autónomas.
Procesado	CSV actualizado	Offline, mediante los scripts de R	Datos normalizados con las tasas de notificación actualizadas.
Procesado	MySQL	Cargado offline y consultado ante una petición	Almacenamiento persistente utilizado por la API.
Dinámico	JSON de FastAPI	Ante una petición del usuario	Respuesta con los datos solicitados por el frontend.
Dinámico	Mapa de Leaflet	Ante una petición, en el navegador	Representación interactiva de los datos seleccionados.

La preparación del CSV, su actualización mediante R y la carga de los registros en MySQL forman la parte offline del proceso. En cambio, las consultas realizadas mediante SQLAlchemy y FastAPI, el procesamiento de la respuesta con JavaScript y la actualización del mapa se ejecutan ante una petición del usuario.

2.3. Arquitectura del software

La aplicación se organiza en diferentes capas, cada una de ellas responsable de una parte concreta del sistema. Esta separación facilita la comprensión del proyecto, el mantenimiento del código y la incorporación de nuevas funcionalidades.

2.3.1. Capa de presentación

La capa de presentación contiene la interfaz con la que interactúa el usuario. Se encarga de mostrar la información, recoger las acciones realizadas en la página y solicitar los datos necesarios a la API. Está formada por los siguientes componentes:

- **HTML, CSS y Bootstrap:** definen la estructura, el diseño y la adaptación de las páginas a distintos tamaños de pantalla.
- **index.js:** carga las enfermedades disponibles y genera dinámicamente las tarjetas de la página de inicio. Realiza las peticiones a la API para obtener la información descriptiva almacenada en el archivo JSON correspondiente a cada enfermedad.

- **enfermedad.js:** gestiona la página de información individual de cada enfermedad. Obtiene su nombre a partir de los parámetros de la URL, realiza una petición a `/informacion/{enfermedad}` y utiliza la respuesta para construir dinámicamente el contenido de la página, incluyendo síntomas, transmisión, prevención, tratamiento e información adicional del archivo JSON asociado.
- **mapa.js:** gestiona los filtros y la representación del mapa epidemiológico.
- **descarga.js:** controla los botones de descarga y genera los archivos CSV con los datos correspondientes a los filtros del mapa o con todos los registros epidemiológicos almacenados en la base de datos. Para la descarga completa, realiza una petición al endpoint `/obtener_todo`.
- **contacto.js:** recoge los datos introducidos en el formulario de contacto y, cuando el usuario pulsa el botón de envío, los envía a la API en formato JSON para almacenar la sugerencia en la base de datos.
- **accesibilidad.js:** controla las opciones de tamaño y color de la interfaz. Permite seleccionar entre los tamaños normal, grande y muy grande y almacena la opción elegida mediante `localStorage`, conservando la preferencia al cambiar de página.
- **componentes.js:** permite reutilizar los elementos comunes de la interfaz. Mediante peticiones con `fetch()`, carga los archivos HTML de la cabecera, el menú lateral y el pie de página en todas las páginas.
- **menu.js:** controla la apertura y el cierre del menú lateral, modificando el estado visual del botón y su etiqueta de accesibilidad.

2.3.2. Capa de servicios y API

Esta capa recibe las peticiones del frontend, ejecuta la operación correspondiente y devuelve una respuesta en formato JSON. Sus componentes principales son:

- **main.py:** define la aplicación FastAPI y sus rutas principales.
- **Endpoints REST:** permiten consultar los datos epidemiológicos, obtener información sobre las enfermedades y enviar sugerencias.
- **schemas.py:** contiene los esquemas de Pydantic que establecen la estructura y los tipos de los datos recibidos y enviados por la API.

2.3.3. Capa de acceso a datos

La capa de acceso a datos comunica la API con la base de datos y agrupa la lógica necesaria para realizar las consultas y modificaciones.

- **crud.py:** contiene las operaciones de consulta, inserción y tratamiento de los registros.

- **models.py:** define los modelos que representan las tablas de la base de datos.
- **database.py:** configura la conexión y las sesiones utilizadas para acceder a MySQL.
- **SQLAlchemy:** permite relacionar los modelos de Python con las tablas de la base de datos.
- **PyMySQL:** actúa como controlador para establecer la comunicación entre SQLAlchemy y MySQL.

2.3.4. Capa de persistencia

Esta capa reúne los recursos en los que se almacena la información utilizada por la aplicación.

- **MySQL:** almacena los registros epidemiológicos y los mensajes enviados mediante el formulario de contacto.
- **Archivos JSON:** contienen la información descriptiva de cada enfermedad.
- **Archivo GeoJSON:** contiene las geometrías de las comunidades autónomas utilizadas para representar el mapa.

2.3.5. ETL y preprocesamiento de datos

Esta parte de la arquitectura se encarga de extraer, transformar y cargar los datos antes de que sean utilizados por la aplicación.

- **Archivos CSV:** contienen los datos epidemiológicos recopilados y los resultados actualizados.
- **Datos poblacionales del INE:** se descargan desde la tabla del padrón municipal y proporcionan la población necesaria para calcular las tasas de notificación.
- **actualizar_datos.R:** normaliza los nombres de las comunidades, combina los registros epidemiológicos con la población del INE, comprueba posibles errores, calcula las tasas y genera el CSV actualizado.
- **Insertar_CSV.R:** carga los registros del CSV actualizado en la base de datos MySQL.

Estas operaciones forman parte de la preparación offline de los datos. Se ejecutan cuando es necesario incorporar o actualizar información, pero no cada vez que un usuario consulta la aplicación.

2.3.6. Dependencias entre procesos y archivos

Las capas de la aplicación mantienen relaciones de dependencia que permiten transmitir los datos desde las fuentes originales hasta la interfaz de usuario. Cada relación utiliza una interfaz concreta y mantiene separadas las responsabilidades de presentación, procesamiento, acceso y almacenamiento.

Capa de presentación → API. Los archivos JavaScript realizan peticiones HTTP a los endpoints definidos en FastAPI. Las dependencias principales son:

- `mapa.js` → `/buscar`, `/genero/...` y `/edad/...`
- `index.js` → `/enfermedades_lista` y `/informacion/...`
- `enfermedad.js` → `/informacion/...`
- `contacto.js` → `/contacto`.
- `descarga.js` → `/obtener_todo` para la descarga completa.

La interfaz entre el frontend y la API es HTTP/REST y los datos se intercambian en formato JSON. En la descarga filtrada, `mapa.js` entrega a `descarga.js` los datos guardados en `datosMapa`, por lo que no se realiza una nueva petición.

FastAPI → Pydantic. Los endpoints de `main.py` utilizan los esquemas definidos en `schemas.py` para validar la estructura de determinados datos de entrada y salida.

FastAPI → CRUD. `main.py` delega las consultas e inserciones en las funciones de `crud.py` mediante llamadas a funciones de Python.

CRUD → SQLAlchemy y modelos. `crud.py` utiliza SQLAlchemy y los modelos ORM de `models.py` para realizar las operaciones sobre las tablas.

Acceso a MySQL. SQLAlchemy utiliza la conexión configurada en `database.py` y PyMySQL actúa como controlador para comunicarse con MySQL.

INE y CSV original → R → CSV actualizado. `actualizar_datos.R` obtiene la población del INE, la combina con el CSV epidemiológico y genera el CSV actualizado. Después, `Insertar_CSV.R` carga sus registros en MySQL.

2.4. Modelo de datos

El modelo de datos de la aplicación está formado principalmente por dos entidades: los registros epidemiológicos y los mensajes enviados mediante el formulario de contacto. Las siguientes tablas describen sus campos, tipos de datos y funciones.

2.4.1. Registro epidemiológico

El modelo `RegistroEpidemiologico` representa la información almacenada sobre una enfermedad en una comunidad autónoma y un año determinados.

Tabla 2.4: Campos del modelo `RegistroEpidemiologico`.

Campo	Tipo	Descripción
id	int	Identificador único del registro.
enfermedad	str	Nombre de la enfermedad.
anyo	int	Año al que corresponden los datos.
comunidad	str	Comunidad autónoma asociada al registro.
casos	int o null	Número de casos notificados.
tasa_notificacion	float o null	Tasa de notificación por cada 100.000 habitantes.
hombres	int o null	Número de casos correspondientes a hombres.
mujeres	int o null	Número de casos correspondientes a mujeres.
rango_edad	str o null	Rango de edad asociado a los casos.
casos_edad	int o null	Número de casos del rango de edad indicado.

Los campos que admiten el valor `null` pueden quedar vacíos cuando la fuente de datos no proporciona esa información.

2.4.2. Representación JSON del registro epidemiológico

Los registros del modelo epidemiológico se devuelven al frontend en formato JSON. La Figura 2.8 muestra un ejemplo de la respuesta generada por el endpoint `/buscar` al consultar los casos de tuberculosis registrados en Cataluña durante el año 2023. Cada objeto contiene los campos del registro, como la enfermedad, el año, la comunidad autónoma, el número de casos y la tasa de notificación. Los campos para los que no existe información aparecen como cadenas vacías.

```
[
  {
    "anyo": 2023,
    "comunidad": "Cataluña",
    "tasa_notificacion": 12.313,
    "mujeres": "",
    "casos_edad": "",
    "casos": 973,
    "id": 117,
    "enfermedad": "Tuberculosis",
    "hombres": "",
    "rango_edad": ""
  }
]
```

Figura 2.8: Ejemplo de representación JSON de un registro epidemiológico.

La Figura 2.9 muestra un segundo ejemplo correspondiente a los datos generales de España. En este registro aparecen el número total de casos de dengue durante el año 2023, la distribución entre mujeres y hombres y los casos asociados al rango de edad de 15 a 25 años. De esta forma, un mismo objeto JSON puede reunir los datos epidemiológicos generales y la información utilizada por los filtros de género y edad.

```
{
  "anyo": 2023,
  "comunidad": "España",
  "tasa_notificacion": 1.281,
  "mujeres": "29",
  "casos_edad": "58",
  "casos": 616,
  "id": 68,
  "enfermedad": "Dengue",
  "hombres": "29",
  "rango_edad": "15-25"
},
```

Figura 2.9: Ejemplo de un registro JSON con datos de género y rango de edad para el conjunto de España.

2.4.3. Mensaje de contacto

El modelo `Mensaje` representa las sugerencias y consultas enviadas por los usuarios mediante el formulario de contacto.

Tabla 2.5: Campos del modelo `Mensaje`.

Campo	Tipo	Descripción
<code>id</code>	<code>int</code>	Identificador único del mensaje.
<code>nombre</code>	<code>str</code>	Nombre de la persona que envía el mensaje.
<code>email</code>	<code>str</code>	Dirección de correo electrónico del usuario.
<code>asunto</code>	<code>str</code>	Asunto principal de la consulta o sugerencia.
<code>mensaje</code>	<code>str</code>	Contenido enviado mediante el formulario.
<code>fecha</code>	<code>datetime</code>	Fecha y hora en las que se almacena el mensaje.

2.4.4. Estructuras de datos utilizadas en JavaScript

El frontend utiliza diferentes estructuras de JavaScript para almacenar las respuestas de la API, transformar la información y generar los elementos que se muestran en las páginas. La Tabla 2.6 resume las estructuras principales de cada archivo sin entrar en el funcionamiento individual de todas sus funciones.

Tabla 2.6: Estructuras de datos utilizadas por los principales archivos JavaScript.

Archivo	Estructuras	Ejemplos	Uso
<code>index.js</code>	Arrays y objetos JSON	<code>enfermedades</code> y <code>datos</code>	Obtiene las enfermedades y genera automáticamente las tarjetas de inicio.
<code>contacto.js</code>	Objeto clave-valor convertido a JSON	<code>datosFormulario</code>	Agrupar los campos del formulario y los envía a la API mediante una petición POST.
<code>enfermedad.js</code>	Objeto JSON y arrays	<code>datos</code> y <code>datos.sintomas.map(...)</code>	Construye la página de una enfermedad y transforma sus listas en elementos HTML.
<code>mapa.js</code>	Arrays, objetos JSON, diccionarios, GeoJSON, objetos DOM y objetos de Leaflet	<code>datosMapa</code> , <code>equivalencias</code> , <code>datos.features</code> y <code>capaComunidades</code>	Relaciona los datos epidemiológicos con el mapa y controla sus modos de visualización.

El archivo `mapa.js` combina un mayor número de estructuras porque debe relacionar la información epidemiológica con las geometrías, delimitar y colorear las

comunidades autónomas y añadir elementos interactivos al mapa. Sus estructuras principales se recogen en la Tabla 2.7.

Tabla 2.7: Estructuras principales utilizadas en `mapa.js`.

Estructura	Ejemplo	Uso
Arrays de valores	<code>enfermedades</code> y <code>anyos</code>	Almacenan las listas utilizadas para completar los filtros de enfermedad y año.
Arrays de objetos	<code>datos</code> en <code>actualizarCasos()</code> y <code>actualizarEdad()</code>	Almacenan los registros devueltos por los endpoints <code>/buscar</code> y <code>/edad</code> .
Objeto JSON	<code>datos</code> en <code>actualizarGenero()</code>	Almacena los campos <code>hombres</code> y <code>mujeres</code> devueltos por el endpoint <code>/genero</code> .
Variable de respuesta	<code>datosMapa</code>	Conserva la respuesta correspondiente al modo seleccionado; puede contener un array o un objeto JSON.
Objeto clave-valor	<code>equivalencias</code>	Relaciona los nombres del archivo GeoJSON con los empleados en la base de datos.
Objetos visuales clave-valor	<code>colores</code> e <code>iconos</code>	Asocian cada rango de edad con el color y la imagen utilizados en su representación.
Objeto GeoJSON y elementos <i>Feature</i>	<code>datos</code> , <code>datos.features</code> y <code>feature</code>	El objeto contiene la colección geográfica, mientras que cada <i>Feature</i> representa una comunidad autónoma mediante sus propiedades y su geometría.
Elementos DOM	<code>filtroEnfermedad</code> , <code>filtroAnyo</code> , <code>filtroMostrar</code> y los botones	Permiten leer los filtros, actualizar la interfaz y responder a las acciones del usuario.
Objetos gráficos de Leaflet	<code>L.circleMarker(...)</code> , <code>L.marker(...)</code> , <code>L.divIcon(...)</code> y <code>L.rectangle(...)</code>	Configuran los círculos, marcadores, iconos y el recuadro de las islas Canarias.
Capa GeoJSON y grupos de capas	<code>capaComunidades</code> , <code>capaCirculos</code> , <code>capaGenero</code> y <code>capaEdad</code>	La primera contiene las comunidades y las demás agrupan los elementos de cada modo de visualización.
Objeto de configuración	<code>{zoomControl: true, ...}</code>	Establece el comportamiento inicial y las opciones de navegación del mapa.

2.5. Organización de los archivos del proyecto

El proyecto se divide en tres carpetas principales: **backend**, **datos** y **frontend**. Esta separación permite localizar fácilmente los archivos según su función y mantener independientes la lógica de la API, la preparación de los datos y la interfaz web.

2.5.1. Backend y archivos de datos

La carpeta **backend** agrupa los módulos de Python que forman la API y la configuración de la conexión con MySQL, definiendo los modelos y esquemas que realizan las consultas a la base de datos. Estos archivos se encuentran directamente en la raíz de la carpeta. Dentro de **backend/datos** se almacenan los archivos JSON con la información descriptiva de las enfermedades. También pueden aparecer carpetas generadas por Python, como el entorno virtual y la caché de ejecución.

La carpeta general **datos** contiene los archivos CSV con la información epidemiológica y los scripts de R empleados para actualizar las tasas de notificación e insertar los registros en la base de datos. De esta forma, los datos utilizados durante el preprocesamiento permanecen separados del código del backend.

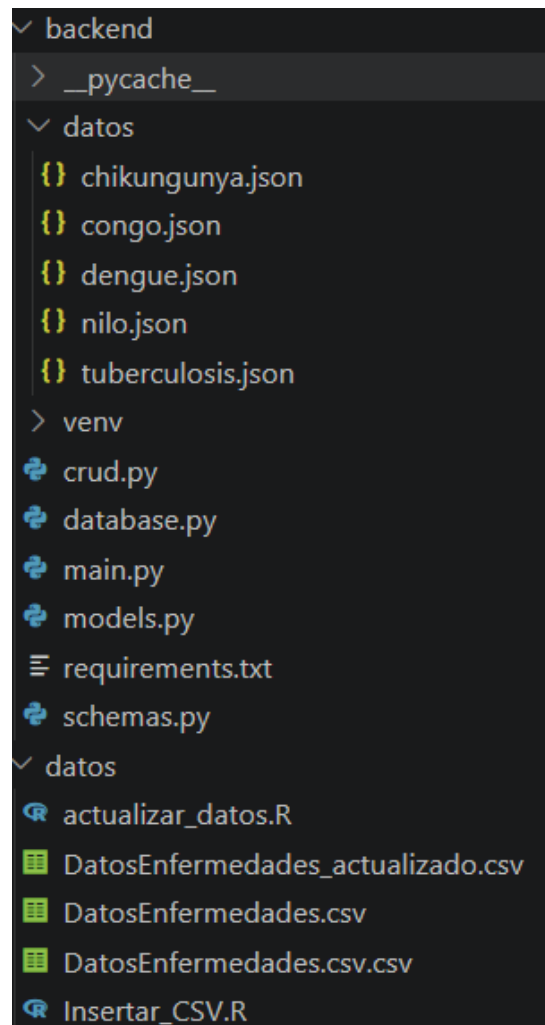


Figura 2.10: Organización de las carpetas backend y datos.

2.5.2. Frontend

La carpeta **frontend** reúne todos los elementos de la interfaz. Las páginas HTML principales se encuentran en su raíz, mientras que los elementos comunes reutilizados por varias páginas se guardan en **frontend/componentes**. Las hojas de estilo se agrupan en **frontend/css**, los scripts en **frontend/js**, las imágenes en **frontend/img** y el archivo geográfico del mapa en **frontend/geojson**. Esta distribución permite identificar el tipo de recurso por la carpeta en la que se encuentra sin mezclar la estructura de las páginas con su diseño o funcionamiento.

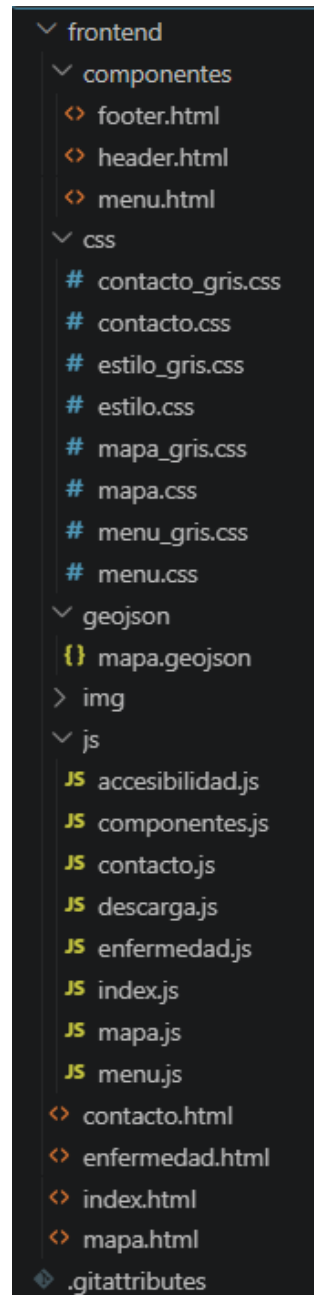


Figura 2.11: Organización de los archivos de la carpeta `frontend`.

Tecnología y lenguajes empleados

3.1. Desarrollo back-end

3.1.1. CSV

Un archivo CSV (Comma-Separated Values) es un formato de texto plano que organiza datos en filas y columnas mediante delimitadores (como la coma o el punto y coma). Lo utilizamos en nuestro proyecto en lugar de otros formatos porque no requiere instalación ni configuración previa y desempeña un excelente papel como fichero intermedio de transferencia [23].

En este proyecto optamos por utilizar el punto y coma (;) como separador de celdas. Esta decisión se debe a que, en español, los decimales se separan con coma (por ejemplo, en la variable tasa de notificación). Si hubiéramos usado la coma como delimitador de columnas, las herramientas de lectura habrían interpretado un los decimales como dos columnas distintas, corrompiendo los datos [23].

Además, el formato CSV es compatible tanto con R Commander, que utilizamos para generar un nuevo CSV con la tasa de notificación actualizada o insertada mediante los datos del padrón del INE, como con XAMPP (MySQL), donde se realiza la inserción de la información en la base de datos.

3.1.2. R Commander

R Commander es una interfaz gráfica de usuario (GUI) para el lenguaje y entorno estadístico R. Utilizamos RStudio que es un entorno de desarrollo integrado para programar en el lenguaje R.

En este proyecto, R se utiliza principalmente para conectarse a los servidores del Instituto Nacional de Estadística (INE) y descargar la tabla 67988 del padrón municipal por comunidades autónomas, cuyos datos se actualizan anualmente. De esta forma, cuando se incorporan al CSV nuevos registros, enfermedades o años de estudio, el script puede recalcular automáticamente las tasas de notificación con los datos poblacionales correspondientes.

Antes de combinar la información, el script convierte los años a formato numérico y normaliza los nombres de las comunidades autónomas tanto en el CSV epidemiológico como en los datos del INE. Este proceso tiene en cuenta los acentos y caracteres propios del español, como la ñ, y evita que las diferencias de escritura impidan relacionar correctamente los registros.

Una vez normalizados los datos, ambas fuentes se unen mediante la combinación de comunidad autónoma y año. Antes de calcular las tasas, el script comprueba si existen registros duplicados o filas para las que no se ha encontrado una población asociada.

La tasa de notificación que calcula el script sigue la siguiente fórmula:

$$\text{Tasa de notificación} = \frac{\text{Casos}}{\text{Población}} \times 100\,000 \quad (3.1)$$

Los casos corresponden a las personas afectadas por esa enfermedad en una comunidad autónoma y un año concretos. El resultado se multiplica por 100.000 porque las tasas de notificación se calculan por cada cien mil habitantes.

Si un registro dispone de población, se calcula su nueva tasa de notificación. En caso contrario, se conserva la tasa anterior para evitar la pérdida de información. Finalmente, se eliminan las columnas auxiliares empleadas durante el procesamiento y se genera el archivo `DatosEnfermedades_actualizado.csv` con codificación UTF-8. Esta codificación conserva correctamente todos los caracteres y permite importar posteriormente los datos en la base de datos MySQL gestionada mediante XAMPP.

El funcionamiento del script `actualizar_datos.R` puede resumirse mediante el siguiente pseudocódigo:

```

INICIO
  Leer el CSV epidemiológico
  Convertir los años a valores numéricos
  Normalizar los nombres de las comunidades
  Descargar los datos de población del INE
  Seleccionar el año, la comunidad y la población total
  Normalizar los nombres de las comunidades del INE
  Unir ambas fuentes mediante la comunidad y el año
  Comprobar poblaciones ausentes y registros duplicados
  PARA cada registro HACER
    SI existe una población asociada ENTONCES
      Calcular la tasa de notificación
    SI NO
      Conservar la tasa anterior
  Eliminar las columnas auxiliares
  Guardar el CSV epidemiológico actualizado
FIN

```

Además, el archivo `Insertar_CSV.R` permite insertar automáticamente las filas del CSV actualizado en la base de datos MySQL, sin necesidad de realizar manualmente la importación desde XAMPP.

3.1.3. MySQL

MySQL es un sistema de gestión de bases de datos relacionales que permite almacenar, organizar y consultar información mediante el lenguaje SQL. En nuestro proyecto, permite realizar consultas y filtros de forma eficiente y facilita que la información pueda ser utilizada posteriormente por la API desarrollada con FastAPI.

El entorno local que hemos utilizado para gestionar y administrar la base de datos es XAMPP. En particular, se ha utilizado phpMyAdmin para crear, consultar y modificar la base de datos durante el desarrollo de la aplicación. Esta herramienta proporciona una interfaz gráfica que facilita la gestión de los datos y permite realizar operaciones complejas sin necesidad de interactuar con la línea de comandos.

La base de datos utilizada por el proyecto se denomina `enfermedades_emergentes` y contiene la tabla `enfermedades`, en la que se almacenan los registros epidemiológicos.

La base de datos no solo contiene la tabla `enfermedades`. La pestaña de contacto dispone de un formulario de sugerencias que permite a los usuarios comunicar posibles mejoras para la página web, necesidades que se hayan podido pasar por alto o nuevas enfermedades detectadas en España que deban incorporarse al estudio, entre otras cuestiones. Una vez que el usuario complete el formulario y pulse el botón de envío, la API recibe el mensaje y lo almacena en la tabla `mensajes` de la base de datos para poder revisarlo posteriormente y estudiar su viabilidad.

3.1.4. FastAPI

FastAPI es un framework web de alto rendimiento para construir API con Python. La API que creamos con FastAPI actúa como intermediaria entre la interfaz web y la información almacenada en la base de datos. Se utiliza para centralizar el acceso a la información y proporcionársela al frontend mediante diferentes endpoints HTTP [17, 18].

FastAPI es compatible con SQL integrando diferentes herramientas de acceso a bases de datos, utilizando en este proyecto SQLAlchemy junto con PyMySQL para trabajar con MySQL. Permite definir endpoints claros y facilita un desarrollo más sencillo y rápido en Python [16].

Una vez definidos los endpoints para todos los casos posibles con los campos modelados en la base de datos, la API recibe las peticiones realizadas desde la interfaz web, las procesa y devuelve los datos correspondientes en formato JSON. Este proceso facilita la obtención de datos mediante peticiones, sin necesidad de que el frontend acceda directamente a la base de datos.

La API dispone de diferentes endpoints destinados a distintas necesidades de la aplicación:

Tabla 3.1: Endpoints disponibles en la API.

Endpoint	Descripción
/	Comprueba que la API está funcionando y devuelve un mensaje de confirmación.
/buscar	Permite realizar búsquedas combinando enfermedad, año y comunidad autónoma. Se utiliza para obtener los datos que se representan en el mapa y poder actualizarlo.
/enfermedades_lista	Proporciona la lista de enfermedades disponibles en nuestra base de datos.
/anyos	Proporciona la lista de años disponibles en nuestro estudio.
/informacion/{enfermedad}	Obtiene la información detallada de una enfermedad a partir de los archivos JSON.
/genero/{enfermedad}/{anyo}	Obtiene los casos diferenciados por género para una enfermedad y un año determinados.
/edad/{enfermedad}/{anyo}	Obtiene los casos agrupados por rango de edad para una enfermedad y un año determinados.
/obtener_todo	Obtiene todos los registros almacenados en la tabla de enfermedades y se utiliza para la descarga completa de los datos en formato CSV.
/contacto	Recibe los datos enviados mediante el formulario de contacto y los almacena en la tabla de mensajes de la base de datos.

Los endpoints, las consultas a la base de datos y la definición de los modelos están separados en la implementación. En `crud.py` encontramos las consultas realizadas mediante SQLAlchemy, mientras que `main.py` se encarga principalmente de definir las rutas de la API. Esta separación se ha realizado con el objetivo de que el código esté mejor organizado y sea más fácil de entender.

FastAPI se integra con SQLAlchemy, que se utiliza como herramienta de acceso a la base de datos MySQL. Se crea una conexión mediante la cual se mantiene una sesión `SessionLocal`, que posteriormente FastAPI proporciona a cada petición mediante la dependencia `get_db()`. Esto permite que cada petición utilice una sesión de la base de datos y que esta se cierre posteriormente, manteniendo así el ciclo de vida de la sesión utilizada para consultar la base de datos.

En el backend también utilizamos Pydantic, que nos permite definir la estructura de los datos que maneja la API. Nos ayuda a especificar los campos esperados y sus tipos. Esta implementación establece una estructura clara y facilita la conversión de los datos sin que se produzcan errores.

Una vez procesada la petición, la respuesta debe enviarse al frontend. Esta conexión permite separar el backend del frontend. Después de que JavaScript envíe una petición HTTP a la API, el backend responde con la información solicitada para que, finalmente, JavaScript la muestre en la página. Para que la comunicación entre ambos sea más sencilla, hemos configurado CORS. CORS permite gestionar las solicitudes entre la interfaz de usuario y la API cuando se encuentran en orígenes diferentes.

En nuestro proyecto, la API no se limita a recibir peticiones del usuario para obtener información y mostrarla visualmente mediante el mapa o las tarjetas. En la pestaña de contacto, cuando el usuario completa el formulario, la API procesa los datos introducidos mediante el endpoint `/contacto`, que implementa el método POST. Estos datos se validan de acuerdo con los esquemas definidos en el archivo `schemas.py`. Una vez validados, se almacenan en la tabla `mensajes` de la base de datos y, para finalizar el proceso, se envía al usuario un mensaje de confirmación o de error.

Por tanto, nuestra API trabaja en ambos sentidos: consulta la información almacenada en la base de datos para enviarla al frontend y también recibe información desde el frontend para validarla y almacenarla en la base de datos.

Comprobación individual de los endpoints.

FastAPI genera automáticamente una interfaz de documentación desde la que se pueden consultar y ejecutar los endpoints. A continuación, se describe el método HTTP, los parámetros de entrada, la estructura de la respuesta y los posibles errores de cada operación. Las capturas muestran ejemplos realizados desde esta interfaz.

3.1.4.1. Endpoint raíz /

Este endpoint permite comprobar rápidamente si la API se encuentra activa.

- **Método HTTP:** GET.
- **Parámetros:** no requiere parámetros.
- **Respuesta:** objeto JSON con el campo `mensaje` y el texto `API funcionando`.
- **Código HTTP:** 200 OK cuando la API responde correctamente.
- **Posibles errores:** si el servidor no se ha iniciado o no está disponible, no se podrá establecer la conexión.

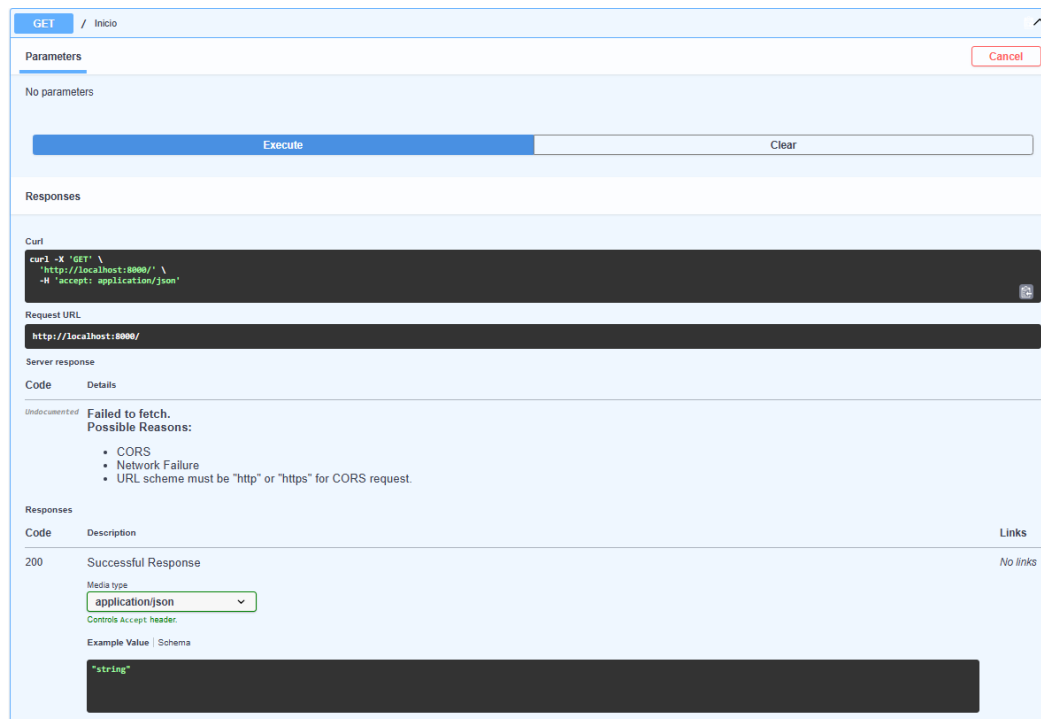


Figura 3.1: Ejecución y respuesta del endpoint raíz de la API.

3.1.4.2. Endpoint /buscar

El endpoint `/buscar` obtiene los registros utilizados para representar los casos en el mapa. La función de `crud.py` construye la consulta y aplica únicamente los filtros que hayan sido indicados. Además, selecciona los registros generales que no contienen un rango de edad específico.

- **Método HTTP:** GET.
- **Parámetros opcionales:** `enfermedad` y `comunidad`, de tipo cadena de texto, y `año`, de tipo entero.
- **Respuesta:** lista de objetos JSON con los registros epidemiológicos que coinciden con los filtros.
- **Códigos HTTP:** 200 cuando la consulta se procesa correctamente y 422 si el año no tiene un formato entero válido.
- **Posibles errores:** si no existen coincidencias, se devuelve una lista vacía. Un problema de conexión con la base de datos puede producir un error interno del servidor.

The screenshot shows a REST client interface with the following details:

- Method:** GET
- Endpoint:** /buscar
- Parameters:**
 - enfermedad:** Tuberculosis (string, query)
 - anyo:** 2023 (integer, query)
 - comunidad:** Asturias (string, query)
- Buttons:** Execute, Clear
- Responses:**
 - Curl:**

```
curl -X 'GET' \
  'http://localhost:8000/buscar?enfermedad=Tuberculosis&anyo=2023&comunidad=Asturias' \
  -H 'accept: application/json'
```
 - Request URL:** http://localhost:8000/buscar?enfermedad=Tuberculosis&anyo=2023&comunidad=Asturias

Figura 3.2: Introducción de los parámetros del endpoint /buscar.

The screenshot shows the server response and HTTP status codes for the /buscar endpoint:

- Server response:**
 - Code:** 200
 - Response body:**

```
{
  "anyo": 2023,
  "comunidad": "Asturias",
  "tasa_notificacion": 7.435,
  "mujeres": "",
  "casos_edad": "",
  "casos": 75,
  "id": 111,
  "enfermedad": "Tuberculosis",
  "hombres": "",
  "rango_edad": ""
}
```
 - Response headers:**

```
content-length: 172
content-type: application/json
date: Thu, 03 Sep 2026 09:58:03 GMT
server: uvicorn
```
- Responses:**
 - 200:** Successful Response (Media type: application/json)
 - 422:** Validation Error (Media type: application/json)

Figura 3.3: Respuesta y códigos HTTP documentados para el endpoint /buscar.

3.1.4.3. Endpoint /enfermedades_lista

Este endpoint consulta los nombres distintos almacenados en el campo de enfermedad y los devuelve sin repeticiones. La respuesta se utiliza para generar las tarjetas y completar los selectores de la aplicación.

- **Método HTTP:** GET.
- **Parámetros:** no requiere parámetros.
- **Respuesta:** lista de cadenas de texto con los nombres de las enfermedades disponibles.
- **Código HTTP:** 200 OK cuando la consulta se completa correctamente.
- **Posibles errores:** un fallo en la conexión o en la consulta a la base de datos puede generar un error interno del servidor.

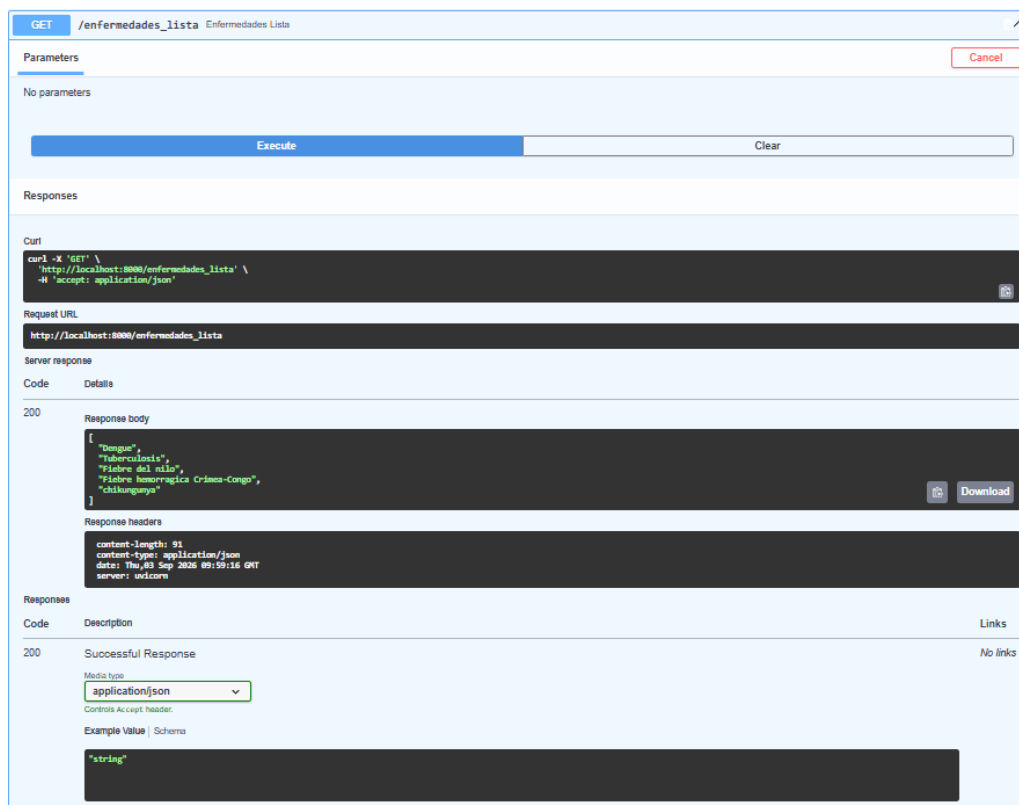


Figura 3.4: Respuesta del endpoint que devuelve la lista de enfermedades.

3.1.4.4. Endpoint /anyos

El endpoint /anyos obtiene los años distintos existentes en la base de datos y los ordena de forma ascendente antes de devolverlos al frontend.

- **Método HTTP:** GET.
- **Parámetros:** no requiere parámetros.
- **Respuesta:** lista ordenada de valores numéricos con los años disponibles.
- **Código HTTP:** 200 OK cuando la consulta se completa correctamente.
- **Posibles errores:** un fallo de acceso a la base de datos puede producir un error interno del servidor.

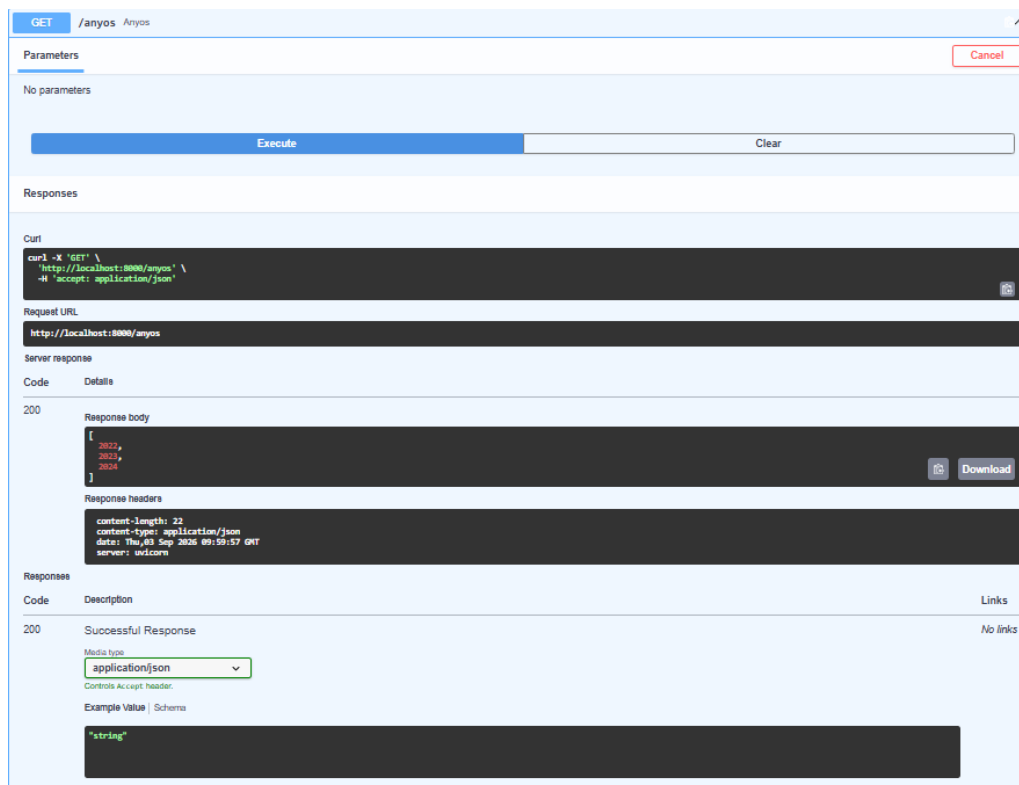


Figura 3.5: Respuesta del endpoint que devuelve los años disponibles.

3.1.4.5. Endpoint /informacion/{enfermedad}

Este endpoint busca en la carpeta `datos` el archivo JSON correspondiente a la enfermedad solicitada. Para localizarlo, abre los archivos con codificación UTF-8 y compara el campo `nombre` con el valor recibido en la URL, ignorando las diferencias entre mayúsculas y minúsculas y los espacios situados en los extremos.

- **Método HTTP:** GET.
- **Parámetro obligatorio:** `enfermedad`, de tipo cadena de texto, incluido en la ruta.
- **Respuesta:** objeto JSON con la descripción, los síntomas, la transmisión, la prevención, el tratamiento y los demás campos informativos de la enfermedad.
- **Código HTTP:** 200 OK cuando se procesa la petición.
- **Posibles errores:** los archivos dañados o que no puedan leerse se ignoran y la búsqueda continúa. Si no se encuentra ninguna coincidencia, la implementación actual devuelve el objeto `error: Enfermedad no encontrada` con código 200, en lugar de generar un código 404.

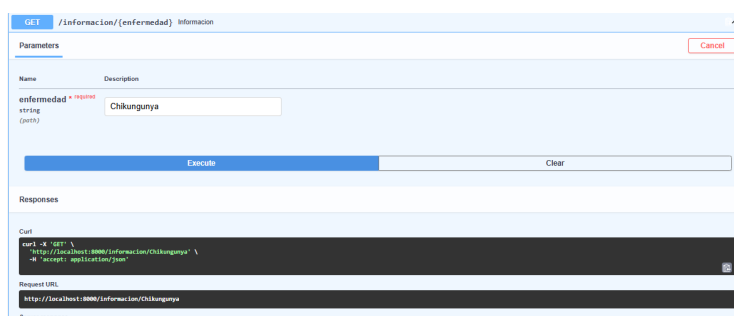


Figura 3.6: Consulta de la información detallada de una enfermedad.

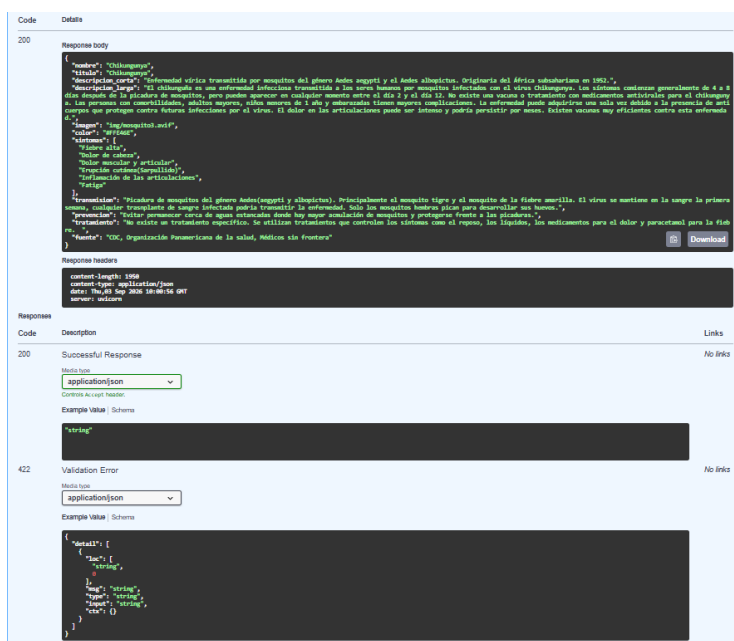


Figura 3.7: Estructura de la respuesta del endpoint de información de enfermedades.

3.1.4.6. Endpoint /genero/{enfermedad}/{anyo}

Este endpoint consulta el registro nacional de la enfermedad y el año indicados. La función selecciona la comunidad **España** y el rango de edad **Total** para evitar que los casos se contabilicen más de una vez.

- **Método HTTP:** GET.
- **Parámetros obligatorios:** enfermedad, de tipo cadena de texto, y anyo, de tipo entero.
- **Respuesta:** objeto JSON con los campos **hombres** y **mujeres**, convertidos a valores enteros cuando existen.
- **Códigos HTTP:** 200 cuando se procesa la consulta correctamente y 422 si el año no es un entero válido.
- **Posibles errores:** si no existe un registro coincidente o los campos están vacíos, se devuelve el valor **null** para ambos géneros. Los errores de la base de datos pueden provocar una respuesta interna del servidor.

The screenshot shows a REST client interface for the endpoint `GET /genero/{enfermedad}/{anyo}`. The parameters section has two fields: `enfermedad` (string, required) with the value `Dengue`, and `anyo` (integer, required) with the value `2024`. Below the parameters are buttons for `Execute` and `Clear`. The `Responses` section shows a successful response with status `200` and a JSON body: `{ "hombres": 512, "mujeres": 305 }`. The request URL is `http://localhost:8080/genero/Dengue/2024`.

Figura 3.8: Introducción de los parámetros del endpoint de distribución por género.

The screenshot shows the response for the endpoint. The status is `200` and the response body is `{ "hombres": 512, "mujeres": 305 }`. The response headers are `content-length: 29`, `content-type: application/json`, `date: Thu, 01 May 2024 18:45:32 GMT`, and `server: uelcom`. Below the response is a table of responses with two rows: `200` (Successful Response) and `422` (Validation Error). The `422` response has a JSON body: `{ "detail": { "loc": [ "string", "msg": "string", "type": "string", "value": "string", "ctx": {} ] } }`.

Figura 3.9: Respuesta y posibles códigos del endpoint de distribución por género.

3.1.4.7. Endpoint /edad/{enfermedad}/{año}

El endpoint de edad consulta los registros nacionales que contienen un rango de edad. Después recorre los resultados, descarta los campos sin número de casos y construye una lista con cada rango y su cantidad correspondiente.

- **Método HTTP:** GET.
- **Parámetros obligatorios:** enfermedad, de tipo cadena de texto, y año, de tipo entero.
- **Respuesta:** lista de objetos JSON con los campos `rango_edad` y `casos`.
- **Códigos HTTP:** 200 cuando se procesa la consulta correctamente y 422 si el año no es un entero válido.
- **Posibles errores:** si no existen datos de edad para la selección, se devuelve una lista vacía. Los errores durante la consulta pueden producir un error interno del servidor.

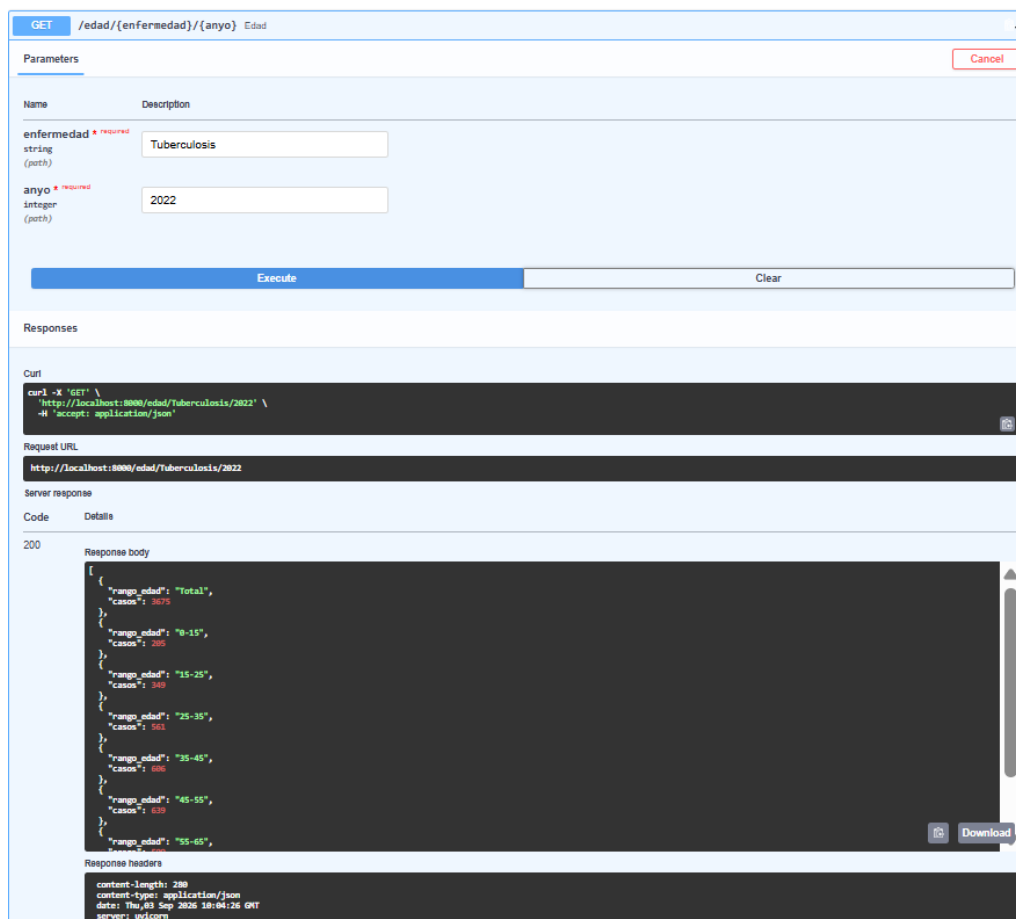


Figura 3.10: Consulta y respuesta del endpoint de distribución por edad.

Response	Description	Links
200	Successful Response Media type: application/json Content Accept Header Example Value: Schema <pre>"string"</pre>	No links
422	Validation Error Media type: application/json Example Value: Schema <pre>{ "meta": { "type": { "string", "string", "string", "string", "string", "string" } }, "data": {} }</pre>	No links

Figura 3.11: Códigos y estructura de respuesta del endpoint de distribución por edad.

3.1.4.8. Endpoint /obtener_todo

Este endpoint recupera todos los registros almacenados en la tabla de enfermedades. Se utiliza principalmente para generar la descarga completa de los datos epidemiológicos en formato CSV.

- **Método HTTP:** GET.
- **Parámetros:** no requiere parámetros.
- **Respuesta:** lista de objetos JSON con todos los registros epidemiológicos de la base de datos.
- **Código HTTP:** 200 OK cuando se recuperan los datos correctamente.
- **Posibles errores:** un problema en la conexión o en la consulta puede generar un error interno del servidor.

GET

/ebitener_todo

Obtainer Todo

Parameters

No parameters

Exclude

Clear

Responses

Curl

```
curl -X GET \
  http://localhost:3000/ebitener_todo/ \
  -H 'accept: application/json'
```

Request URL:

http://localhost:3000/ebitener_todo

Server response

Code

Details

200

Response body

```
{
  "range": 200,
  "timestamp": "2024-10-10T10:10:10.000Z",
  "http_status": 200,
  "message": "Success",
  "data": {
    "id": 1,
    "username": "john_doe",
    "password": "123456",
    "email": "john.doe@example.com",
    "phone": "1234567890"
  }
}
```

Response headers

```
content-length: 4401
content-type: application/json
date: Thu, 01 Sep 2024 10:01:29 GMT
server: Node.js
```

Responses

Code

Description

200

Successful Response

Media type:

application/json

Controls All: Toggle

Example Value: Schema

Links

No links

string

Figura 3.12: Ejecución del endpoint que devuelve todos los registros epidemiológicos.

3.1.4.9. Endpoint /contacto

El endpoint `/contacto` recibe los datos introducidos en el formulario y los valida mediante el esquema `MensajeCrear`. La función `crear_mensaje()` construye el objeto, lo añade a la sesión de SQLAlchemy, confirma la operación y actualiza el registro para devolver los campos generados por la base de datos.

- **Método HTTP:** POST.
- **Cuerpo obligatorio:** objeto JSON con los campos `nombre`, `email`, `asunto` y `mensaje`, todos ellos definidos en `schemas.py`.
- **Respuesta:** objeto JSON basado en `MensajeRespuesta`, que incluye los datos almacenados, el identificador y la fecha de creación.
- **Códigos HTTP:** 201 cuando el mensaje se almacena correctamente y 422 si faltan campos obligatorios o sus valores no superan la validación.
- **Posibles errores:** un fallo durante la inserción o la confirmación de la transacción puede producir un error interno del servidor.

POST /contacto Crear Contacto

Parameters

No parameters

Request body required

application/json

```
{
  "nombre": "Ramona",
  "email": "Ramona@gmail.com",
  "asunto": "Colores",
  "mensaje": "Me gustaría más colores en la página web"
}
```

Execute

Figura 3.13: Cuerpo JSON utilizado para enviar un mensaje mediante el endpoint `/contacto`.

Responses

Code	Description	Links
201	Successful Response	No links
422	Validation Error	No links

Media type: application/json

Controls Accept header

Example Value | Schema

```
{
  "nombre": "string",
  "email": "user@example.com",
  "asunto": "string",
  "mensaje": "string",
  "id": 0,
  "fecha": "2020-09-03T10:06:51.289Z"
}
```

Media type: application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```


Figura 3.14: Respuestas documentadas para el endpoint de contacto.

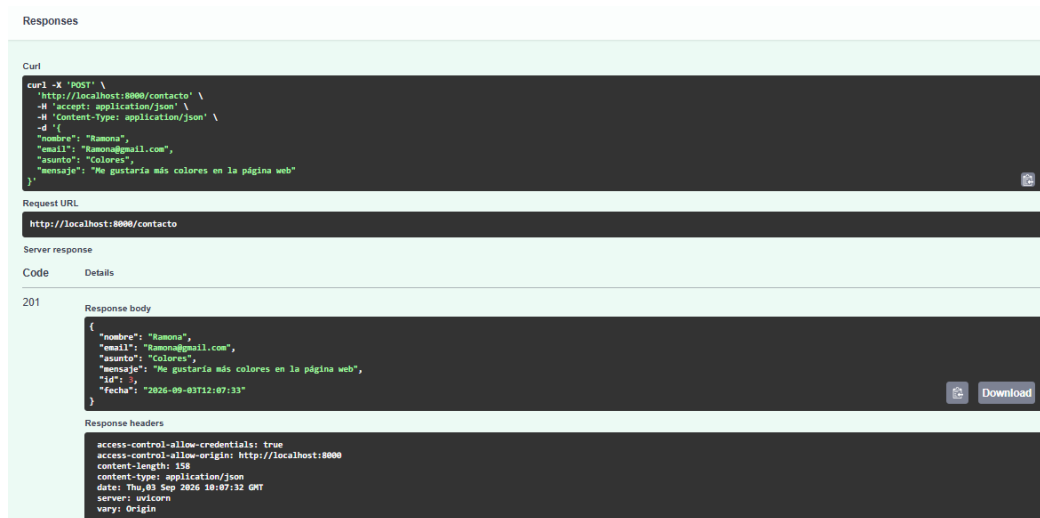


Figura 3.15: Confirmación de la creación y almacenamiento de un mensaje de contacto.

3.2. Desarrollo front-end

3.2.1. HTML, CSS y Bootstrap

HTML (HyperText Markup Language) es el lenguaje utilizado para definir la estructura y el contenido de las páginas web. Se ha utilizado HTML en este proyecto para construir la estructura de las diferentes páginas de la aplicación, definiendo elementos como encabezados, textos, formularios, botones, tablas, tarjetas y contenedores.

CSS (Cascading Style Sheets) se utiliza para definir la apariencia visual de la aplicación. Se emplea para establecer colores, tamaños, bordes, fondos, posiciones y otros aspectos relacionados con el diseño de la interfaz. CSS se integra a la perfección con HTML y JavaScript, lo que permite separar la estructura, la presentación y el comportamiento de la página web.

Hemos implementado dos hojas de estilo CSS prácticamente iguales para cada archivo HTML. Ambas comparten la misma estructura y disposición, pero utilizan colores diferentes. La aplicación cuenta con una opción de accesibilidad que permite alternar entre estos dos estilos para facilitar la navegación a las personas con daltonismo u otros problemas visuales. La hoja de estilo alternativa utiliza tonos blancos, negros y grises para que los usuarios puedan visualizar y comprender la página web sin dificultades.

Bootstrap es un framework de código abierto para el desarrollo frontend.

En el proyecto, Bootstrap se utiliza principalmente mediante sus clases de utilidad y su sistema de rejilla. Por ejemplo, las clases `container`, `row` y `col` permiten organizar los elementos de las páginas y distribuirlos de forma adaptable según las diferentes resoluciones. También se utilizan componentes y clases como `card`, `btn`, `form-select`, `nav` y `table` para simplificar la construcción de diferentes elementos de la interfaz.

En la aplicación, estas tecnologías se utilizan en las diferentes páginas que forman la interfaz, como la página principal, las páginas de información de las enfermedades y el mapa epidemiológico. También se utilizan para crear elementos comunes a las distintas páginas, como el encabezado, el menú lateral y el pie de página.

3.2.2. JavaScript

JavaScript es un lenguaje de programación esencial en la web que permite crear páginas dinámicas e interactivas.

En este proyecto se utiliza principalmente para realizar peticiones a la API, procesar los datos recibidos y modificar el contenido de las páginas web sin necesidad de recargarlas. Se ha elegido JavaScript porque está integrado de forma nativa en los navegadores y permite trabajar directamente con el HTML y el DOM de la página. Frente a otras alternativas que requieren tecnologías e instalaciones adicionales, JavaScript permite implementar de forma sencilla la comunicación entre el frontend y la API mediante peticiones HTTP con `fetch()`. Cuando el usuario realiza una acción, como consultar una enfermedad o seleccionar los filtros del mapa, JavaScript realiza una petición al endpoint correspondiente de FastAPI. Una vez recibida la respuesta, los datos se procesan y se muestran dinámicamente en la interfaz.

No solo hemos utilizado JavaScript para el mapa interactivo de las enfermedades, sino también para crear automáticamente las tarjetas que contienen información complementaria sobre ellas a partir de la API y de los datos adicionales almacenados en archivos JSON. Además, utilizamos JavaScript para recoger la información introducida en el formulario de contacto y enviársela a la API, con el fin de almacenar el mensaje en la base de datos y poder estudiarlo posteriormente.

3.2.2.1. Procesos principales implementados en JavaScript

Dentro del código JavaScript destacan varios procesos encargados de preparar los datos antes de representarlos o descargarlos. Entre ellos se encuentran la normalización de los nombres de las comunidades autónomas, la selección de los valores predominantes del mapa y la generación de los archivos CSV.

Normalización de las comunidades autónomas. Los nombres recibidos desde la API deben coincidir con los utilizados en el archivo GeoJSON para poder asociar cada registro epidemiológico con su territorio. A diferencia de la normalización del archivo `actualizar_datos.R`, *están normalizados en función de un diccionario que cambia los nombres para que*

Después de esta normalización general se aplican equivalencias para aquellos territorios cuyos nombres son diferentes en ambas fuentes. Por ejemplo: *Baleares* → *Illes Balears*, *Madrid* → *Comunidad de Madrid* y *Navarra* → *Comunidad Foral de Navarra*.

Este proceso evita que una diferencia ortográfica o de denominación impida localizar los datos de una comunidad autónoma en el mapa.

Selección del género predominante. Para representar la distribución por género, JavaScript recibe de la API los valores correspondientes a hombres y mujeres. Ambos valores se convierten a formato numérico y se comparan entre sí. Si el número de hombres es mayor o igual que el de mujeres, se selecciona el género masculino; en caso contrario, se selecciona el femenino. Una vez determinado el género predominante, se aplican al mapa su color y su símbolo asociados. En caso de igualdad, el código considera predominante el género masculino debido al uso de la comparación “mayor o igual que”.

Selección del rango de edad predominante. Para determinar el rango de edad predominante, JavaScript recorre los registros recibidos de la API e ignora el registro denominado *Total*. Durante este recorrido conserva el rango que presenta el mayor número de casos. Cuando finaliza la comparación, utiliza el color y el icono asociados a dicho rango para actualizar la representación del mapa. Este cálculo se realiza de acuerdo con la enfermedad y el año seleccionados por el usuario.

Descarga de los datos filtrados. La descarga correspondiente a los filtros utiliza la información almacenada en *datosMapa*, que contiene la última respuesta obtenida con los filtros activos. Antes de generar el archivo, la función comprueba que existan datos y consulta los valores actuales de enfermedad, año y modo de visualización. A continuación, establece las columnas del CSV según se estén mostrando casos notificados, género o rangos de edad.

Una vez seleccionada la estructura, el proceso de descarga sigue estos pasos:

1. Construye el contenido del archivo en formato CSV.
2. Añade la marca BOM para conservar correctamente los caracteres especiales al abrir el archivo.
3. Crea un objeto *Blob* con el contenido generado.
4. Genera una URL temporal mediante *URL.createObjectURL()*.
5. Crea un enlace de descarga, lo activa automáticamente y lo elimina del documento.
6. Libera la URL temporal mediante *URL.revokeObjectURL()*.

El navegador descarga finalmente un archivo cuyo nombre identifica los filtros empleados, por ejemplo, *Dengue_2024_casos.csv*.

Descarga de todos los datos. La descarga completa sigue un flujo diferente, ya que no utiliza únicamente la información representada en el mapa. En este caso, JavaScript realiza una petición al endpoint `/obtener_todo`. FastAPI consulta la base de datos y devuelve todos los registros epidemiológicos en formato JSON. El archivo `descarga.js` recorre la respuesta y genera un CSV con las columnas de enfermedad, año, comunidad autónoma, casos, tasa de notificación, hombres, mujeres, rango de edad y casos asociados a dicho rango. El archivo resultante se descarga con el nombre `Datos_completos.csv`.

3.2.3. Leaflet

Leaflet es una biblioteca de JavaScript de código abierto que sirve para crear mapas interactivos en la web con gran flexibilidad y capacidad de adaptación a distintos tamaños de pantalla [22].

En este proyecto se utiliza para implementar el mapa epidemiológico de España, lo que permite representar visualmente la distribución de los casos notificados de las diferentes enfermedades entre las comunidades autónomas. Se ha elegido Leaflet frente a otras alternativas de representación cartográfica porque es una biblioteca ligera y muy sencilla de integrar con JavaScript. Además, permite trabajar fácilmente con datos geográficos y añadir capas, colores, marcadores y eventos interactivos sin necesidad de tener conocimientos avanzados sobre cartografía y geolocalización.

Junto con GeoJSON, Leaflet permite delimitar visualmente las comunidades autónomas de España y colocar marcadores en cada una de ellas para trabajar posteriormente con los datos obtenidos.

Mediante Leaflet y JavaScript, hemos conseguido representar inicialmente todas las comunidades con un color blanco semitransparente para facilitar su visualización antes de aplicar ningún filtro. El funcionamiento interno consiste en actualizar el mapa dinámicamente a partir de los filtros seleccionados por el usuario. Cuando se seleccionan una enfermedad y un año, JavaScript realiza una petición a la API desarrollada con FastAPI para obtener los datos correspondientes. Estos datos se utilizan posteriormente para modificar la representación del mapa.

Una vez seleccionados los filtros de enfermedad, año y casos notificados, el mapa de España se coloreará comunidad por comunidad según la tasa de notificación y el número de casos registrados. La intensidad del color representará la gravedad de la situación. A su vez, se mostrará un punto que, al igual que los colores, irá cambiando según la tasa de notificación para mostrar rápidamente sus variaciones.

Si se selecciona el filtro de género, el mapa se coloreará con el color de la leyenda asociado al género predominante y mostrará el símbolo correspondiente: femenino (♀) o masculino (♂).

Si se selecciona el filtro de edad, el mapa se coloreará con el color de la leyenda asociado al rango de edad que presente más casos y mostrará un símbolo que represente la evolución de la edad de las personas afectadas.

La mayor ventaja de Leaflet es su flexibilidad. Mediante JavaScript, se pueden

ajustar los valores predeterminados del mapa para mostrar siempre una zona concreta con un nivel de zoom determinado e incluso cambiar de posición regiones enteras. Canarias es una comunidad autónoma de España que, debido a su ubicación geográfica, aparece en el mapa muy alejada de la península ibérica, a diferencia de las Islas Baleares, Ceuta y Melilla. Su posición predeterminada dificultaría que los usuarios pudieran observar los datos recogidos en esta región. Gracias a Leaflet y JavaScript, hemos podido reubicar las islas Canarias junto a la península ibérica y rodearlas con un recuadro. De esta manera, se pueden visualizar todas las comunidades autónomas de España sin necesidad de desplazar el mapa. Para ello, se han modificado las coordenadas del archivo GeoJSON y se han adaptado a la ubicación visual más adecuada.

Otra herramienta que permite utilizar Leaflet es el tooltip. Un tooltip es una pequeña ventana o cuadro de texto emergente que aparece al pasar el cursor o hacer clic sobre un elemento gráfico de la pantalla, como un botón o un icono. En nuestro atlas también funciona sobre el mapa: al pasar el cursor por cada comunidad autónoma, se muestra una pequeña ventana flotante semitransparente con los datos específicos de los filtros seleccionados, de modo que el usuario pueda obtener la información de forma fácil y directa.

3.2.4. JSON y GeoJSON

JSON (JavaScript Object Notation) es un formato de texto ligero y sencillo diseñado para guardar e intercambiar datos.

Cada enfermedad dispone de un archivo JSON con los datos que posteriormente se enviarán al frontend. La estructura contiene campos como **nombre**, **título**, **descripcion_corta**, **descripcion_larga**, **imagen**, **sintomas**, **transmision**, **prevencion**, **tratamiento** y **fuentes**. También se incluye el campo **color**, utilizado para asociar a cada enfermedad un color y diferenciarlas mejor. Las tarjetas se crean automáticamente mediante la API y el archivo vinculado a cada una de ellas proporciona toda la información complementaria, automatizando el proceso en la medida de lo posible.

Para vincular cada archivo JSON con su enfermedad, el fichero recibe el mismo nombre o abreviado de la enfermedad, seguido de la extensión **.json**. Además, JavaScript identifica su contenido mediante el campo **nombre** del propio archivo y comprueba que coincida con la enfermedad solicitada.

La Figura 3.16 muestra un ejemplo de la organización de estos archivos mediante el JSON correspondiente a la fiebre del Nilo occidental.

```
{
  "nombre": "Fiebre del nilo",
  "titulo": "Fiebre del nilo Occidental",
  "descripcion_corta": "Enfermedad vírica transmitida por mosquitos del género Culex. Originaria de África",
  "descripcion_larga": "Es una enfermedad Emergente en España que suele detectarse en agosto o septiembre",
  "imagen": "img/mosquito_azul.jpg",
  "color": "#87E6FF",
  "síntomas": [
    "Dolor abdominal",
    "Dolor de cabeza",
    "Dolor muscular",
    "Erupción cutánea(Sarpullido)",
    "Fiebre",
    "Dolor de garganta",
    "Náuseas, vómitos y diarrea",
    "Inflamación de ganglios linfáticos",
    "Falta de apetito",
    "Debilidad muscular",
    "Rigidez cuello",
    "Confusión o pérdida del conocimiento"
  ],
  "transmisión": "Picadura de mosquitos del género Culex.",
  "prevención": "Evitar permanecer cerca de aguas estancadas donde hay mayor acumulación de mosquitos y prot",
  "tratamiento": "No existe un tratamiento específico. Consultar a un médico y tratamientos que disminuyan",
  "fuente": "MedlinePlus, CDC,semFYC"
}
```

Figura 3.16: Estructura del archivo JSON correspondiente a la fiebre del Nilo occidental.

Estos archivos se consultan mediante el endpoint `/informacion/{enfermedad}` de la API. FastAPI localiza el archivo correspondiente, lee su contenido y lo devuelve al frontend en formato JSON. Posteriormente, JavaScript recibe estos datos y los utiliza para generar dinámicamente las tarjetas de las enfermedades y las páginas de información detallada.

El uso de JSON permite mantener esta información separada del código de la aplicación, de manera que se pueda modificar o ampliar la información de una enfermedad sin tener que alterar el código JavaScript o Python. Además, es un formato sencillo, ligero y fácilmente interpretable tanto por Python como por JavaScript.

GeoJSON es un formato de código abierto basado en JSON para representar datos geográficos y espaciales (puntos, líneas y polígonos) junto con sus atributos.

Utilizamos un archivo GeoJSON para la representación del mapa epidemiológico. Este documento contiene las geometrías de las diferentes comunidades autónomas de España. Cada comunidad se representa mediante una geometría geográfica, que permite a Leaflet dibujarla correctamente sobre el mapa [14].

El archivo completo contiene una gran cantidad de coordenadas, por lo que la Figura 3.17 muestra únicamente un fragmento de su estructura. En él se observan las propiedades asociadas a una comunidad autónoma y parte de las coordenadas que forman su geometría.

```
"properties": {
  "geo_point_2d": {
    "lon": -4.57573069129274,
    "lat": 37.4632845166749
  },
  "year": "2022",
  "acom_code": "01",
  "acom_name": "Andalucía",
  "acom_area_code": "ESP",
  "acom_type": "autonomous communities",
  "acom_name_local": null,
  "acom_iso3166_code": "AN"
},
{
  "type": "Feature",
  "geometry": {
    "coordinates": [
      [
        [1.188330903, 38.863197137078],
        [1.19103036, 38.863094829078],
        [1.192446476, 38.864062873078],
        [1.194581322, 38.863501212078],
        [1.195917077, 38.864458960078],
        [1.197532341, 38.863934267078],
        [1.199170415, 38.864788519078],
        [1.202045968, 38.864102986078],
        [1.202586522, 38.866436036078],
        [1.203655226, 38.867020151078],
        [1.203369061, 38.868772807078],
        [1.201634994, 38.870349985078],
```

Figura 3.17: Fragmento de la estructura del archivo GeoJSON utilizado por el mapa.

Además de la geometría, cada elemento del archivo GeoJSON contiene características asociadas a la comunidad, como su nombre. Estas propiedades permiten relacionar posteriormente cada territorio con los datos epidemiológicos ya normalizados obtenidos de la API. GeoJSON es un gran aliado de Leaflet para cargar, modificar y representar visualmente los datos seleccionados por el usuario.

Capítulo 4

Funcionamiento

En este capítulo se describe el funcionamiento de la página web y se presentan los principales contenidos y elementos de navegación que ofrece al usuario.

4.1. Página de inicio

El primer elemento que se visualiza en la página web es la cabecera que muestra el título del tema principal y las características generales de la aplicación. La página de inicio cuenta con una portada que introduce de forma general el contenido del sitio web y permite al usuario conocer su finalidad.



Figura 4.1: Cabecera y portada de la página de inicio.

Más adelante se incluye una breve explicación sobre las enfermedades emergentes y reemergentes, así como sobre la importancia de conocerlas y disponer de información detallada acerca de su evolución y sus consecuencias.



Figura 4.2: Contenido introductorio sobre las enfermedades emergentes y reemergentes.

Por último, la página de inicio presenta un listado de tarjetas con información sobre las enfermedades estudiadas. Cada tarjeta contiene el nombre de la enfermedad, una imagen representativa y una breve descripción. Al pulsar el botón *Saber más*, el usuario accede a una página con información detallada sobre la enfermedad seleccionada. Cada enfermedad tiene un color asociado que se puede observar en tanto en el botón de saber más como en el título de la enfermedad dentro de su página detallada.

Listado de enfermedades

Enfermedades principales

Consulta información sobre cada enfermedad y conoce sus principales características.

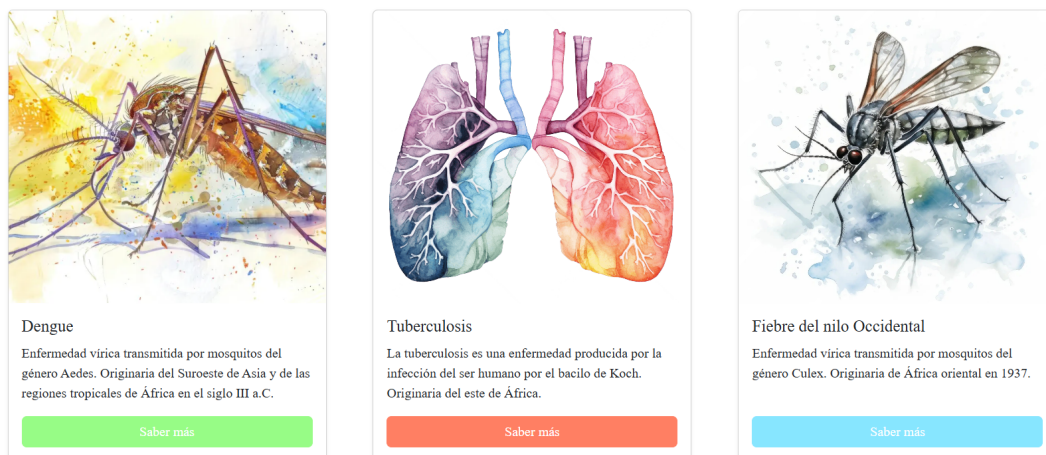


Figura 4.3: Listado de tarjetas de las enfermedades estudiadas.

4.2. Información detallada de las enfermedades

La página individual de cada enfermedad contiene una descripción general más detallada, los síntomas que provoca, sus vías de transmisión, los métodos de prevención, los posibles tratamientos y la fuente de la información recopilada. Estas tarjetas se generan automáticamente a partir de la lista de enfermedades disponible en la base de datos. La API lee la información complementaria almacenada en el archivo JSON correspondiente y la envía al frontend, que genera dinámicamente el contenido de la página para cada enfermedad.



Tuberculosis

La tuberculosis es una enfermedad grave de las más antiguas de la humanidad que afecta principalmente los pulmones. Es una enfermedad reemergente debido a que la bacteria ha mutado y desarrollado resistencia a los fármacos que curaban sus síntomas. Los microorganismos que causan la tuberculosis son un tipo de bacteria. Las personas con VIH/SIDA y otras personas que tienen un sistema inmunitario debilitado corren un mayor riesgo de contagiarse y presentar daños más graves. No toda persona infectada con el bacilo de Koch (*Mycobacterium tuberculosis*) desarrolla tuberculosis. La gran mayoría de las personas sufren síntomas leves idénticos a los de una infección respiratoria viral. Del 10% de personas que acaban padeciendo tuberculosis, el 5% se desarrolla en los primeros meses y el otro 5% se desarrolla tardíamente, incluso pasadas varias décadas. España es uno de los países occidentales con incidencia más elevada de tuberculosis. No hay una vacuna que la erradique por completa, pero la que sanaba los síntomas más graves desde hace años empieza a perder efecto con las nuevas variantes.

Síntomas

- Tos que dura 3 semanas o más. Puede incluir mocos o sangre
- Dolor en el pecho o dolor al respirar o toser
- Pérdida de peso involuntaria
- Fatiga
- Fiebre
- Sudoraciones nocturnas
- Escalofríos
- Pérdida del apetito

Figura 4.4: Página de información detallada de una enfermedad.

4.3. Elementos comunes y opciones de accesibilidad

Todas las páginas disponen de un pie de página que contiene información básica sobre el proyecto, las instituciones y las personas implicadas, además de enlaces que permiten acceder rápidamente a los principales apartados del sitio web.

En el lateral de la pantalla se encuentra el menú desplegable, representado inicialmente mediante una flecha situada dentro de un recuadro fijo. Al pulsarla, se muestran los enlaces de navegación y las opciones de accesibilidad. El usuario puede modificar el tamaño de los elementos de la interfaz, como el texto, las imágenes y el mapa, entre otros. Los tamaños disponibles son normal, grande y muy grande, de modo que cada usuario pueda ajustar la visualización según sus preferencias. También puede activar el modo de color en blanco y negro, que modifica todos los elementos excepto las imágenes para facilitar el acceso y la visualización a las personas con dificultades para distinguir determinados colores o con otros problemas de visión. Las preferencias seleccionadas se conservan al cambiar de página.

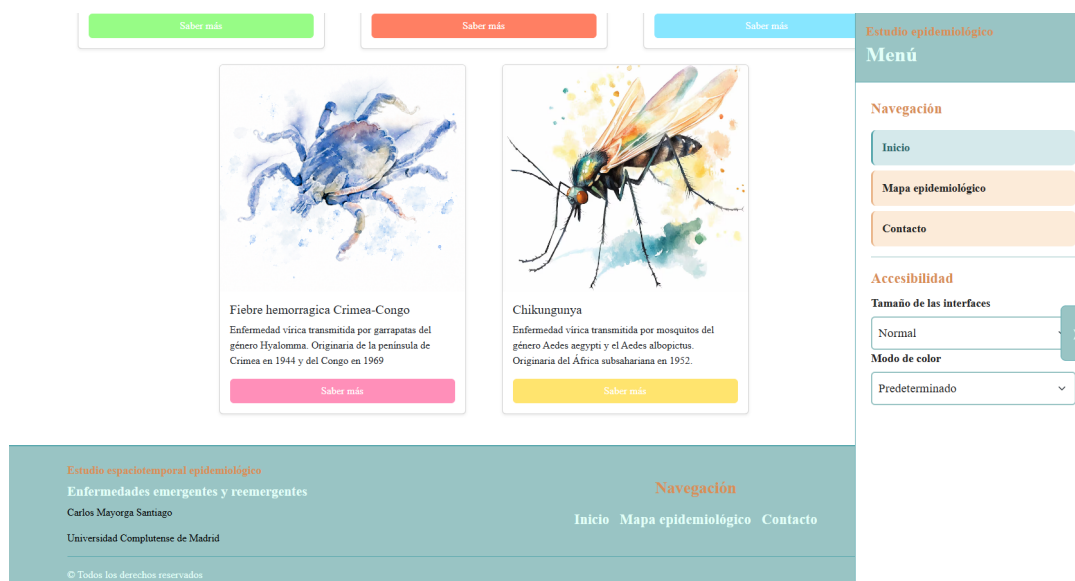


Figura 4.5: Menú lateral, opciones de accesibilidad y pie de página.

La siguiente figura muestra la página de inicio con el modo de color en blanco y negro y el tamaño de la interfaz ampliado.



Figura 4.6: Página de inicio con el modo en blanco y negro y la interfaz ampliada.

4.4. Mapa epidemiológico

La página `mapa.html` constituye uno de los apartados principales de la aplicación. En primer lugar, presenta una descripción general del mapa epidemiológico y de la información que se puede consultar. A continuación, ofrece una explicación más detallada de su funcionamiento y de la interpretación de sus elementos, con el fin de que cualquier usuario pueda interactuar con el mapa y comprender correctamente los datos representados.

La página está diseñada para que los usuarios de España y de otros países puedan consultar información sobre las enfermedades estudiadas de forma clara y sencilla. Por este motivo, resulta especialmente importante explicar el funcionamiento del mapa, facilitar la interpretación de sus resultados y que su visualización sea sencilla y clara.



Figura 4.7: Introducción y explicación del funcionamiento del mapa epidemiológico.

4.4.1. Selección de filtros y estado inicial

Tras la explicación inicial, se muestran el mapa y los filtros disponibles. El usuario debe seleccionar una enfermedad, un año y el tipo de información que desea visualizar antes de pulsar el botón *Buscar*.

En su estado inicial, las comunidades autónomas aparecen delimitadas mediante un color blanco translúcido. Aunque el usuario puede desplazar el mapa y modificar el nivel de zoom, la vista predeterminada está configurada para mostrar todas las comunidades autónomas sin necesidad de realizar ningún ajuste. Las islas Canarias se representan dentro de un recuadro próximo a la Península para facilitar su visualización.

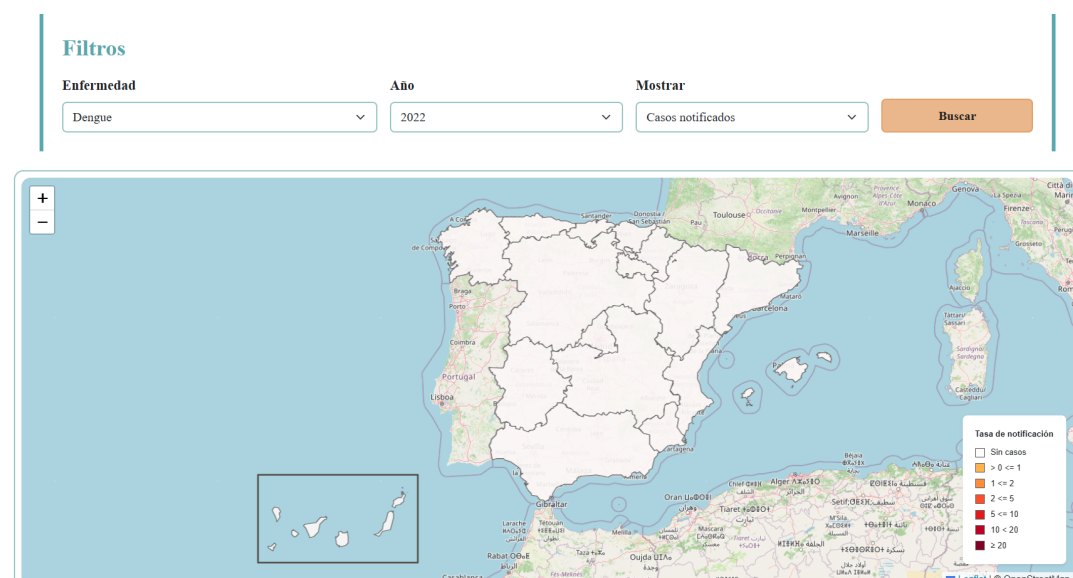


Figura 4.8: Filtros disponibles y estado inicial del mapa epidemiológico.

4.4.2. Representación de los casos notificados

La siguiente figura muestra el resultado obtenido al seleccionar dengue, el año 2023 y la opción de casos notificados. La representación del mapa cambia según la enfermedad, el año y el tipo de información elegidos.

Las comunidades autónomas que presentan casos notificados se colorean según los casos y su tasa de notificación. La escala cromática permite interpretar la magnitud de los valores mediante la leyenda situada en la esquina inferior derecha. Además, se muestran círculos rojos cuyo tamaño varía en función de la tasa de notificación, lo que permite identificar rápidamente las regiones más afectadas.

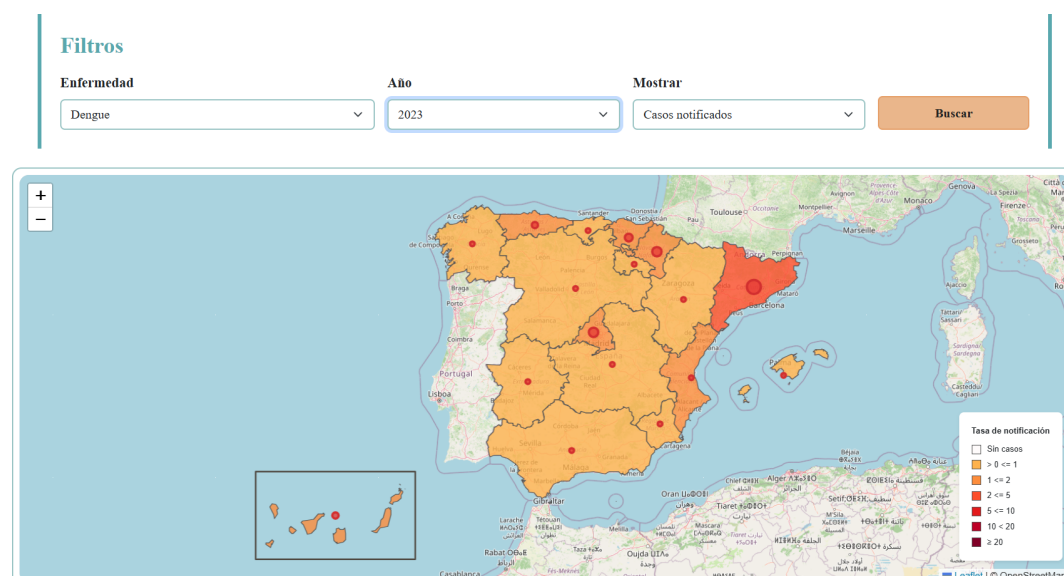


Figura 4.9: Representación de los casos notificados de dengue durante el año 2023.

4.4.3. Representación del género predominante

El mapa no solo permite visualizar los casos notificados, sino también consultar qué género presenta un mayor número de casos para una enfermedad y un año determinados. Al seleccionar la opción de género, cada comunidad autónoma se colorea según el género predominante, utilizando los colores indicados en la leyenda. Además, se incorpora el símbolo correspondiente al género en cada territorio.

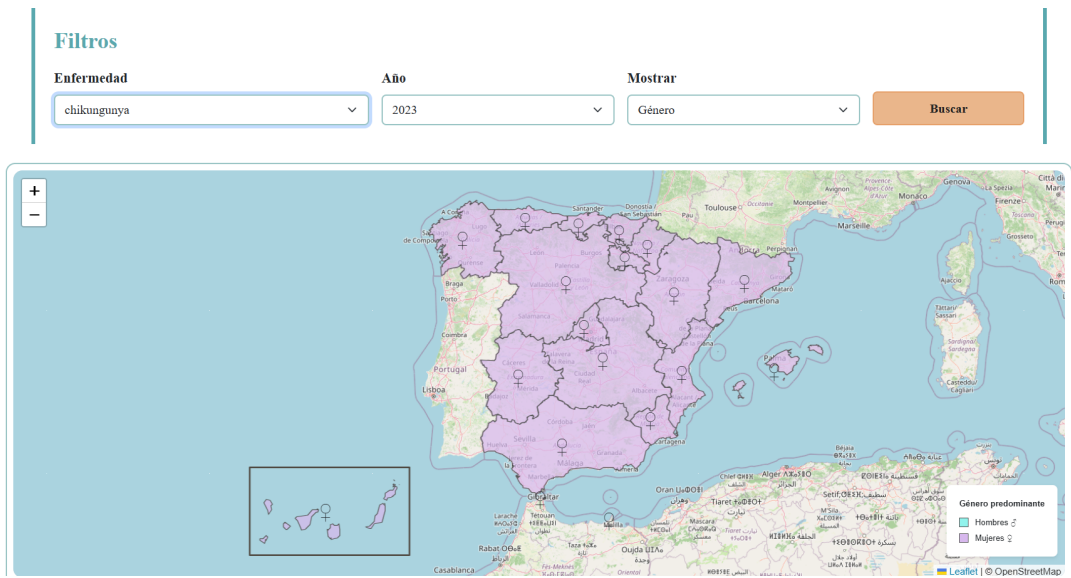


Figura 4.10: Representación del género predominante en los casos de chikungunya durante el año 2023.

4.4.4. Representación del rango de edad predominante

Por último, el mapa permite consultar el rango de edad que presenta un mayor número de casos para la enfermedad y el año seleccionados. Cada comunidad autónoma se colorea con el tono asociado al rango predominante en la leyenda y muestra un pictograma representativo de dicho grupo de edad.

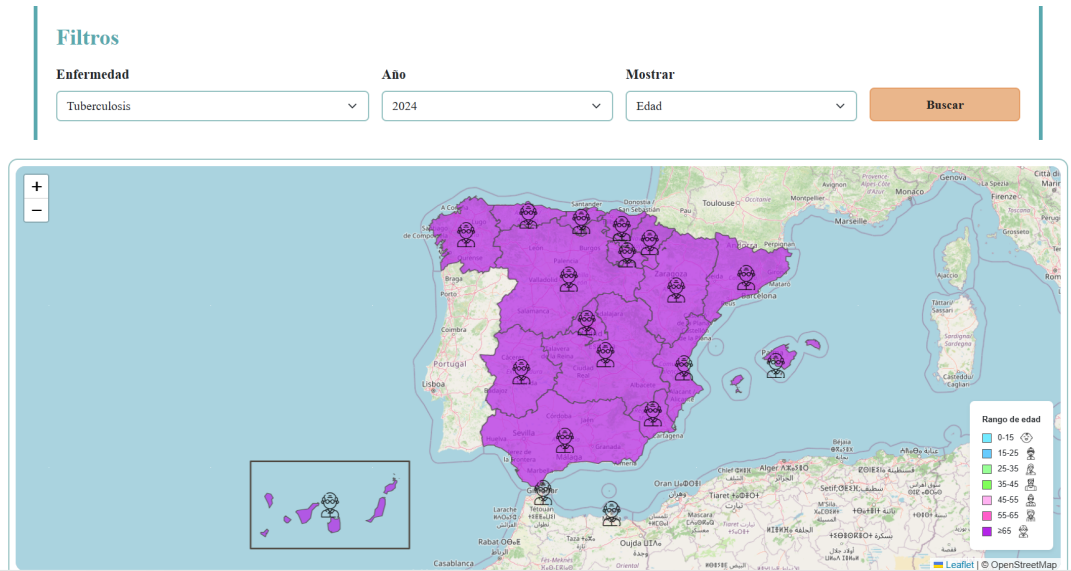


Figura 4.11: Representación del rango de edad predominante en los casos de tuberculosis durante el año 2024.

Para completar la información representada en el mapa, al pasar el cursor sobre una comunidad autónoma se muestra una pequeña ventana emergente con los datos recopilados de acuerdo con los filtros seleccionados más detallados.

El uso conjunto de colores y pictogramas facilita la interpretación de los resultados y permite que la información llegue a un público más amplio. Los símbolos proporcionan una alternativa visual para aquellas personas que puedan tener dificultades para distinguir o asociar los colores utilizados en el mapa.

4.4.5. Descarga de datos

La página del mapa también permite descargar los datos epidemiológicos en formato CSV. Esta función ofrece al usuario la posibilidad de guardar la información en su dispositivo para analizarla con mayor detenimiento o utilizarla en otros proyectos relacionados.



Figura 4.12: Opciones disponibles para la descarga de datos en formato CSV.

Existen dos opciones de descarga:

- **Descargar CSV:** descarga únicamente los datos correspondientes a los filtros seleccionados. Para utilizar esta opción, es necesario pulsar previamente el botón *Buscar* y actualizar la información representada en el mapa.
- **Descargar CSV completo:** descarga todos los registros epidemiológicos almacenados en la base de datos, independientemente de los filtros seleccionados.

Después de pulsar uno de estos botones, el navegador descarga automáticamente un archivo CSV con los datos solicitados.

4.5. Contacto y sugerencias

La página `contacto.html` proporciona al usuario diferentes medios para comunicarse con los responsables de la aplicación y enviar mensajes o sugerencias relacionadas con el proyecto.

En primer lugar, se presenta una portada que explica la finalidad de este apartado. A continuación, se muestran los datos generales del responsable de la página web y diferentes medios de contacto directo, como el correo electrónico y el teléfono.



Figura 4.13: Portada y datos disponibles en la página de contacto.

4.5.1. Formulario de contacto

La página también incluye un formulario que permite enviar mensajes o sugerencias destinadas a mejorar la aplicación. Estas aportaciones pueden comunicar errores que deban corregirse, proponer mejoras de accesibilidad, solicitar la incorporación de nuevas enfermedades o sugerir nuevos filtros y funcionalidades para el mapa y la página de inicio, entre otras posibilidades.

Para completar el formulario, el usuario debe introducir obligatoriamente su nombre, una dirección de correo electrónico, el asunto y el contenido del mensaje. La dirección de correo permite contactar posteriormente con el usuario y dar una respuesta a su mensaje. Una vez introducidos todos los campos, se debe pulsar el botón *Enviar sugerencia*.

El formulario está centrado en la pantalla y tiene un fondo de color claro. Encabezado: "Tu opinión siempre cuenta" en naranja y "Enviar un mensaje" en gris. Campos: "Nombre completo" con el valor "Margarita", "Correo electrónico" con el valor "Margaret@ucm.es", "Asunto" con el valor "Añadir sarna", y "Mensaje o Sugerencia" con el texto "Me preocupa el brote de sarna que se está dando en el incidente en Ceuta y me gustaría que la añadieseis a la página web y así poder informarme mejor sobre esta enfermedad.". Botón: "Enviar sugerencia" en naranja.

Figura 4.14: Formulario para el envío de mensajes y sugerencias.

Si los datos son válidos y el mensaje se almacena correctamente, la aplicación muestra al usuario un aviso de confirmación.

¡Tu sugerencia ha sido enviada con éxito!

Figura 4.15: Mensaje de confirmación tras enviar correctamente una sugerencia.

Si se produce un problema durante la validación, el envío o el almacenamiento de los datos como poner mal el correo electrónico, la aplicación muestra un mensaje de error para informar al usuario y permitirle intentarlo de nuevo.

El formulario es idéntico al de la figura 4.14, pero con los siguientes datos: "Nombre completo" es "Pepe", "Correo electrónico" es "pepe@gmail", "Asunto" es "Mortalidad", y "Mensaje o Sugerencia" es "Añadir mortalidad!!". Debajo del botón "Enviar sugerencia" hay un mensaje de error en un recuadro rojo: "Ocurrió un error al enviar tu sugerencia. Por favor, inténtalo de nuevo."

Figura 4.16: Mensaje mostrado cuando se produce un error al enviar una sugerencia.

Cuando el mensaje se envía correctamente, la API lo almacena en la tabla `mensajes` de la base de datos con los datos introducidos y la fecha de envío. De esta forma, las sugerencias recibidas pueden consultarse y revisarse posteriormente.

id	nombre	email	asunto	mensaje	fecha
2	Margarita	Margaret@ucm.es	Añadir sarna	Me preocupa el brote de sarna que se está dando en...	2026-08-31 12:20:12

Figura 4.17: Mensaje recibido y almacenado en la tabla `mensajes` de la base de datos.

4.6. Ampliación de la aplicación

La estructura del proyecto permite incorporar nuevas enfermedades y ampliar los filtros disponibles sin modificar por completo la aplicación.

4.6.1. Incorporación de una nueva enfermedad

Para añadir una nueva enfermedad, primero se deben recopilar sus datos epidemiológicos e incorporarlos al archivo `DatosEnfermedades.csv`. A continuación, se ejecuta el script `actualizar_datos.R`, que normaliza la información y calcula las tasas de notificación sin necesidad de modificar su código para cada enfermedad.

El CSV actualizado debe cargarse posteriormente en la base de datos. Esta operación puede realizarse reemplazando los registros existentes por el contenido completo del archivo o insertando únicamente los correspondientes a la nueva enfermedad.

La lista y las tarjetas de enfermedades se cargarán y se actualizarán automáticamente a partir de los datos disponibles. Sin embargo, también será necesario crear el archivo JSON asociado a la nueva enfermedad para proporcionar su información detallada y que su pestaña asociada esté completa. Una vez añadido, los endpoints y el mapa podrán utilizar los nuevos registros siguiendo el mismo funcionamiento que con las enfermedades existentes.

4.6.2. Incorporación de un nuevo filtro

Para incluir un filtro adicional, como la tasa de mortalidad, primero se deben añadir sus datos al CSV y a la estructura correspondiente de la base de datos. Después, se debe crear o adaptar un endpoint que consulte esta información.

Por último, se incorpora la nueva opción en la interfaz y se desarrolla en `mapa.js` la función encargada de solicitar los datos, procesarlos y representarlos. También debe actualizarse la leyenda para que el usuario pueda interpretar correctamente el nuevo filtro.

4.7. Pruebas

Las siguientes tablas resumen las pruebas y los resultados comunicados por el autor. El estado *OK* indica que el resultado obtenido coincide con el esperado.

4.7.1. Pruebas de los requisitos funcionales

La Tabla 4.1 recoge las pruebas asociadas a todos los requisitos de la Tabla 2.1, incluyendo casos de uso válidos y entradas incorrectas.

Tabla 4.1: Tabla de trazabilidad de requisitos funcionales.

ID	Requisito	Prueba	Resultado esperado	Resultado obtenido	Estado
P01	RF-01	GET /enfermedades_lista.	Lista no vacía y sin duplicados.	5 enfermedades.	OK
P02	RF-02	Seleccionar dengue y 2024.	Actualización del mapa según los filtros.	Mapa actualizado.	OK
P03	RF-03	Consultar los casos de dengue de 2023.	Colores y círculos según los datos.	Mapa con colores y círculos por comunidad.	OK
P04	RF-04	Seleccionar género para chikungunya de 2023.	Color y símbolo del género predominante.	Mapa coloreado con el símbolo correspondiente.	OK
P05	RF-05	Seleccionar edad para tuberculosis de 2024.	Color e icono del rango con más casos.	Mapa coloreado con el icono correspondiente.	OK
P06	RF-06	Descargar los datos filtrados.	CSV con los datos seleccionados.	CSV descargado con los datos del mapa.	OK
P07	RF-07	Enviar un mensaje válido y otro con correo incorrecto.	Aceptación del válido y rechazo del inválido.	Mensaje de éxito o error.	OK
P08	RF-08	Ejecutar <code>actualizar_datos.r.</code>	Actualización de las tasas con la población del INE.	CSV actualizado generado.	OK
P09	RF-09	Consultar la tabla <code>mensajes</code> tras un envío.	Mensaje guardado con sus datos.	Mensaje almacenado en la tabla.	OK
P10	RF-10	Pulsar <i>Saber más</i> en una tarjeta.	Información de la enfermedad seleccionada.	Página con la información detallada.	OK
P11	RF-11	Activar el modo en blanco y negro.	Cambio de color de la interfaz.	Interfaz en blanco y negro, salvo las imágenes.	OK
P12	RF-12	Cambiar el tamaño de la interfaz.	Elementos ajustados al tamaño elegido.	Interfaz ampliada según la opción seleccionada.	OK
P13	RF-13	Descargar el CSV completo.	Archivo con todos los datos.	Descarga de <code>Datos_completos.csv</code> .	OK

ID	Requisito	Prueba	Resultado esperado	Resultado obtenido	Estado
P14	RF-14	GET /anyos.	Lista de años disponibles.	Lista de años devuelta por la API.	OK
P15	RF-15	Cargar las tarjetas y las páginas de enfermedades.	Contenido generado desde los JSON.	Tarjetas y páginas generadas automáticamente.	OK
P16	RF-16	Revisar los avisos tras enviar el formulario.	Aviso de confirmación o error.	Mensaje correspondiente al resultado del envío.	OK

4.7.2. Pruebas de los requisitos no funcionales

La Tabla 4.2 recoge las comprobaciones de calidad y organización del sistema.

Tabla 4.2: Tabla de trazabilidad de requisitos no funcionales.

ID	Requisito	Prueba	Resultado esperado	Resultado obtenido	Estado
P17	RNF-01	Probar la aplicación en distintos navegadores.	Funcionamiento correcto.	Funcionamiento correcto en Chrome y Microsoft Edge.	OK
P18	RNF-02	Revisar distintos tamaños de pantalla.	Interfaz adaptada a la pantalla.	Interfaz ajustada correctamente a los tamaños probados.	OK
P19	RNF-03	Utilizar el menú y la accesibilidad.	Navegación sencilla y opciones accesibles.	Menú y cambios de tamaño y color disponibles.	OK
P20	RNF-04	Comprobar la rapidez de respuesta.	Respuesta sin esperas excesivas.	Consultas y actualizaciones sin esperas excesivas.	OK
P21	RNF-05	Revisar cómo añadir enfermedades y filtros.	Ampliación sin rehacer la aplicación.	Nuevas enfermedades y filtros incorporados sin rehacer la aplicación.	OK
P22	RNF-06	Comparar los datos del CSV, la base de datos y el mapa.	Datos conservados sin alteraciones.	Datos coincidentes en el CSV, la base de datos y el mapa.	OK
P23	RNF-07	Enviar un correo incorrecto.	Rechazo de los datos inválidos.	HTTP 422.	OK
P24	RNF-08	Cambiar los filtros del mapa.	Actualización sin recargar ni pulsar <i>Buscar</i> .	Mapa actualizado automáticamente al cambiar los filtros.	OK
P25	RNF-09	Revisar el acceso a la base de datos.	Acceso únicamente desde el backend.	Frontend conectado a los datos mediante la API.	OK
P26	RNF-10	Revisar la organización de los archivos.	Código separado según su función.	Archivos organizados en módulos.	OK

Análisis

En este capítulo se analizan los datos recopilados sobre las enfermedades emergentes y reemergentes mediante las visualizaciones del mapa epidemiológico. La comparación entre años permite observar la evolución temporal de cada enfermedad, tanto en las comunidades autónomas como en el conjunto de España. No obstante, se trata de un análisis descriptivo: las diferencias mostradas en el mapa permiten identificar patrones, pero no determinan por sí solas sus causas [21, 32].

5.1. Análisis del dengue

El dengue es una enfermedad vírica transmitida por mosquitos del género *Aedes*. Su seguimiento resulta especialmente relevante debido a la presencia del mosquito tigre en España y a la posibilidad de que se produzcan casos importados y episodios de transmisión local [24, 25, 30].

5.1.1. Evolución de los casos notificados

Al comparar en el mapa los datos de dengue correspondientes a los años 2022, 2023 y 2024, se observa un aumento del número de casos notificados. El incremento más destacado se produce entre 2023 y 2024. En 2022 ya existía un número de casos que debía tenerse en cuenta y la evolución posterior refuerza la importancia de mantener las medidas de prevención, vigilancia y control. No obstante, el aumento observado no permite afirmar por sí solo que las medidas aplicadas durante esos años fueran insuficientes, ya que para llegar a esa conclusión sería necesario analizar información adicional [3, 4, 6].

Entre las comunidades autónomas representadas, Madrid y Cataluña presentan algunos de los valores más elevados. Una posible explicación es que ambas cuentan con grandes núcleos de población y una elevada movilidad de residentes y visitantes, lo que puede favorecer la llegada y detección de casos importados. En Cataluña destaca especialmente Barcelona, una ciudad con una gran actividad turística y numerosas conexiones nacionales e internacionales.

Otro factor que podría influir en Cataluña es la presencia del mosquito tigre, uno de los vectores capaces de transmitir el dengue. Esta especie se detectó por primera vez en España en Cataluña y posteriormente se extendió por buena parte del litoral mediterráneo. Las condiciones ambientales de estas zonas pueden favorecer

la presencia del mosquito al estar cerca del mar ya que los mosquitos habitan en regiones húmedas [24, 30].

También podría esperarse una presencia importante de casos en Andalucía, Ceuta y Melilla debido a su proximidad y sus conexiones con el continente africano. Sin embargo, los datos representados no muestran en estos territorios valores tan elevados como los de Madrid o Cataluña. Es posible que Ceuta y Melilla apliquen medidas especiales que expliquen sus resultados al encontrarse en África y estar más preparados que en la península.

Por tanto, el mapa permite localizar los territorios con más casos o con tasas superiores y plantear posibles explicaciones, pero no atribuir los resultados a una única causa. Para confirmar estas hipótesis sería necesario diferenciar entre casos importados y autóctonos y disponer de información sobre movilidad, población, presencia del vector y medidas de vigilancia en cada comunidad autónoma.

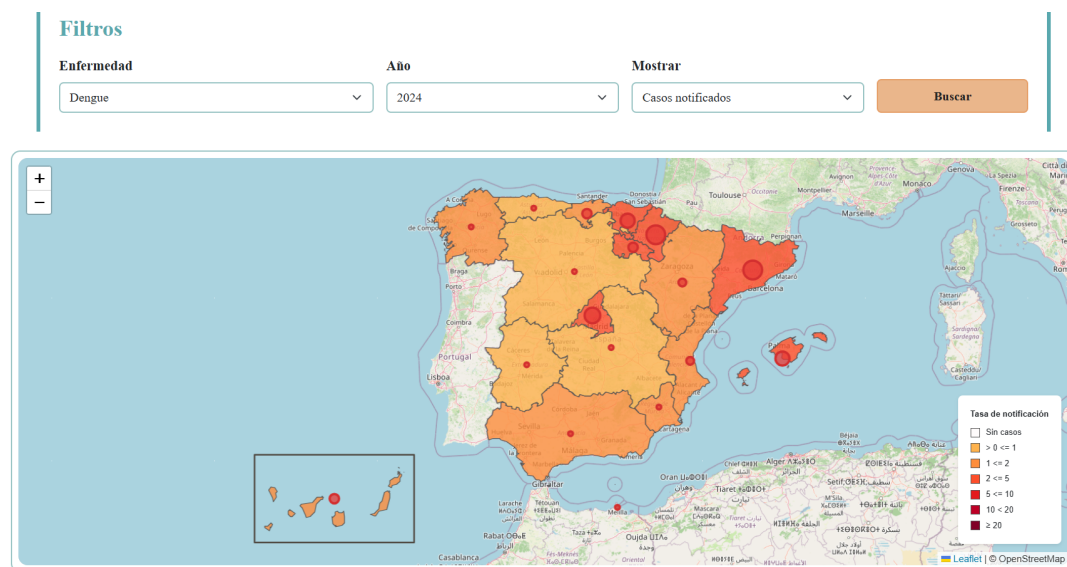


Figura 5.1: Distribución de los casos notificados de dengue durante el año 2024.

5.1.2. Distribución por género

En los datos analizados, el género femenino presenta un mayor número de casos en 2022 y 2024, mientras que el género masculino predomina en 2023. Por tanto, no se observa un patrón constante durante todo el periodo estudiado, aunque las mujeres aparecen como el grupo predominante en dos de los tres años [3, 4, 6].

Una posible explicación podría estar relacionada con las diferencias individuales en la atracción de los mosquitos. Estos insectos localizan a las personas mediante distintos estímulos, entre los que se encuentran el dióxido de carbono expulsado al respirar, el olor corporal, las sustancias presentes en la piel, el calor y determinadas señales visuales, como el color de la ropa. También se suelen mencionar factores como el grupo sanguíneo o la cantidad de vello corporal, aunque los datos representados en este proyecto no permiten comprobar su influencia ni relacionarlos con el predominio de un género.

5.1.3. Distribución por edad

En los años estudiados, el rango de 25 a 35 años aparece como el grupo con mayor número de casos de dengue. Una posible interpretación es que las personas de estas edades suelen presentar una mayor movilidad por motivos laborales, sociales o turísticos y pueden realizar más actividades al aire libre. Estas circunstancias podrían aumentar las oportunidades de exposición a los mosquitos.

Los jóvenes frecuentan jardines, parques, piscinas u otros espacios en los que se encuentra agua y sobre todo de a primera hora del día o al anochecer que es cuando estos mosquitos suelen estar más activos.. Sin embargo, el resultado no implica necesariamente que los mosquitos prefieran a las personas jóvenes, sino que este grupo podría coincidir con determinadas condiciones y hábitos que favorecen el contacto con el vector.

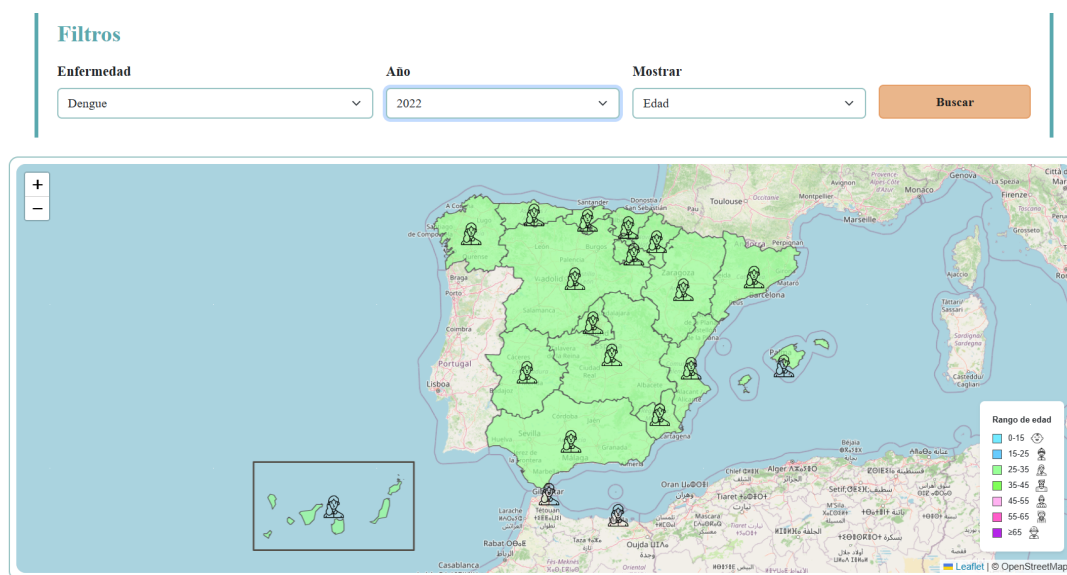


Figura 5.2: Rango de edad predominante en los casos de dengue durante el año 2022.

En conjunto, los datos muestran una evolución ascendente de los casos notificados durante el periodo estudiado y permiten identificar diferencias territoriales, temporales, de género y de edad. Estos resultados indican que la enfermedad no se ha erradicado y su tasa de notificación va en aumento, por lo que es necesario estudiarla más y tomar mayores medidas para prevenirla.

5.2. Análisis de la tuberculosis

La tuberculosis es una enfermedad infecciosa causada por la bacteria *Mycobacterium tuberculosis* y puede considerarse reemergente debido a que, pese a los avances logrados para controlarla, continúa presente y sigue suponiendo un problema de salud pública [9, 13, 29].

5.2.1. Evolución de los casos notificados

Al estudiar los casos y las tasas de notificación de los años disponibles en el mapa, se observa un número elevado de personas afectadas y una tendencia ascendente en la mayoría de las comunidades autónomas. Entre los territorios que presentan más casos y un crecimiento más destacado se encuentran Cataluña, Madrid, Galicia y Andalucía. El elevado número de habitantes de estas comunidades puede influir en la cantidad total de casos registrados, aunque la tasa de notificación permite comparar su impacto teniendo en cuenta las diferencias de población [2, 9, 27, 29].

La tuberculosis se transmite por el aire mediante partículas respiratorias expulsadas por una persona con tuberculosis pulmonar, especialmente al toser, estornudar o hablar. Por este motivo, las interacciones prolongadas en espacios cerrados, concurridos y con poca ventilación pueden favorecer su transmisión [13].

Uno de los resultados que más llama la atención es la situación de Galicia, que alcanza valores destacados junto con Cataluña y Madrid. Una posible explicación podría estar relacionada con su clima lluvioso, que puede hacer que una mayor parte de las actividades y relaciones sociales se desarrollen en espacios interiores. No obstante, esta interpretación tendría que contrastarse con otros factores, como la estructura de edad de la población, las condiciones de las viviendas, la ventilación, la situación socioeconómica y el seguimiento sanitario de los casos.

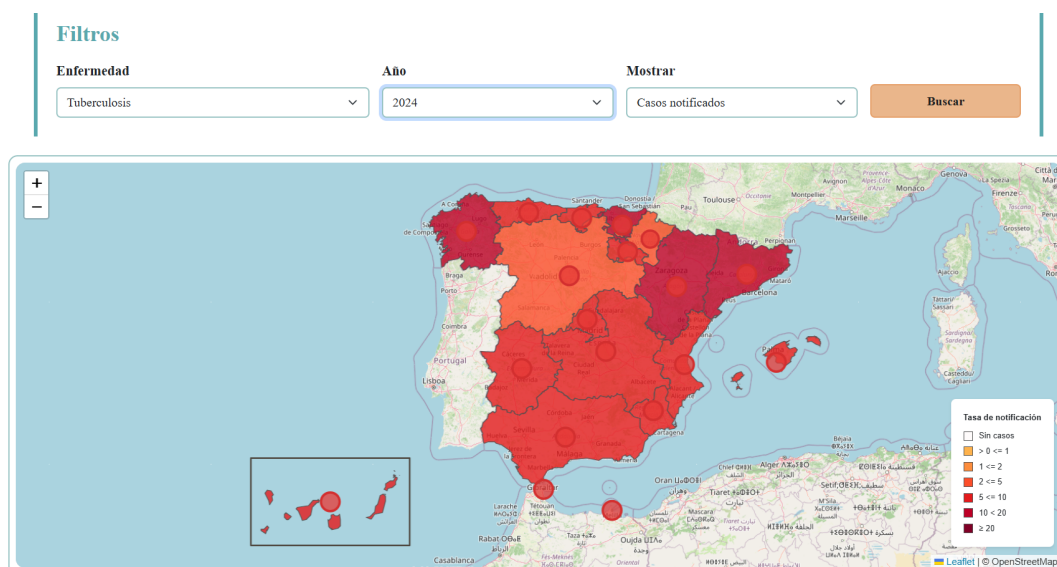


Figura 5.3: Distribución de los casos notificados de tuberculosis durante el año 2024.

La tuberculosis es una de las enfermedades estudiadas desde hace más tiempo y ha estado presente en España durante un periodo mucho mayor que otras enfermedades incluidas en el proyecto. Aunque durante años se consideró que estaba controlada y que su peligrosidad había disminuido, la enfermedad no ha desaparecido. Además, existen cepas resistentes a determinados antibióticos que dificultan y alargan el tratamiento. Esto no significa que la bacteria sea completamente inmune a los medicamentos, ya que la tuberculosis continúa siendo una enfermedad prevenible y curable, pero la resistencia farmacológica obliga a mantener una vigilancia constante y aplicar tratamientos adecuados [9, 13, 29].

Un ejemplo reciente de la importancia de la vigilancia sanitaria se produjo en Ceuta tras la entrada de aproximadamente 70.000 personas procedentes de Marruecos los días 30 y 31 de julio de 2026. Durante la atención a las personas llegadas se detectaron algunos casos de enfermedades transmisibles, incluida la tuberculosis.

Este tipo de acontecimientos imprevistos puede aumentar la presión sobre los servicios sanitarios y exigir una respuesta rápida de identificación, atención y seguimiento. Por ello, disponer de información sobre estas enfermedades y de sistemas de vigilancia preparados proporciona una ventaja importante para responder ante posibles riesgos sanitarios sin que nos pillen desprevenidos.

5.2.2. Distribución por género

En todos los años analizados, el género masculino presenta un mayor número de casos de tuberculosis. La información mostrada en el mapa no permite determinar con exactitud por qué los hombres aparecen más afectados que las mujeres, pero pueden plantearse diferentes hipótesis [9, 27, 29].

Entre las posibles explicaciones se encuentran las diferencias biológicas en la respuesta frente a la infección, una mayor exposición en determinados entornos laborales o sociales, la adopción de menos medidas preventivas, distintos hábitos de interacción o la posibilidad de presentar síntomas más graves que deban ser notificados y revisados por un médico.

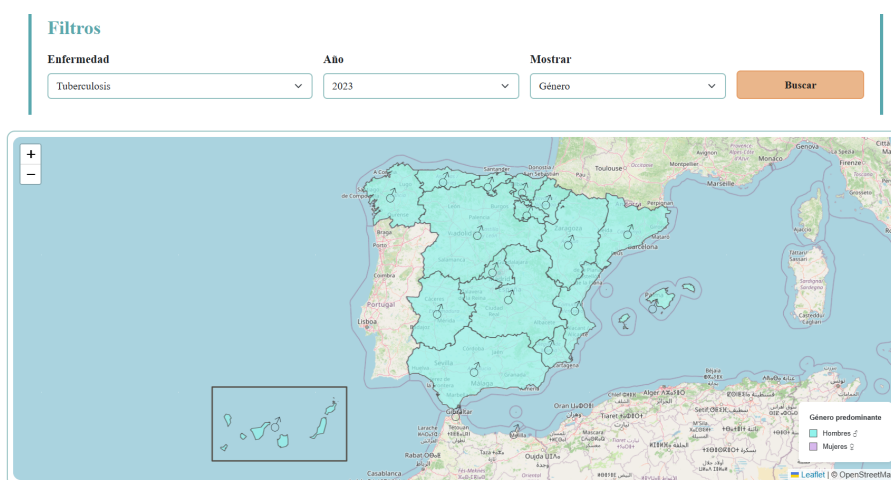


Figura 5.4: Género predominante en los casos de tuberculosis durante el año 2023.

5.2.3. Distribución por edad

En cuanto a la edad, el rango predominante corresponde a las personas mayores de 65 años. Este grupo es el único intervalo abierto de la clasificación utilizada, ya que incluye todas las edades superiores a 65 años, mientras que los demás abarcan periodos más reducidos. Esta diferencia puede contribuir a que reúna un número mayor de casos.

La edad también puede influir debido al debilitamiento del sistema inmunitario y a la posibilidad de que la infección adquirida años atrás permanezca latente y se active posteriormente. El segundo grupo con más casos corresponde a las personas de entre 45 y 55 años, relativamente próximo al anterior dentro de la clasificación por edades y en volumen de casos notificados.

Otra posible explicación es que parte de la población de estos grupos frecuente espacios cerrados y concurridos, como bares, restaurantes, teatros, espectáculos u otros recintos de ocio. Si estos lugares presentan una ventilación insuficiente y se permanece en ellos durante periodos prolongados, pueden favorecer la transmisión de la tuberculosis. No obstante, estos hábitos no pueden considerarse la única causa de la distribución observada.

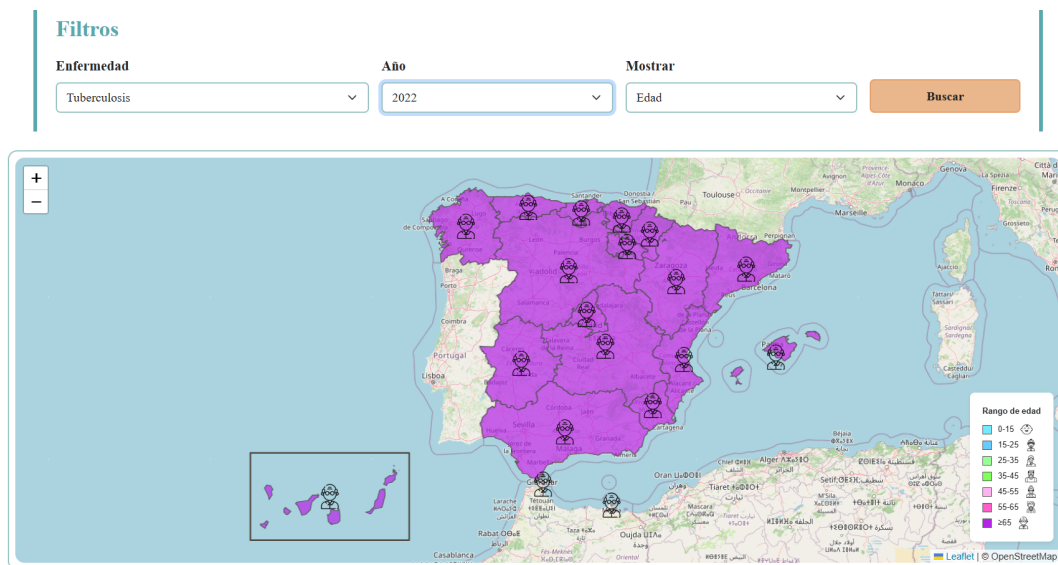


Figura 5.5: Rango de edad predominante en los casos de tuberculosis durante el año 2022.

Los casos estudiados y los conocimientos acumulados sobre esta enfermedad demuestran que la tuberculosis no ha desaparecido y que todavía puede provocar daños graves. Por este motivo, es necesario continuar informando a la población, mantener la vigilancia epidemiológica, favorecer el diagnóstico temprano y aplicar correctamente las medidas de prevención y tratamiento para reducir su impacto en la sociedad.

5.3. Análisis de la fiebre del Nilo occidental

La fiebre del Nilo occidental es una enfermedad vírica transmitida principalmente por la picadura de mosquitos infectados del género *Culex*. En España se considera una enfermedad emergente y, debido al reducido número de casos registrado en algunos años, cualquier cambio en su distribución resulta especialmente relevante para la vigilancia epidemiológica [8, 28].

5.3.1. Evolución de los casos notificados

En 2022, los casos incluidos en el mapa se localizaron únicamente en Andalucía y Cataluña. La presencia de casos en Cataluña podría estar relacionada con factores como la movilidad de personas, la actividad turística de Barcelona y las condiciones ambientales favorables para los mosquitos. Sin embargo, al tratarse habitualmente de casos autóctonos vinculados al ciclo entre aves y mosquitos, los desplazamientos humanos no pueden considerarse por sí solos la causa principal de la transmisión.

En Andalucía, la proximidad al continente africano podría influir especialmente a través de las rutas de aves migratorias, que participan en el ciclo natural del virus. Las conexiones y los desplazamientos entre territorios también pueden afectar a la detección y al seguimiento de los casos, aunque la fiebre del Nilo occidental no se transmite normalmente de una persona a otra mediante el contacto cotidiano.

En 2023 se notificaron casos en más comunidades próximas a las zonas afectadas durante el año anterior y Extremadura se convirtió en la región con mayor número de registros. En 2024 dejaron de notificarse casos en algunas de las comunidades que habían aparecido en 2023 y la distribución quedó concentrada en Andalucía, Extremadura y Castilla-La Mancha. Dentro de este grupo, Extremadura continuó mostrando un impacto destacado [8, 28].

Una posible explicación de la situación de Extremadura es la existencia de amplias zonas agrícolas, humedales, sistemas de riego y otros espacios en los que puede acumularse agua estancada, lo que favorece la reproducción y supervivencia de los mosquitos. Además, las temperaturas elevadas pueden aumentar su actividad y prolongar la temporada en la que actúan y se mantienen activos. No obstante, la evolución de los casos también depende de las precipitaciones, los usos del suelo, las poblaciones de aves, la vigilancia sanitaria y otras condiciones ambientales [26, 28].

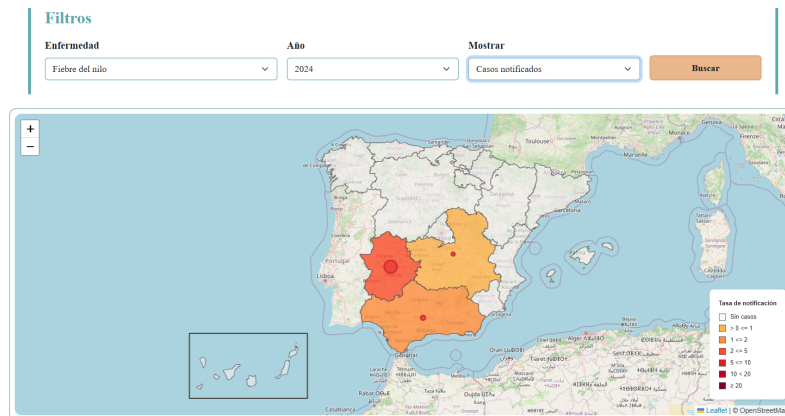


Figura 5.6: Distribución de los casos notificados de fiebre del Nilo occidental durante el año 2024.

5.3.2. Distribución por género

En los datos analizados, el género predominante es el masculino. Este resultado contrasta con los datos de dengue, en los que el género femenino predominaba en dos de los tres años estudiados, a pesar de que ambas enfermedades son transmitidas por mosquitos.

La mayor presencia de casos en hombres también se observa en los informes epidemiológicos nacionales, pero no puede explicarse simplemente afirmando que sus condiciones biológicas sean más débiles frente a la enfermedad. Esta diferencia podría estar relacionada con factores biológicos, laborales, ambientales o de exposición, así como con la edad y las patologías previas de las personas afectadas [8].

5.3.3. Distribución por edad

La base de datos utilizada en el proyecto no dispone de información suficiente para representar los casos de fiebre del Nilo occidental mediante los rangos de edad definidos en el mapa. El reducido número de registros de algunos años dificulta, además, la realización de comparaciones detalladas entre grupos.

No obstante, los informes epidemiológicos públicos sí incluyen datos generales sobre la edad de las personas afectadas. En 2024, por ejemplo, los casos autóctonos presentaron una mediana de edad de 66 años y se observó un mayor riesgo entre las personas de edad avanzada. Esta información no se ha incorporado al mapa porque no se encuentra organizada según los mismos intervalos de edad utilizados por la aplicación [8].

En conjunto, los datos muestran una expansión de los casos entre 2022 y 2023 y una fuerte concentración en Andalucía y Extremadura durante 2024. El comportamiento variable de la enfermedad demuestra la importancia de mantener la vigilancia de los mosquitos, las aves y los casos humanos, especialmente durante los meses cálidos y en las regiones con condiciones ambientales favorables de enfermedades como la fiebre del Nilo.

5.4. Análisis de la fiebre hemorrágica de Crimea-Congo

La fiebre hemorrágica de Crimea-Congo es una enfermedad vírica zoonótica transmitida principalmente por la picadura de garrapatas infectadas, especialmente las del género *Hyalomma*. En España se han notificado pocos casos humanos, aunque se trata de una enfermedad grave que requiere una vigilancia constante [5, 20].

5.4.1. Evolución de los casos notificados

Según los datos incorporados al mapa, en 2022 se notificaron dos casos en Castilla y León. En 2023 se registró un único caso en la misma comunidad autónoma. En 2024, la aplicación representa tres casos en toda España: uno en Castilla y León, otro en Castilla-La Mancha y otro en Andalucía. Por tanto, ese año aumentó en uno el número máximo de casos anuales incluidos en el estudio y la enfermedad apareció en varias comunidades autónomas [5, 20].

Aunque el número de casos humanos es reducido, no debe interpretarse que la enfermedad carezca de peligrosidad. La fiebre hemorrágica de Crimea-Congo puede producir cuadros graves y presentar una elevada letalidad. Además, la detección de casos en nuevas zonas hace necesario estudiar su evolución para evitar que pueda convertirse en un problema mayor y para mejorar las medidas de prevención, diagnóstico y tratamiento.

Los animales domésticos y silvestres pueden participar en el ciclo de transmisión al ser parasitados por garrapatas infectadas. Generalmente no desarrollan síntomas, pero pueden transportar garrapatas y mantener el virus temporalmente en su sangre, lo que supone un riesgo para las personas que trabajan con ellos o entran en contacto con sus tejidos [5, 20].

Una posible explicación de la presencia repetida de casos en el centro y oeste de la Península, especialmente en Castilla y León, es la importancia de la ganadería y de las actividades rurales en estos territorios. El contacto con ganado puede aumentar la exposición a garrapatas, pero no es el único factor de riesgo. También pueden producirse picaduras durante paseos por el campo, actividades de caza, trabajos agrícolas o el cuidado de huertos.

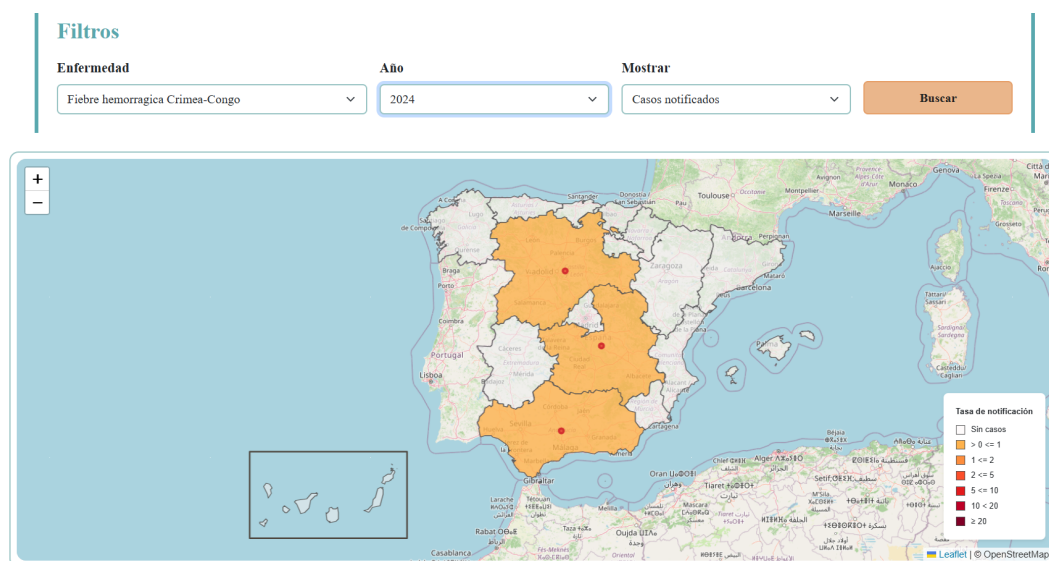


Figura 5.7: Distribución de los casos de fiebre hemorrágica de Crimea-Congo incluidos en el mapa durante el año 2024.

5.4.2. Distribución por género

En los tres años analizados, el género predominante es el masculino. Esta distribución podría estar relacionada con una mayor presencia de hombres en algunas actividades ganaderas, agrícolas, o forestales que implican una exposición frecuente al campo, a los animales y a las garrapatas.

5.4.3. Distribución por edad

La base de datos del proyecto no contiene información suficiente para clasificar los casos de esta enfermedad mediante los rangos de edad utilizados en el mapa. Esta ausencia puede deberse al reducido número de registros disponibles y a que los datos recopilados inicialmente no incluían esta variable con el nivel de detalle necesario.

No obstante, la información epidemiológica pública sí recoge la edad de los casos notificados. Entre 2016 y 2025, las personas afectadas tenían entre 29 y 80 años y la mediana de edad fue de 62 años. Estos datos no se han representado en la aplicación porque no se encuentran agrupados según los mismos intervalos de edad definidos para el resto de las enfermedades [5, 12].

En conjunto, la evolución observada muestra pocos casos, pero también una ampliación territorial durante 2024. El seguimiento de esta enfermedad resulta esencial debido a su gravedad y a la expansión de las garrapatas capaces de transmitirla.

5.5. Análisis del chikungunya

El chikungunya es una enfermedad vírica transmitida por mosquitos infectados del género *Aedes*. El análisis de los datos comprendidos entre 2022 y 2024 permite observar cambios importantes en el número de casos notificados y en su distribución entre las comunidades autónomas [7, 11, 15].

5.5.1. Evolución de los casos notificados

En 2022 se notificaron algunos casos en comunidades autónomas como Madrid, Cataluña, las Islas Baleares, Andalucía, el País Vasco y Comunidad Valenciana. La mayoría de estos territorios son costeros, con la excepción de Madrid. En las regiones con mar se encuentra mayor humedad que facilita la supervivencia y reproducción de los mosquitos. En el caso de Madrid, el número de habitantes, su condición de capital, la actividad turística y la elevada movilidad nacional e internacional podrían contribuir a que se notifiquen más casos importados. Estos mismos factores también pueden influir en otras comunidades con grandes ciudades y conexiones internacionales [1, 15].

En 2023 se produjo un aumento muy destacado y se notificaron casos en la mayoría de las comunidades autónomas. Las únicas que no registraron casos fueron Extremadura, La Rioja, Ceuta y Melilla. Por tanto, puede afirmarse que las notificaciones se extendieron por gran parte de España y que aumentó considerablemente el número de personas afectadas. Una parte importante de este aumento estuvo relacionada con personas que habían viajado o visitado a familiares en países de América Latina en los que circulaba el virus [11].

En 2024 se produjo un descenso hasta los 54 casos y se redujo el número de comunidades autónomas que notificaron la enfermedad. Cataluña, Madrid y Andalucía concentraron los valores más elevados. Esta reducción podría indicar una mejora de la situación respecto a 2023, pero no permite asegurar que el contagio se hubiera controlado únicamente gracias a las medidas aplicadas [7].

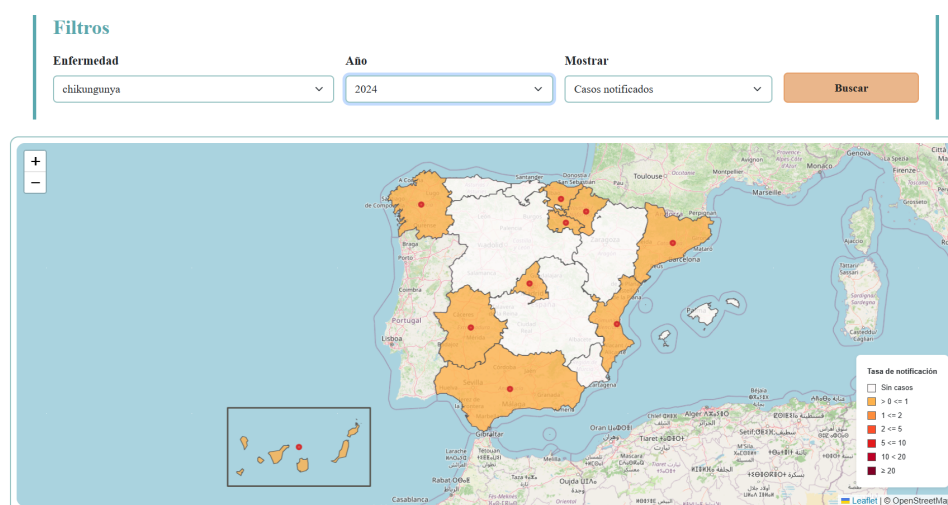


Figura 5.8: Distribución de los casos notificados de chikungunya durante el año 2024.

5.5.2. Distribución por género

En los datos incorporados al proyecto, el género predominante fue el masculino en 2022 y 2024, mientras que en 2023 predominó el femenino. Esta evolución presenta un comportamiento prácticamente inverso al observado en el dengue durante el mismo periodo.

Las posibles razones por las que un género aparece más afectado pueden ser similares a las planteadas para el dengue y la fiebre del Nilo occidental, ya que las tres enfermedades son transmitidas por mosquitos. Entre ellas se encuentran las diferencias en la movilidad, los viajes, las actividades al aire libre y determinadas características individuales que pueden influir en la atracción de los mosquitos. El cambio observado entre los tres años indica que no existe un patrón constante de predominio masculino o femenino.

5.5.3. Distribución por edad

En 2022 no se encontraron datos agrupados por rangos de edad que pudieran incorporarse al mapa. En 2023, el grupo con mayor número de casos fue el comprendido entre los 35 y los 45 años, mientras que en 2024 predominó el rango de 25 a 35 años. Estos resultados indican una mayor presencia de casos entre la población adulta joven durante los dos últimos años del estudio [7, 11].

Una posible explicación es que estos grupos presentan una mayor movilidad por motivos laborales, sociales, familiares o turísticos. También pueden realizar más actividades al aire libre o permanecer con mayor frecuencia cerca de jardines, piscinas y otros lugares en los que pueda acumularse agua. A diferencia de lo que podría pensarse, los mosquitos *Aedes* transmisores del chikungunya presentan una actividad principalmente diurna siendo sus horas de mayor actividad a primera hora de la mañana y al anochecer, por lo que la vida nocturna no explicaría por sí sola la distribución observada [19, 31].

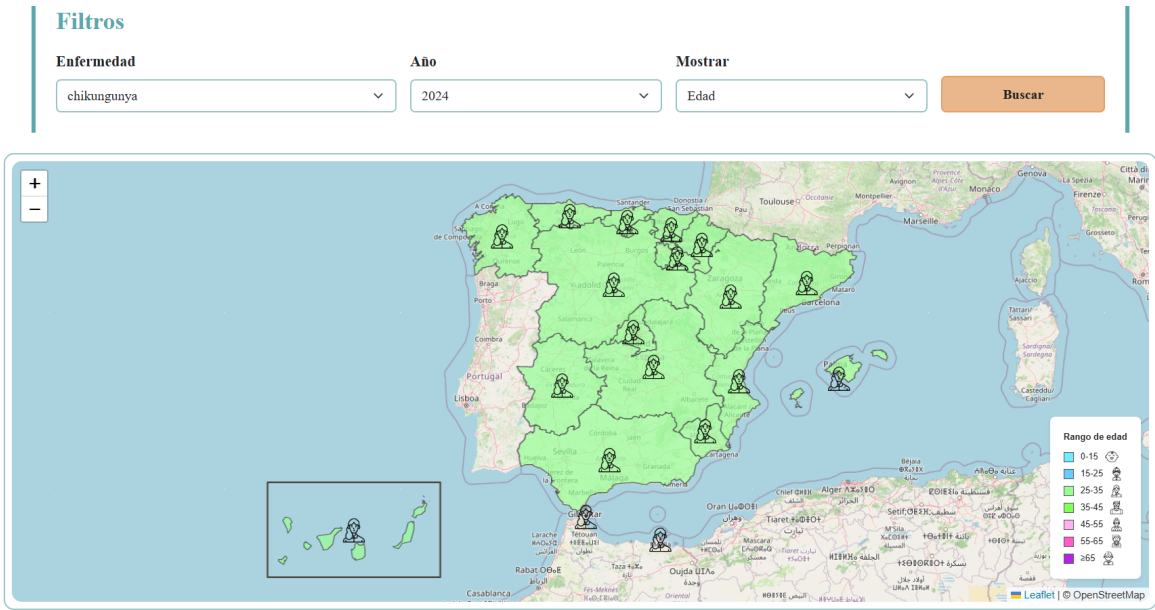


Figura 5.9: Rango de edad predominante en los casos de chikungunya durante el año 2024.

Este análisis muestra que el mapa puede interpretarse de forma sencilla incluso sin disponer de conocimientos avanzados sobre biología o transmisión de enfermedades. La visualización de la evolución temporal y de las zonas afectadas ayuda a identificar patrones, formular posibles explicaciones y comprender con mayor rapidez la información obtenida de otras fuentes. Por este motivo, el mapa constituye un instrumento útil para consultar datos y ampliar los conocimientos sobre las enfermedades emergentes y reemergentes en nuestro país.

Conclusiones y trabajo futuro

6.1. Conclusiones

El desarrollo de esta aplicación ha sido un proceso muy interesante y formativo. Me ha permitido ampliar mis conocimientos sobre los diferentes procesos necesarios para crear una página web interactiva y profesional. También he aprendido a utilizar tecnologías y herramientas con las que no había trabajado anteriormente, como Python, las API y los mapas interactivos, además de otros recursos que complementan el funcionamiento de un portal web.

Se han alcanzado los objetivos previstos para este proyecto y se ha obtenido una aplicación con un acabado cuidado y sencillo de utilizar. Al mismo tiempo, el resultado presenta un amplio margen de mejora y numerosas posibilidades de trabajo futuro. La aplicación cumple su finalidad como página web informativa e interactiva sobre las enfermedades emergentes y reemergentes en España y ofrece contenidos accesibles para un amplio sector de la población. No se necesita estudios o conocimientos previos para entender y estudiar estos datos y aplicarlos a nuestro día a día.

Al comienzo del proyecto no era plenamente consciente del alcance y la utilidad que podía llegar a tener. Sin embargo, a medida que fue tomando forma y se incorporaron nuevas funcionalidades, pude observar su potencial si se le dedicara más tiempo, recursos y presupuesto. Si el trabajo tuviera continuidad y se ampliara con más enfermedades, filtros, territorios y características, podría convertirse en una herramienta de consulta de gran utilidad para la población. Además, se trata de un ámbito que evoluciona constantemente, ya que pueden aparecer nuevas enfermedades capaces de representar un riesgo para la sociedad, independientemente del momento en el que nos encontremos.

6.2. Trabajo futuro

Este proyecto puede seguir tres caminos principales de mejora. El primero consistiría en extender el mapa más allá de las comunidades autónomas de España hasta abarcar todos los países de Europa. Este crecimiento no podría realizarse a la ligera, ya que, para que el atlas fuera lo más completo posible, cada país tendría que dividirse también en ciudades, regiones o estados, dependiendo de su organización territorial, del mismo modo que se hace con España. Si solo se representaran los países de Europa, se perderían muchos datos y matices que podrían resultar decisivos en la lucha contra las enfermedades emergentes y reemergentes. Lo más idóneo sería completar un país antes de pasar al siguiente hasta abarcar todo el continente. El objetivo más ambicioso sería que los habitantes de cualquier parte del mundo pudieran consultar su región y conocer qué enfermedades pueden poner en peligro sus vidas, cómo identificarlas y qué hacer en caso de contagio.

El segundo camino principal consistiría en aumentar el número de enfermedades, detalles y filtros disponibles en la página web. Este proyecto se ha realizado durante los años 2025 y 2026, pero las enfermedades emergentes y reemergentes continuarán apareciendo y desapareciendo en los distintos países del mundo [10, 21, 32]. Sería interesante incorporar al trabajo tanto enfermedades antiguas como nuevas y visualizarlas en el mapa para obtener todo tipo de datos. Además de añadir enfermedades para que el atlas fuera lo más completo posible, se podrían incluir muchos más detalles en las tarjetas: el animal o medio a través del cual se produce el contagio; el país de procedencia o el primero en detectar un caso; los grupos de personas más vulnerables según sus problemas de salud previos; el origen de la enfermedad; las causas que han provocado que vuelva a ser peligrosa para la sociedad; las mutaciones producidas a lo largo del tiempo; la existencia de vacunas o medicamentos capaces de erradicarla, o si la enfermedad ya no supone una amenaza, entre otros datos.

También se podría completar la información proporcionada por la página web mediante nuevos filtros y formas de visualización en el mapa. Podrían incorporarse datos como el número de fallecidos por cada enfermedad en cada comunidad autónoma, su género y rango de edad, o el mes en que se produjo cada caso, además del año, para disponer de una mayor precisión. A su vez, la incorporación de más filtros permitiría flexibilizar la descarga de archivos CSV y seleccionar datos adicionales, aunque no estuvieran representados en ese momento en el mapa.

Por último, el tercer camino que podría mejorar considerablemente el proyecto sería automatizarlo por completo. En el momento de realizar este trabajo no es posible automatizar todo el proceso, ya que los datos recopilados no proceden de una misma fuente y es necesario introducirlos uno a uno en un archivo CSV. Si una organización relacionada con la epidemiología publicara todos los datos necesarios en un mismo lugar y una API pudiera descargarlos y añadirlos automáticamente a archivos CSV, se facilitaría en gran medida el desarrollo de una aplicación más completa. De este modo, podrían incorporarse automáticamente nuevas enfermedades y detalles a medida que avanzaran los años.

Al tratarse de resultados médicos, los hospitales y las organizaciones sanitarias

suelen ser muy prudentes con sus datos y, con frecuencia, no los hacen públicos. Si el proyecto recibiera financiación y aumentara su alcance, se podría establecer una colaboración con alguna de estas instituciones, como el Instituto de Salud Carlos III, del que se han obtenido muchos de los datos utilizados. Esta sería una opción más realista. Una vez implementado un método para descargar y actualizar los datos automáticamente, la automatización del backend y del frontend no supondría un gran problema, siempre que todos los datos se proporcionaran en el mismo formato y las tarjetas y los filtros se generaran automáticamente con la información necesaria.

Además de estos tres caminos principales, existen muchas otras formas de ampliar el proyecto y aumentar su utilidad para la población. Entre ellas se encuentran el desarrollo de una interfaz aún más intuitiva y accesible, el tratamiento de otros temas relacionados con la salud, una versión simplificada y específica para dispositivos móviles, una mayor precisión geográfica de los casos mostrados en el mapa mediante la identificación de los hospitales en los que fueron notificados, el desarrollo de una aplicación móvil y la incorporación de elementos sonoros, entre otras posibilidades.

Además de ampliar los datos y las funcionalidades, también sería necesario mejorar algunos aspectos relacionados con la ingeniería de software. Una evolución importante consistiría en desplegar la aplicación en la nube y utilizar contenedores Docker para separar el backend, la base de datos y los procesos de actualización. Esto facilitaría la instalación, el mantenimiento y el acceso a la aplicación desde cualquier lugar.

También sería útil desarrollar un panel de administración independiente de la interfaz pública. Este permitiría que los usuarios autorizados pudieran añadir enfermedades, actualizar los datos epidemiológicos, modificar la información descriptiva y revisar los cambios antes de publicarlos, sin necesidad de trabajar directamente sobre los archivos o la base de datos. El acceso debería estar protegido mediante autenticación y distintos niveles de permisos.

Para soportar un mayor número de usuarios, habría que mejorar el rendimiento mediante una organización eficiente de la base de datos y, cuando fuera necesario, un sistema de caché para las consultas frecuentes. Asimismo, deberían incorporarse mecanismos de monitorización, protección de credenciales y pruebas automáticas. Estas mejoras permitirían transformar el proyecto a una versión mucho más profesional, estable y de mucha utilidad.

Conclusions and Future Work

6.3. Conclusions

Developing this application has been a highly engaging and educational process. It has allowed me to broaden my knowledge of the different stages involved in creating an interactive and professional website. I have also learned to use technologies and tools with which I had not previously worked, including Python, APIs, and interactive maps, together with other resources that support the operation of a web portal.

The objectives established for this project have been achieved, resulting in a polished and user-friendly application. At the same time, there is considerable room for improvement and a wide range of possibilities for future work. The application fulfills its purpose as an informative and interactive website about emerging and re-emerging diseases in Spain and offers accessible content to a broad audience. No previous training or specialist knowledge is required to understand and examine these data or relate them to everyday life.

At the beginning of the project, I was not fully aware of the scope and usefulness it could achieve. However, as it took shape and new features were incorporated, its potential became increasingly clear. With more time, resources, and funding, the project could be expanded to include additional diseases, filters, territories, and features, becoming a valuable source of information for the public. This field is also constantly evolving, as new diseases capable of posing a risk to society may emerge at any time.

6.4. Future Work

This project offers three main paths for future development. The first would be to extend the map beyond Spain's autonomous communities to include every European country. Such an expansion would need to be carried out gradually because, for the atlas to be as complete as possible, each country would also have to be divided into cities, regions, or states according to its territorial organization, as has been done for Spain. Representing countries alone would omit data and details that could be important in the fight against emerging and re-emerging diseases. The most appropriate approach would therefore be to complete one country before moving on to the next until the entire continent was covered. The most ambitious objective would be to allow people anywhere in the world to consult their region and learn

which diseases may endanger their lives, how to identify them, and what to do in the event of infection.

The second path would be to increase the number of diseases, details, and filters available on the website. This project was developed during 2025 and 2026, but emerging and re-emerging diseases will continue to appear and disappear in different countries [10, 21, 32]. Both older and newly emerging diseases could be incorporated into the study and represented on the map. The information cards could also include further details, such as the animal or medium through which transmission occurs; the country of origin or the first country to detect a case; the population groups most vulnerable because of pre-existing health conditions; the origin of the disease; the causes of its renewed danger to society; mutations over time; the availability of vaccines or medicines capable of eradicating it; and whether the disease no longer represents a threat.

The website could also offer additional filters and forms of map visualization. These might include the number of deaths associated with each disease in every autonomous community, their distribution by gender and age group, or the month in which each case occurred, in addition to the year. Additional filters would also make CSV downloads more flexible by allowing users to select information that was not being displayed on the map at that moment.

The third path would be to automate the project completely. At present, this is not feasible because the collected data do not come from a single source and must be entered individually into a CSV file. If an organization working in epidemiology published all the required data in one place, an API could download them and add them automatically to the CSV files. This would greatly facilitate the development of a more complete application and allow new diseases and information to be incorporated automatically over time.

Hospitals and healthcare organizations are understandably cautious with medical data and often do not make them public. If the project received funding and its scope increased, collaboration could be established with an institution such as the Carlos III Health Institute, from which much of the data used in this project were obtained. This would be a more realistic option. Once a reliable method for downloading and updating the data had been established, automating the backend and frontend would not pose a major problem, provided that all information followed a consistent format and that the cards and filters could be generated automatically.

Other possible improvements include creating an even more intuitive and accessible interface, covering additional health-related topics, developing a simplified version specifically for mobile devices, increasing the geographic precision of the cases shown on the map by identifying the hospitals where they were reported, creating a mobile application, and incorporating audio elements.

In addition to expanding the data and functionality, several software engineering improvements could be introduced. One important step would be to deploy the application in the cloud and use Docker containers to separate the backend, database, and update processes. This would simplify installation and maintenance

while making the application accessible from any location.

It would also be useful to develop an administration panel separate from the public interface. Authorized users could use it to add diseases, update epidemiological data, modify descriptive information, and review changes before publication without working directly with files or database tables. Access should be protected through authentication and different permission levels.

Supporting a larger number of users would require better performance through efficient database organization and, where appropriate, caching for frequently repeated queries. Monitoring, credential protection, and automated testing should also be incorporated. Together, these improvements would transform the project into a more professional, stable, and useful application.

Bibliografía

*Todo final puede ser un nuevo comienzo,
pero todo comienzo debería tener un final.*

Victoria Elizabeth Schwab

- [1] AMILLATEGUI-DOS-SANTOS, R., CANO PORTERO, R., SASTRE-GARCÍA, M., MARTÍN MESONERO, C. y SOLER-SONEIRA, M. Resultados de la vigilancia de las enfermedades transmisibles notificadas a la Red Nacional de Vigilancia Epidemiológica (RENAVE) en 2022. *Boletín Epidemiológico Semanal*, vol. 31(4), páginas 221–225, 2023. Disponible en <https://cne.isciii.es/documents/d/cne/ano-2022> (último acceso, 1 de septiembre de 2026).
- [2] ANDREU IVORRA, B., SÁNCHEZ-MIGALLÓN NARANJO, A. y CHIRLAQUE LÓPEZ, M. D. Vigilancia epidemiológica de tuberculosis. Años 2022 y 2023. Región de Murcia. *Informes Epidemiológicos*, 2025. Disponible en <https://sms.carm.es/ricsmur/bitstream/handle/123456789/19264/ie.2025.04.pdf?sequence=1&isAllowed=y> (último acceso, 1 de septiembre de 2026).
- [3] CENTRO NACIONAL DE EPIDEMIOLOGÍA. Informe epidemiológico sobre la situación de dengue en España. Año 2022. 2023. Disponible en https://cne.isciii.es/documents/d/cne/informe_renave_dengue-2022 (último acceso, 1 de septiembre de 2026).
- [4] CENTRO NACIONAL DE EPIDEMIOLOGÍA. Informe epidemiológico sobre la situación de dengue en España. Año 2023. 2024. Disponible en https://cne.isciii.es/documents/d/cne/informe_renave_dengue-2023 (último acceso, 1 de septiembre de 2026).
- [5] CENTRO NACIONAL DE EPIDEMIOLOGÍA. Informe epidemiológico sobre la situación de la fiebre hemorrágica de Crimea-Congo en España, 2016–2023. 2024. Disponible en https://cne.isciii.es/documents/d/cne/informe_renave_fhcc-2016-2023 (último acceso, 1 de septiembre de 2026).
- [6] CENTRO NACIONAL DE EPIDEMIOLOGÍA. Informe epidemiológico sobre la situación de dengue en España. Año 2024. 2025. Disponible en https://cne.isciii.es/documents/d/cne/informe_renave_dengue-2024 (último acceso, 1 de septiembre de 2026).
- [7] CENTRO NACIONAL DE EPIDEMIOLOGÍA. Informe epidemiológico sobre la situación de la enfermedad por virus chikungunya en España. Año 2024. 2025.

- Disponible en https://cne.isciii.es/documents/d/cne/informe_renave_chikungunya-2024 (último acceso, 1 de septiembre de 2026).
- [8] CENTRO NACIONAL DE EPIDEMIOLOGÍA. Informe epidemiológico sobre la situación de la fiebre del Nilo occidental en España. Año 2024. 2025. Disponible en https://cne.isciii.es/documents/d/cne/informe_renave_fno-2024 (último acceso, 1 de septiembre de 2026).
- [9] CENTRO NACIONAL DE EPIDEMIOLOGÍA. Informe preliminar de la vigilancia de la tuberculosis en España. Año 2024. 2025. Disponible en https://cne.isciii.es/documents/d/cne/renave_informe_vigilancia-tb-2024 (último acceso, 1 de septiembre de 2026).
- [10] CENTRO NACIONAL DE EPIDEMIOLOGÍA. Informe semanal de vigilancia epidemiológica en España. Número 37. Año 2025. 2025. Disponible en https://cne.isciii.es/documents/d/cne/is_n-37-20250923_web (último acceso, 1 de septiembre de 2026).
- [11] COMITÉ ASESOR DE VACUNAS DE LA ASOCIACIÓN ESPAÑOLA DE PEDIATRÍA. Dengue, zika y chikunguña en España, 2023. 2024. Disponible en <https://vacunasaep.org/profesionales/noticias/dengue-zika-y-chikunguna-en-espana-2023> (último acceso, 1 de septiembre de 2026).
- [12] DE FRANCISCO, P. España notifica tres casos de fiebre hemorrágica de Crimea-Congo en 2025. 2025. Disponible en <https://www.consalud.es/pacientes/infecciosas/espana-tres-casos-fiebre-hemorragica-crimea-congo.html> (último acceso, 1 de septiembre de 2026).
- [13] DÍAZ, A. Situación actual de la tuberculosis y la infección por VIH en España. 2025. Disponible en <https://www.uitb.cat/wp-content/uploads/2025/11/03-asuncion-diaz.pdf> (último acceso, 1 de septiembre de 2026).
- [14] ESPAI DE DADES DE L'ECONOMIA SOCIAL I SOLIDÀRIA. Mapa Espanya per CCAA: georef-spain-comunidad-autonoma.geojson. 2025. Disponible en <https://espaidedades.dadess.cat/dataset/ccaa/resource/28d9b826-abd7-4380-84cb-253c6afc4705> (último acceso, 1 de septiembre de 2026).
- [15] EUROPEAN CENTRE FOR DISEASE PREVENTION AND CONTROL. Chikungunya virus disease: Annual Epidemiological Report for 2022. 2024. Disponible en https://www.ecdc.europa.eu/sites/default/files/documents/CHIK_AER_2022_report.pdf (último acceso, 1 de septiembre de 2026).
- [16] FASTAPI. Bases de datos SQL (relacionales). s. f.. Disponible en <https://fastapi.tiangolo.com/es/tutorial/sql-databases/> (último acceso, 1 de septiembre de 2026).
- [17] FASTAPI. Primeros pasos. s. f.. Disponible en <https://fastapi.tiangolo.com/es/tutorial/first-steps/> (último acceso, 1 de septiembre de 2026).

- [18] FERNÁNDEZ JAUREGUI, A. Cómo crear una API en Python. s. f. Disponible en <https://anderfernandez.com/blog/como-crear-api-en-python/> (último acceso, 1 de septiembre de 2026).
- [19] GACETA DE SALUD. Situación actual de chikungunya en España: se detectan casos, pero importados. 2025. Disponible en <https://www.gacetadesalud.com/actualidad-gs/noticias/13519989/08/25/situacion-actual-de-chikungunya-en-espana-se-detectan-casos-pero-importados.html> (último acceso, 1 de septiembre de 2026).
- [20] HIGIENE AMBIENTAL. Fiebre hemorrágica de Crimea-Congo en España. 2025. Disponible en <https://higieneambiental.com/fiebre-hemorragica-de-crimea-congo> (último acceso, 1 de septiembre de 2026).
- [21] INSTITUTO EUROPEO DE FORMACIÓN EN SALUD. Enfermedades infecciosas reemergentes: causas y consecuencias. 2025. Disponible en <https://iefs.es/enfermedades-infecciosas-reemergentes/> (último acceso, 1 de septiembre de 2026).
- [22] LEAFLET. Leaflet: an open-source JavaScript library for mobile-friendly interactive maps. 2025. Disponible en <https://leafletjs.com/> (último acceso, 1 de septiembre de 2026).
- [23] LÓPEZ, P. ¿Qué es un archivo CSV y para qué sirve? 2020. Disponible en <https://www.geeknetic.es/Archivo-CSV/que-es-y-para-que-sirve> (último acceso, 1 de septiembre de 2026).
- [24] MASET, J. Dengue: transmisión, síntomas, tratamiento y prevención. 2024. Disponible en <https://cinfasalud.cinfa.com/p/dengue/> (último acceso, 1 de septiembre de 2026).
- [25] MEDLINEPLUS. Fiebre del dengue. 2024. Disponible en <https://medlineplus.gov/spanish/ency/article/001374.htm> (último acceso, 1 de septiembre de 2026).
- [26] MINISTERIO DE AGRICULTURA, PESCA Y ALIMENTACIÓN. Actualización de la situación epidemiológica de la fiebre del Nilo occidental (West Nile Fever). 18 de septiembre de 2025. 2025. Disponible en <https://www.mapa.gob.es/es/dam/jcr:b36e3d65-735c-4186-a009-cae61b42d3e3/Informe%20FN0%20septiembre%202025%20.pdf> (último acceso, 1 de septiembre de 2026).
- [27] MINISTERIO DE SANIDAD. Indicadores de seguimiento del Plan para la prevención y control de la tuberculosis en España. Año 2022. 2023. Disponible en https://www.sanidad.gob.es/profesionales/saludPublica/prevPromocion/PlanTuberculosis/docs/Resumen_PlanTB2022_SEPTIEMBRE_2023.pdf (último acceso, 1 de septiembre de 2026).
- [28] MINISTERIO DE SANIDAD. Evaluación rápida de riesgo sobre la fiebre del Nilo occidental en España. Temporada 2023. 2024. Disponible en <https://www.sanidad.gob.es/areas/alertasEmergenciasSanitarias/alertasActuale>

- s/meningoencefalitis/docs/20240118_ERR_Nilo_Occidental_2023.pdf (último acceso, 1 de septiembre de 2026).
- [29] MINISTERIO DE SANIDAD. Indicadores de seguimiento del Plan para la prevención y control de la tuberculosis en España. Año 2023. 2024. Disponible en https://www.sanidad.gob.es/areas/DCVIHT/tuberculosis/docs/IND_SEG_PLAN_TB_2023_SEPTIEMBRE_2024_FINAL.pdf (último acceso, 1 de septiembre de 2026).
- [30] MINISTERIO DE SANIDAD. Evaluaciones de riesgo y seguimiento de enfermedades transmitidas por vectores. 2026. Disponible en <https://www.sanidad.gob.es/areas/alertasEmergenciasSanitarias/preparacionRespuesta/evaluacionesRiesgoVectores.htm> (último acceso, 1 de septiembre de 2026).
- [31] SCIENCE MEDIA CENTRE ESPAÑA. La chikunguña y el dengue aumentan en Europa: qué sabemos del riesgo de enfermedades transmitidas por mosquitos. 2025. Disponible en <https://sciencemediacentre.es/la-chikunguna-y-el-dengue-aumentan-en-europa-que-sabemos-del-riesgo-de-enfermedades-transmitidas> (último acceso, 1 de septiembre de 2026).
- [32] UNIVERSIDAD EUROPEA. Importancia de la investigación clínica frente a las enfermedades emergentes. 2022. Disponible en <https://universidadeuropea.com/blog/enfermedades-emergentes/> (último acceso, 1 de septiembre de 2026).

Manual de instalación

En esta sección se proporcionan, paso a paso, las instrucciones necesarias para instalar y probar el proyecto. Los pasos deben seguirse en el orden indicado para configurar correctamente la aplicación.

Para configurar e instalar el proyecto se requieren los siguientes programas y herramientas. La Tabla A.1 recoge las versiones con las que se ha probado correctamente la aplicación.

Componente	Versión probada
Sistema operativo	Windows 11 25H2 (compilación 26200.9168)
Google Chrome	152.0.7977.75, compilación oficial de 64 bits
XAMPP y MariaDB	10.4.32-MariaDB
Python	3.14.6
FastAPI	0.139.2
R y RStudio	R 4.4.2 (2024-10-31 ucrt)
Visual Studio Code y Live Server	1.135.0 (x64) Revisión: 08d4889f9ec4a1685d257b9b95de036c8e1ce1e5

Tabla A.1: Versiones utilizadas durante las pruebas de la aplicación.

A.1. Ubicación y estructura del proyecto

El código del proyecto está disponible en el siguiente repositorio de GitHub:

<https://github.com/Solrak12/TFG.git>

Una vez descargado o clonado el repositorio en el equipo, los comandos de este manual deben ejecutarse desde la carpeta local que se indique en cada paso. El proyecto se divide en tres directorios principales: **backend**, que contiene la API y el acceso a la base de datos; **datos**, que almacena los archivos CSV y los scripts de R; y **frontend**, que contiene las páginas HTML, los estilos, los archivos JavaScript y los demás recursos visuales.

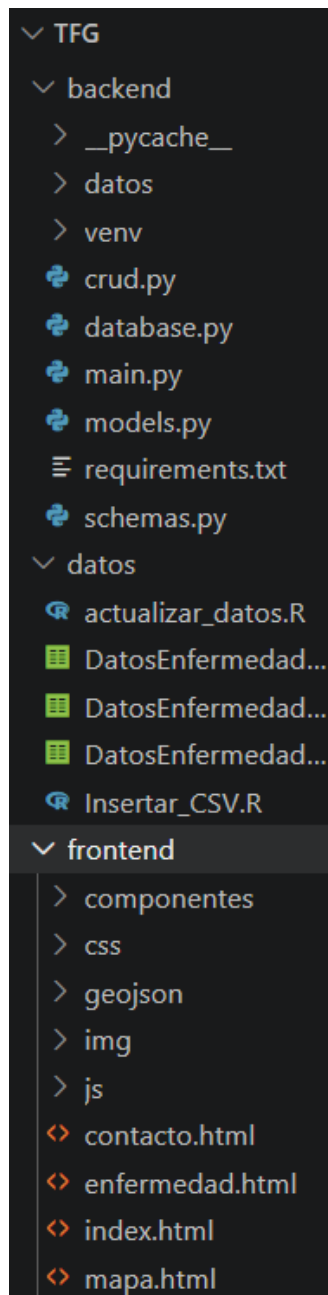


Figura A.1: Estructura de carpetas y archivos principales del proyecto.

A.2. Pasos de instalación

1. Preparar el archivo de datos inicial.

El punto de partida es el archivo `DatosEnfermedades.csv`, que contiene los datos epidemiológicos recopilados y organizados que posteriormente se incorporarán a la base de datos.

2. Actualizar las tasas de notificación.

Una vez localizado el archivo `DatosEnfermedades.csv`, se debe ejecutar el script `actualizar_datos.r` en RStudio que nos ayuda a programar en R.

Este script actualiza las tasas de notificación utilizando los datos del padrón y genera un nuevo archivo denominado `DatosEnfermedades_actualizado.csv`. Este será el archivo que se insertará en la base de datos.

```

R Console
> setwd("C:/Users/mayor/Documents/Universidad/TFG")
> source("actualizar_datos.r")

Adjuntando el paquete: 'dplyr'

The following objects are masked from 'package:stats':

  filter, lag

The following objects are masked from 'package:base':

  intersect, setdiff, setequal, union

'curl' package not installed, falling back to using 'url()'
Duplicados: ninguno

Filas sin población: 0

Archivo actualizado correctamente.
Avisos:
1: package 'dplyr' was built under R version 4.4.3
2: package 'stringi' was built under R version 4.4.3
3: package 'stringr' was built under R version 4.4.3
4: package 'readr' was built under R version 4.4.3
>

```

Figura A.2: Ejecución correcta del script `actualizar_datos.r` en R.

Es importante que estés en la dirección donde se encuentra el fichero `.r`.

3. Crear la base de datos y sus tablas.

La base de datos debe denominarse `enfermedades_emergentes`. Para crearla, se debe abrir phpMyAdmin desde XAMPP, seleccionar la pestaña *SQL* y ejecutar el siguiente código:

```

CREATE DATABASE enfermedades_emergentes
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

```

La aplicación utiliza SQLAlchemy para establecer la conexión entre el backend desarrollado con FastAPI y la base de datos MySQL. Los parámetros de conexión se definen en el archivo `database.py` mediante la variable `DATABASE_URL`. La configuración necesaria para instalar el proyecto se muestra en la Tabla A.2.

Parámetro	Valor
Sistema gestor de bases de datos	MySQL mediante MariaDB
Usuario	root
Contraseña	Vacía
Servidor	localhost
Puerto	3306
Base de datos	enfermedades_emergentes
Driver	PyMySQL
ORM	SQLAlchemy

Tabla A.2: Configuración de la conexión con la base de datos.

Con estos parámetros, la variable de conexión se configura de la siguiente forma:

```
DATABASE_URL = "mysql+pymysql://root:@localhost:3306/
enfermedades_emergentes"
```

Una vez creada la base de datos, se debe seleccionar y ejecutar el siguiente código para crear la tabla `enfermedades`:

```
CREATE TABLE enfermedades (
    id INT(11) NOT NULL AUTO_INCREMENT,
    `Enfermedad` VARCHAR(150) NOT NULL,
    `Año` INT(4) NOT NULL,
    `Comunidad` VARCHAR(100) NOT NULL,
    `Casos` INT(11) DEFAULT NULL,
    `Tasa notificacion` FLOAT DEFAULT NULL,
    `Hombres` INT(11) DEFAULT NULL,
    `Mujeres` INT(11) DEFAULT NULL,
    `Rango edad` VARCHAR(100) DEFAULT NULL,
    `Casos edad` INT(11) DEFAULT NULL,
    PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

A continuación, se debe crear la tabla `mensajes` ejecutando el siguiente código en la misma pestaña:

```
CREATE TABLE mensajes (
    id INT(11) NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(150) NOT NULL,
    email VARCHAR(150) NOT NULL,
    asunto VARCHAR(200) NOT NULL,
    mensaje TEXT NOT NULL,
    fecha DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Los nombres de la base de datos, las tablas y los campos deben coincidir exactamente con los indicados para evitar errores durante el proceso.

4. Insertar los datos epidemiológicos.

Los datos del archivo `DatosEnfermedades_actualizado.csv` pueden incorporarse a la tabla `enfermedades` de dos formas:

- **Insertión manual:** mediante consultas en la pestaña *SQL* de phpMyAdmin o introduciendo los registros uno a uno desde la pestaña *Insertar*.

phpMyAdmin

Reciente Favoritas

Nueva

enfermedades_emergentes

Nueva

enfermedades

mensajes

information_schema

mysql

performance_schema

phpmyadmin

test

Servidor: 127.0.0.1 Base de datos: enfermedades_emergentes Tabla: enfermedades

Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones

Columna	Tipo	Función	Nulo	Valor
id	int(11)		☐	
Enfermedad	varchar(32)		☒	
Año	double		☒	
Comunidad	varchar(18)		☒	
Casos	int(11)		☒	
Tasa notificación	double		☒	
Hombres	varchar(4)		☒	
Mujeres	varchar(4)		☒	
Rango edad	varchar(5)		☒	
Casos edad	varchar(4)		☒	

Continuar

Figura A.3: Formulario de inserción manual de un registro en la tabla enfermedades mediante phpMyAdmin.

- **Inserción automática:** mediante la ejecución del script `insertar_CSV.r` en RStudio, que carga automáticamente en la base de datos los registros del archivo CSV actualizado.

phpMyAdmin

Reciente Favoritas

Nueva

enfermedades_emergentes

Nueva

enfermedades

mensajes

information_schema

mysql

performance_schema

phpmyadmin

test

Servidor: 127.0.0.1 Base de datos: enfermedades_emergentes Tabla: enfermedades

Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones Seguimiento Disparadores

1 > >> Mostrar todo Número de filas: 25 Filtrar filas Buscar en esta tabla Ordenar según la clave: Ninguna

Opciones extra

	id	Enfermedad	Año	Comunidad	Casos	Tasa notificación	Hombres	Mujeres	Rango edad	Casos edad
☐ Editar ☒ Copiar ☒ Borrar	1	Dengue	2022	Andalucía	36	0.423				
☐ Editar ☒ Copiar ☒ Borrar	2	Dengue	2023	Andalucía	47	0.548				
☐ Editar ☒ Copiar ☒ Borrar	3	Dengue	2024	Andalucía	87	1.008				
☐ Editar ☒ Copiar ☒ Borrar	4	Dengue	2022	Aragón	7	0.527				
☐ Editar ☒ Copiar ☒ Borrar	5	Dengue	2023	Aragón	6	0.447				
☐ Editar ☒ Copiar ☒ Borrar	6	Dengue	2024	Aragón	22	1.828				
☐ Editar ☒ Copiar ☒ Borrar	7	Dengue	2022	Asturias	1	0.1				
☐ Editar ☒ Copiar ☒ Borrar	8	Dengue	2023	Asturias	12	1.193				
☐ Editar ☒ Copiar ☒ Borrar	9	Dengue	2024	Asturias	7	0.693				
☐ Editar ☒ Copiar ☒ Borrar	10	Dengue	2022	Baleares	9	0.758				
☐ Editar ☒ Copiar ☒ Borrar	11	Dengue	2023	Baleares	9	0.744				
☐ Editar ☒ Copiar ☒ Borrar	12	Dengue	2024	Baleares	38	3.085				
☐ Editar ☒ Copiar ☒ Borrar	13	Dengue	2022	Canarias	32	1.464				
☐ Editar ☒ Copiar ☒ Borrar	14	Dengue	2023	Canarias	29	1.31				
☐ Editar ☒ Copiar ☒ Borrar	15	Dengue	2024	Canarias	42	1.876				
☐ Editar ☒ Copiar ☒ Borrar	16	Dengue	2022	Cantabria	6	1.025				
☐ Editar ☒ Copiar ☒ Borrar	17	Dengue	2023	Cantabria	5	0.85				
☐ Editar ☒ Copiar ☒ Borrar	18	Dengue	2024	Cantabria	11	1.862				
☐ Editar ☒ Copiar ☒ Borrar	19	Dengue	2022	Castilla y León	15	0.631				
☐ Editar ☒ Copiar ☒ Borrar	20	Dengue	2023	Castilla y León	13	0.545				
☐ Editar ☒ Copiar ☒ Borrar	21	Dengue	2024	Castilla y León	22	0.92				

Figura A.4: Registros epidemiológicos almacenados en la tabla enfermedades.

5. Iniciar los servicios de XAMPP.

Una vez insertados los datos, se debe abrir el panel de control de XAMPP e iniciar los servicios Apache y MySQL. Ambos deben permanecer activos mientras se utiliza la aplicación.

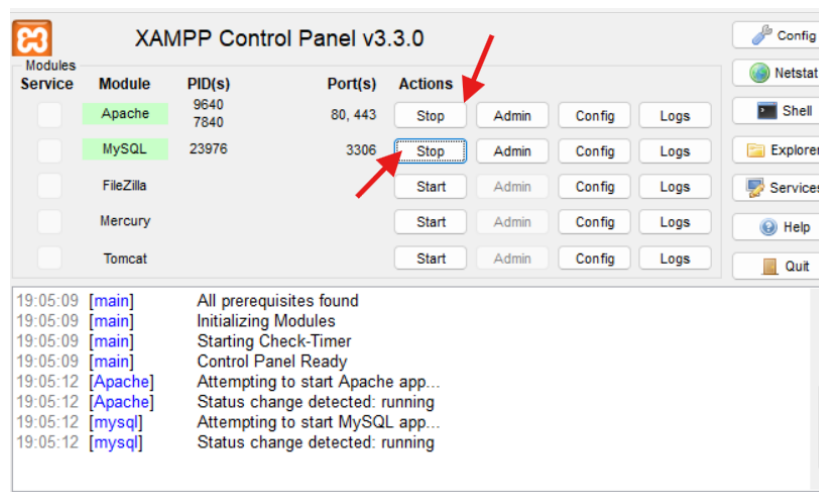


Figura A.5: Servicios Apache y MySQL iniciados en el panel de control de XAMPP.

6. Iniciar el backend.

Antes de iniciar el backend, se debe preparar el entorno virtual de Python. La carpeta `venv` queda asociada a la ubicación en la que se creó, por lo que no debe reutilizarse si el proyecto se ha trasladado a otro directorio o equipo. Si ya existe una carpeta `venv` creada en otra ubicación, se debe eliminar y generar de nuevo dentro de la carpeta `backend`.

En primer lugar, se debe abrir una terminal en la carpeta del backend, es decir, en el directorio que contiene archivos como `main.py`, `crud.py`, `database.py` y `requirements.txt`. A continuación, se crea el entorno virtual mediante el siguiente comando:

```
python -m venv venv
```

Este comando crea una nueva carpeta `venv` dentro del directorio del backend. En Windows, el entorno virtual se activa ejecutando:

```
venv\Scripts\activate
```

Una vez activado el entorno, se deben instalar las dependencias especificadas en el archivo `requirements.txt`:

```
pip install -r requirements.txt
```

Una vez instaladas las dependencias y todos los pasos anteriores, se puede iniciar el backend ejecutando:

```
uvicorn main:app --reload
```

Si el servidor se inicia correctamente, la terminal mostrará la dirección en la que se está ejecutando la API y varios mensajes de confirmación, como se observa en la siguiente figura:

```
(venv) PS C:\Users\mayor\Documents\Universidad\TFG\backend> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\Users\mayor\Documents\Universidad\TFG\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [26216] using StatReload
INFO: Started server process [22272]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Figura A.6: Ejecución del servidor de desarrollo mediante Uvicorn.

7. Abrir la aplicación web.

Para abrir el frontend desde Visual Studio Code, se debe seleccionar el archivo `index.html`, pulsar sobre él con el botón derecho y elegir la opción *Open with Live Server*.

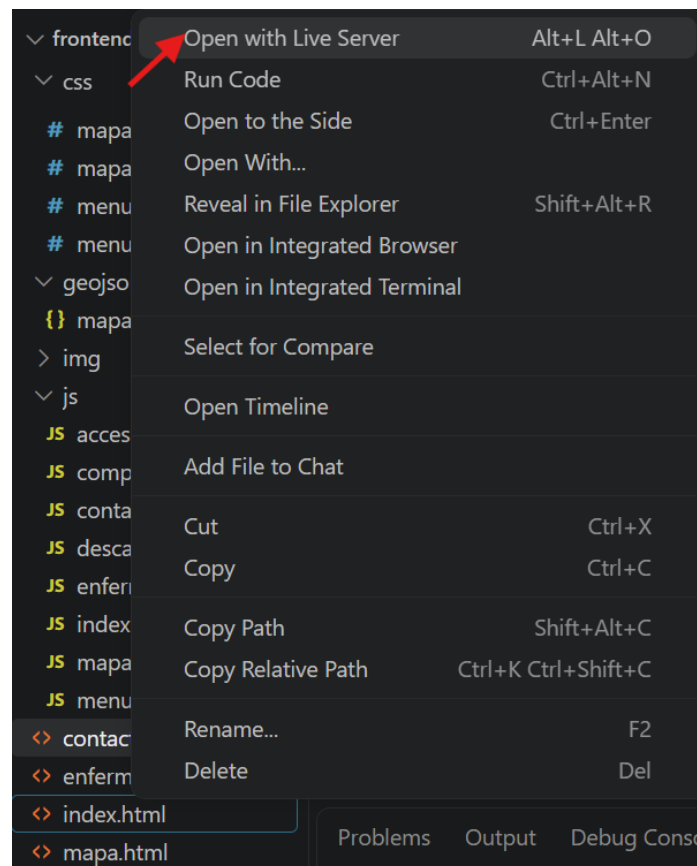


Figura A.7: Apertura del frontend mediante la extensión Live Server de Visual Studio Code.

Para utilizar este procedimiento, es necesario instalar previamente la extensión Live Server en Visual Studio Code. Una vez abierta la aplicación en el navegador, se podrá acceder a sus diferentes apartados, utilizar el menú de navegación y consultar el mapa epidemiológico interactivo.



Figura A.8: Página principal de la aplicación abierta en el navegador.

A.3. Comprobación de la instalación

La instalación se habrá completado correctamente si los servicios Apache y MySQL aparecen iniciados en XAMPP, la terminal muestra que Uvicorn se está ejecutando sin errores y la aplicación se abre desde Live Server en Google Chrome. También se debe comprobar que las tarjetas de las enfermedades cargan correctamente, que el mapa responde a los filtros y que el formulario de contacto muestra el mensaje correspondiente después de realizar un envío.

Además, se puede acceder a la documentación automática de la API desde la dirección `http://127.0.0.1:8000/docs`. Si esta página muestra los endpoints disponibles y permite realizar consultas, la conexión con el backend se ha iniciado correctamente.